Doku

Application de caisse et de gestion commerciale
pensée pour fonctionner sans Internet (offline-first)

Dossier de modélisation UML et Merise

Analyse → UML / Merise → Base de données → Django → API REST → Next.js / PWA

VERSION	1.0 : document de conception pour validation
DATE	4 octobre 2026
PÉRIMÈTRE	MVP : caisse, produits, stock, ventes CDF/USD, reçus WhatsApp, hors ligne, synchronisation
PILE CIBLE	PostgreSQL · Django REST Framework (JWT) · Next.js / React (PWA, IndexedDB)

Objectif : permettre à chaque commerçant de connaître ses ventes, son stock et son bénéfice sans cahier ni calculatrice.

Sommaire

Synthèse des décisions structurantes . . . 7

1. Présentation de Doku . . . 8

1.1 Contexte et vision . . . 8
1.2 Problèmes à résoudre et réponses de Doku . . . 8
1.3 Périmètre du MVP . . . 8
1.4 Parties prenantes et utilisateurs . . . 9
1.5 Contraintes de conception . . . 9
1.6 Risques majeurs . . . 10

2. Analyse fonctionnelle . . . 11

2.1 Démarche et traçabilité . . . 11
2.2 Besoins fonctionnels . . . 11
2.3 Besoins non fonctionnels . . . 12
2.4 Règles de gestion . . . 12
2.5 Informations manquantes et hypothèses . . . 13

3. Acteurs . . . 15

3.1 Définition et méthode d'analyse . . . 15
3.2 Acteur principal : le commerçant . . . 15
3.3 Acteur secondaire : WhatsApp . . . 15
3.4 Analyse des acteurs candidats . . . 15

4. Diagramme de cas d'utilisation . . . 16

4.1 Liste des cas d'utilisation . . . 16
4.2 Relations UML utilisées . . . 17
4.3 Cas d'utilisation ajoutés ou précisés . . . 17
4.4 Lecture des diagrammes . . . 17
4.5 Explication des vues . . . 25

5. Diagrammes d'activités . . . 26

5.1 Rôle des diagrammes et conventions de lecture . . . 26
5.2 Activité 1 : enregistrement d'une vente . . . 26
5.3 Activité 2 : fonctionnement hors ligne . . . 28
5.4 Activité 3 : synchronisation . . . 30
5.5 Activité 4 : gestion du stock . . . 33
5.6 Activité 5 : envoi du reçu . . . 34

6. Diagrammes de séquence . . . 37

6.1 Rôle des diagrammes et conventions de lecture . . . 37
6.2 Séquence 1 : enregistrer une vente . . . 37
6.3 Séquence 2 : vente hors ligne . . . 39
6.4 Séquence 3 : synchronisation . . . 41
6.5 Séquence 4 : authentification . . . 43
6.6 Séquence 5 : envoi d'un reçu via WhatsApp . . . 47

7. Diagramme de classes UML . . . 50

7.1 Rôle et conventions . . . 50
7.2 Lecture et choix de conception . . . 51
7.3 Opérations principales et services applicatifs . . . 52

8. MCD Merise : modèle conceptuel de données . . . 53

8.1 Démarche et lecture . . . 53
8.2 Analyse des entités candidates . . . 53
8.3 Entités retenues . . . 54
8.4 Associations et cardinalités . . . 57
8.5 Données calculables exclues du MCD . . . 58
8.6 Diagrammes MCD . . . 58
8.7 Explication du modèle . . . 64

9. MLD Merise : modèle logique de données . . . 65

9.1 Règles de transformation MCD → MLD . . . 65
9.2 Schéma relationnel (notation Merise) . . . 65
9.3 Spécification des tables . . . 67
9.4 Compatibilité PostgreSQL et MySQL . . . 69
9.5 Vérification de la normalisation . . . 70
9.6 Redondances et instantanés assumés . . . 70
9.7 Colonnes techniques ajoutées au niveau logique . . . 70

10. Dictionnaire de données . . . 71

10.1 currency (Devise) . . . 71
10.2 app_user (Utilisateur) . . . 71
10.3 shop (Boutique) . . . 72
10.4 category (Catégorie) . . . 72
10.5 product (Produit) . . . 73
10.6 exchange_rate (Taux de change) . . . 73
10.7 sale (Vente) . . . 74

10.8 sale_item (Ligne de vente) . . . 74
10.9 stock_movement (Mouvement de stock) . . . 75
10.10 sync_session (Synchronisation) . . . 75

11. Gestion du stock . . . 76

11.1 Principe : un journal, pas un compteur . . . 76
11.2 Types de mouvements . . . 76
11.3 Exemple chiffré : le journal d'un produit . . . 76
11.4 Répondre aux questions du commerçant . . . 76
11.5 Intégrité et concurrence . . . 77
11.6 Stock insuffisant et stock négatif . . . 77
11.7 Alerte de stock faible . . . 77

12. Gestion des ventes . . . 79

12.1 Structure d'une vente . . . 79
12.2 Cycle de vie d'une vente . . . 79
12.3 Calcul d'une vente . . . 79
12.4 Bénéfice : formules et exemple . . . 80
12.5 Reçu et numéro de reçu . . . 80
12.6 Statistiques : un calcul, deux endroits . . . 81

13. Gestion des devises CDF et USD . . . 82

13.1 Pourquoi une simple conversion automatique ne suffit pas . . . 82
13.2 Devises prises en charge . . . 82
13.3 Le taux de change . . . 82
13.4 Conversion et arrondi . . . 82
13.5 Ce qui est conservé : montant, devise, taux, montant converti . . . 83
13.6 Garantie d'immuabilité : démonstration . . . 83
13.7 Cas limites et règles d'interface . . . 83

14. Fonctionnement offline-first . . . 84

14.1 Principes . . . 84
14.2 Architecture technique cible . . . 84
14.3 Base locale (IndexedDB) . . . 86
14.4 Détection de la connectivité . . . 86
14.5 Déclencheurs de synchronisation . . . 87
14.6 Persistance du stockage et risque de perte . . . 87
14.7 Ce qui fonctionne sans Internet . . . 88
14.8 Limites connues . . . 88

15. Stratégie de synchronisation . . . 89

15.1 Choix de conception . . . 89
15.2 Périmètre : que synchronise-t-on, dans quel sens, avec quelle règle ? . . . 89
15.3 Protocole d'envoi (push) . . . 89
15.4 Protocole de réception (pull) . . . 90
15.5 Idempotence . . . 90
15.6 Types de conflits et résolutions . . . 91
15.7 Le stock courant côté client . . . 91
15.8 Erreurs et nouvelles tentatives . . . 91
15.9 États d'un enregistrement local . . . 91
15.10 Journal de synchronisation (serveur) . . . 92
15.11 Plan de test de la synchronisation . . . 92

16. Gestion des identifiants UUID . . . 93

16.1 Pourquoi des UUID . . . 93
16.2 Décision entité par entité . . . 93
16.3 Version, génération et format . . . 93
16.4 Sécurité des identifiants fournis par le client . . . 93
16.5 Numéro de reçu : lisible, sans compteur . . . 94

17. Principes de sécurité . . . 95

17.1 Actifs à protéger et menaces . . . 95
17.2 Authentification . . . 95
17.3 Autorisation et isolation par boutique . . . 95
17.4 Validation des données . . . 96
17.5 Injections, XSS et CSRF . . . 96
17.6 CORS, transport et configuration . . . 97
17.7 Protection des données commerciales et personnelles . . . 97
17.8 Gestion des erreurs . . . 97
17.9 Données stockées sur le téléphone . . . 97

18. Correspondance avec Django . . . 99

18.1 Organisation en applications . . . 99
18.2 Table → modèle → ressource API . . . 99
18.3 Règles de nommage entre couches . . . 99
18.4 Modèles . . . 100
18.5 Services métier (couche applicative) . . . 103
18.6 Réglages de sécurité et vues . . . 104
18.7 Dépendances proposées . . . 105

19. Correspondance avec l'API REST . . . 106

19.1 Conventions . . . 106

19.2 Verbes HTTP . . . 106

19.3 Catalogue des routes . . . 106

19.4 Exemples . . . 107

20. Vérification de cohérence globale . . . 109

20.1 Méthode . . . 109

20.2 Contrôles automatiques . . . 109

20.3 Matrices de traçabilité . . . 109

20.4 Points d'attention relevés . . . 112

21. Conclusion et étapes suivantes . . . 113

21.1 Feuille de route proposée . . . 113

Annexe A : glossaire . . . 114

Annexe B : MPD, script SQL de référence . . . 115

Annexe C : références . . . 118

Synthèse des décisions structurantes

Ce dossier transforme la description de Doku en une modélisation cohérente, de l'analyse jusqu'à l'API. Voici, en une page, les douze décisions qui structurent tout le reste. Chacune est justifiée dans le chapitre indiqué ; les points à valider par le porteur du projet sont regroupés au chapitre 2 (hypothèses H-01 à H-19).

#	Décision	Chap.
D1	**Offline-first** : le client écrit toujours d'abord dans une base locale (IndexedDB) ; la synchronisation est asynchrone, idempotente et rejouable.	14, 15
D2	**UUID v4 générés côté client** pour toutes les données créées hors ligne ; UUID aussi pour utilisateur et boutique (sécurité, uniformité) ; la devise garde sa clé naturelle ISO.	16
D3	**Stock par journal** (ledger) : mouvements en ajout seul ; *current_stock* est une colonne dénormalisée calculée par le serveur ; on synchronise des variations, jamais des valeurs.	11, 15
D4	**Ventes immuables** : jamais de suppression ; une annulation change le statut et ajoute des mouvements compensatoires.	12
D5	**Instantanés financiers** : la ligne de vente fige désignation, prix et coût ; la vente référence un taux de change immuable. Les anciens bénéfices ne bougent plus.	12, 13
D6	**Bi-devise** : une vente = une devise ; conversion et arrondi à la valorisation ; devise de référence de la boutique pour les statistiques ; montant converti non stocké (dérivable).	13
D7	**Entités non retenues en MVP** : Commerçant (fusionné dans Utilisateur), Stock (dérivé), Reçu (document dérivé), Paiement, Client. Chacune a une justification et un chemin d'évolution.	8
D8	**Sécurité** : JWT à access court + refresh rotatif en cookie HttpOnly ; isolation par boutique imposée côté serveur ; aucune confiance dans les identifiants envoyés par le client.	17
D9	**Conflits** : données en ajout seul sans conflit ; modifiables en « dernière modification gagnante » avec numéro de version ; stock négatif toléré et signalé.	15
D10	**WhatsApp** : partage par lien wa.me ou feuille de partage (MVP) ; l'API WhatsApp Business est une évolution payante qui exige le réseau.	6, 14
D11	**PostgreSQL** recommandé (CHECK, index partiels, JSONB) ; le MLD reste neutre et les écarts MySQL sont listés.	9
D12	**Calculs uniques** : mêmes règles de calcul (total, coût, bénéfice, arrondi) côté client et serveur, validées par un jeu de tests commun.	12

Tableau 0.1 : Les douze décisions structurantes de la modélisation

Comment lire ce document

Conventions de ce document. Les analyses (UML, Merise) sont en français ; les identifiants techniques (tables, colonnes, modèles Django, routes et champs JSON) sont en anglais *snake_case* pour rester identiques de la base jusqu'au frontend. Les encadrés signalent : **hypothèse**, **décision**, **attention**, **bonne pratique**, **erreur fréquente**.

1. Présentation de Doku

1.1 Contexte et vision

Beaucoup de petits commerçants gèrent leur activité avec un cahier et une calculatrice. Cette méthode fonctionne tant que l'activité reste petite, mais elle produit des erreurs de calcul, des oublis d'enregistrement, des ruptures de stock imprévues et aucune vision fiable du bénéfice. Dans une économie où les prix et les encaissements se font en deux monnaies (franc congolais, CDF, et dollar américain, USD) et où la connexion Internet est intermittente, une application de caisse classique, qui dépend du réseau, n'est pas adaptée.

Doku est une application de caisse et de gestion commerciale conçue d'abord pour smartphone. Elle permet au commerçant d'enregistrer ses ventes, de gérer ses produits, de suivre son stock en temps réel, de connaître son chiffre d'affaires et son bénéfice, de travailler en CDF et en USD, de produire des reçus et de les envoyer par WhatsApp, **même sans connexion Internet**, les données étant synchronisées au retour du réseau.

> **Vision**
> Objectif principal du produit : **permettre à chaque commerçant de connaître ses ventes, son stock et son bénéfice sans cahier ni calculatrice.** Toute fonctionnalité, tout champ et toute table de ce dossier doit pouvoir être rattaché à cet objectif.

1.2 Problèmes à résoudre et réponses de Doku

Problème rencontré	Réponse de Doku	Éléments du modèle
Erreurs de calcul	Total calculé automatiquement par ligne et par vente, arrondi unique à la précision de la devise.	sale_item, sale.total_amount, currency.decimal_places
Erreurs de caisse	Devise choisie à l'encaissement, taux de change affiché et mémorisé, reçu systématique.	sale.currency_code, exchange_rate
Oublis d'enregistrement	Saisie rapide d'un panier ; écriture locale immédiate qui ne dépend pas du réseau.	Base locale IndexedDB, statut PENDING
Ruptures de stock	Stock courant en temps réel, seuil d'alerte par produit, alerte de stock faible.	product.current_stock, low_stock_threshold
Pertes liées à une mauvaise gestion	Chaque variation de stock est tracée avec sa cause (vente, réapprovisionnement, ajustement motivé).	stock_movement
Bénéfice réel inconnu	Coût d'achat figé sur chaque ligne de vente ; bénéfice = (prix − coût) × quantité.	sale_item.unit_cost, unit_price
Absence d'historique	Ventes jamais supprimées, annulations tracées, historique consultable et filtrable.	sale (statut), sale_item

Tableau 1.1 : Du problème métier au modèle de données

1.3 Périmètre du MVP

Le MVP (produit minimum viable) couvre exactement les fonctionnalités décrites par le porteur du projet, plus quelques cas d'utilisation induits (taux de change, réapprovisionnement, ajustement, annulation) identifiés au chapitre 4. Tout le reste est explicitement exclu pour éviter d'inventer des fonctionnalités inutiles.

Dans le périmètre (MVP)	Hors périmètre (évolutions possibles)
• Compte commerçant, connexion JWT, profil • Produits, catégories, prix en CDF/USD • Stock en temps réel, mouvements, alertes • Ventes multi-produits, choix de la devise, annulation • Chiffre d'affaires, bénéfice, statistiques • Reçus et partage WhatsApp (lien) • Fonctionnement hors ligne et synchronisation	• Vente à crédit, dettes clients, paiements partiels • Plusieurs employés, rôles, plusieurs boutiques • Paiement mobile money intégré • API WhatsApp Business (envoi automatique) • Impression thermique, code-barres • Fournisseurs et commandes d'achat • Facturation fiscale normalisée, TVA • Services financiers ou de crédit (voir encadré)

Tableau 1.2 : Périmètre fonctionnel du MVP

Précision importante

L'accès au crédit n'est pas une fonctionnalité de Doku. Un historique commercial fiable (ventes, stock, bénéfice) pourrait, à terme, servir de base à des services financiers ; il s'agit d'un objectif ou d'une possibilité future, jamais d'une garantie. Aucune promesse de ce type ne doit apparaître dans l'application ni dans la communication tant qu'un partenaire financier n'est pas engagé.

1.4 Parties prenantes et utilisateurs

Partie prenante	Rôle	Attentes principales
Commerçant (utilisateur)	Utilise l'application au quotidien, propriétaire des données	Simplicité, rapidité, fiabilité des chiffres, aucune perte de données
Client final du commerçant	Reçoit le reçu ; n'utilise pas Doku	Reçu clair et lisible sur WhatsApp ; respect de sa vie privée
Porteur du projet / équipe Doku	Conçoit, développe, exploite et fait évoluer le produit	Architecture maintenable, sécurité, évolutivité
Partenaires futurs (finance, fournisseurs)	Non impliqués en MVP	Données fiables et exploitables, uniquement avec le consentement du commerçant

Tableau 1.3 : Parties prenantes

1.5 Contraintes de conception

- **Terminal** : smartphone Android d'entrée ou de milieu de gamme, mémoire et stockage limités (H-14).
- **Réseau** : connexion intermittente et coûteuse ; échanges sobres, jamais bloquants pour la vente.
- **Monnaies** : CDF et USD en parallèle, taux saisi par le commerçant (H-09).
- **Utilisateurs** : faible aisance numérique possible ; interface en français, vocabulaire du commerce, gros éléments tactiles.
- **Technologies imposées** : Django, Django REST Framework, django-cors-headers, JWT, UUID côté serveur ; Next.js, React, Axios, Lucide React, React Toastify côté client.

1.6 Risques majeurs

ID	Risque	Prob.	Impact	Mesure de maîtrise
R-01	Perte de données locales (stockage navigateur purgé, téléphone perdu)	Moyenne	Élevé	Demande de stockage persistant, synchronisation opportuniste dès que possible, compteur « à synchroniser » visible, export de secours.
R-02	Bénéfice faux par prix d'achat absent ou obsolète	Élevée	Élevé	Prix d'achat obligatoire, rappel lors du réapprovisionnement, instantané de coût sur chaque ligne.
R-03	Taux de change non mis à jour	Élevée	Moyen	Alerte « taux ancien » (ex. > 48 h), taux obligatoire à l'onboarding, historique visible.
R-04	Divergence entre calculs client et serveur (statistiques)	Moyenne	Moyen	Règles de calcul uniques (chapitre 12) et jeu de tests communs aux deux côtés.
R-05	Bugs de synchronisation (doublons, perte, boucles de retry)	Moyenne	Élevé	UUID, opérations idempotentes, journal de synchronisation, tests de bout en bout dès le sprint 1.
R-06	Horloges d'appareils erronées	Moyenne	Moyen	Curseur basé sur l'horloge serveur ; contrôle de plausibilité de sold_at ; conservation de server_updated_at.
R-07	Appareil partagé ou perdu : accès aux données commerciales	Moyenne	Moyen	Session limitée, déconnexion explicite, verrouillage par code PIN envisagé (évolution).
R-08	Limites des PWA sur iOS (stockage, notifications)	Moyenne	Moyen	Support iOS en best effort, message d'installation, pas de dépendance à la synchronisation en arrière-plan.
R-09	Adoption : interface jugée trop complexe	Moyenne	Élevé	Parcours de vente minimal, tests utilisateurs avec de vrais commerçants avant chaque jalon.
R-10	Attentes irréalistes sur l'accès au crédit	Moyenne	Moyen	Communication claire : objectif futur non garanti, aucune promesse dans l'application.

Tableau 1.4 : Registre des risques initial

2. Analyse fonctionnelle

2.1 Démarche et traçabilité

L'analyse suit une chaîne de traçabilité unique : **besoin → cas d'utilisation → activité / séquence → entité Merise → table → route d'API**. Chaque élément reçoit un identifiant (BF, UC, A, S, RG, H) et le chapitre 20 vérifie automatiquement que la chaîne n'est jamais rompue. Les informations manquantes sont traitées comme des **hypothèses explicites** (section 2.5) : on indique l'information absente, l'hypothèse retenue et son impact sur la modélisation.

2.2 Besoins fonctionnels

ID	Besoin	Priorité	Cas d'utilisation
BF-01	Créer un compte, se connecter, se déconnecter, gérer le profil	Must	UC01, UC02, UC03, UC04
BF-02	Gérer les produits (ajout, modification, archivage, consultation) et les catégories	Must	UC06, UC07, UC08, UC09, UC10
BF-03	Enregistrer une vente multi-produits avec total automatique	Must	UC18, UC19, UC20
BF-04	Gérer les prix et les ventes en CDF et en USD, avec taux de change	Must	UC05, UC21
BF-05	Suivre le stock en temps réel et tracer chaque mouvement	Must	UC11, UC12, UC13, UC14, UC15, UC17
BF-06	Alerter en cas de stock faible	Should	UC16
BF-07	Consulter l'historique des ventes et annuler une vente	Must	UC22, UC23
BF-08	Consulter chiffre d'affaires, bénéfice et statistiques	Must	UC24, UC25, UC26
BF-09	Générer un reçu	Must	UC27
BF-10	Partager un reçu via WhatsApp	Should	UC28
BF-11	Fonctionner sans connexion Internet	Must	UC29
BF-12	Synchroniser les données au retour de la connexion et gérer les conflits	Must	UC30, UC31

Tableau 2.1 : Besoins fonctionnels (priorités MoSCoW : Must = indispensable, Should = important)

2.3 Besoins non fonctionnels

ID	Catégorie	Exigence
BNF-01	Disponibilité hors ligne	Caisse, produits, stock, historique et statistiques utilisables sans Internet, indéfiniment.
BNF-02	Fiabilité des données	Aucune vente perdue ni dupliquée ; zéro écart entre stock local et stock serveur après synchronisation.
BNF-03	Performance (cibles proposées)	Ouverture de la caisse < 3 s sur un téléphone d'entrée de gamme ; validation d'une vente < 1 s (écriture locale) ; synchronisation de 100 ventes < 30 s en 3G.
BNF-04	Utilisabilité	Conception mobile-first, gros éléments tactiles, ajout d'un produit au panier en 2 interactions, vocabulaire du commerce (pas de jargon).
BNF-05	Sobriété réseau	Échanges JSON compacts, lots limités, pas de ressources lourdes (pas d'images produit en MVP).
BNF-06	Sécurité	Authentification JWT, isolation par boutique, validation systématique, HTTPS, journalisation sans donnée sensible.
BNF-07	Maintenabilité	Architecture en couches, services métier testés, conventions de nommage et de commits, intégration continue.
BNF-08	Évolutivité	Modèle prêt pour multi-boutiques, employés, paiements, clients et API WhatsApp Business sans refonte.
BNF-09	Compatibilité	Chrome Android récent ; Safari iOS en PWA installée (best effort) ; écrans de 360 px de large minimum.
BNF-10	Observabilité	Journaux applicatifs structurés, journal des synchronisations, supervision des erreurs.

Tableau 2.2 : Besoins non fonctionnels (les valeurs chiffrées sont des cibles proposées, à valider)

2.4 Règles de gestion

Les règles de gestion (RG) sont les règles métier que le système doit garantir. Elles sont référencées par les diagrammes, le MLD (contraintes) et les tests.

ID	Règle
RG-01	Un compte correspond à un commerçant et à une boutique (MVP).
RG-02	Toute donnée métier appartient à une boutique ; le serveur impose shop_id à partir de l'utilisateur authentifié (jamais depuis la requête).
RG-03	Un produit a un nom, un prix d'achat ≥ 0 et un prix de vente ≥ 0, chacun avec sa devise (CDF ou USD).
RG-04	Supprimer un produit ou une catégorie = archiver (suppression logique). Aucune suppression physique de données financières ou de stock.
RG-05	Le stock ne se modifie que par la création de mouvements de stock ; le stock courant est la somme des mouvements.
RG-06	Types de mouvements : INITIAL, RESTOCK et SALE_CANCEL (quantité > 0) ; SALE (quantité < 0) ; ADJUSTMENT (non nul, motif obligatoire).
RG-07	Une vente comporte au moins une ligne ; chaque quantité est un entier > 0 ; toutes les lignes sont exprimées dans l'unique devise de la vente.
RG-08	La caisse refuse une quantité supérieure au stock local disponible ; le serveur accepte toute vente déjà réalisée et signale un stock négatif.
RG-09	Une ligne de vente fige la désignation, le prix unitaire et le coût unitaire ; elle n'est jamais recalculée.
RG-10	Une vente validée est immuable ; son annulation change le statut en CANCELLED et ajoute des mouvements SALE_CANCEL. Elle n'est jamais supprimée.
RG-11	Toute vente référence le taux de change en vigueur (dernier taux de la boutique à l'instant de la vente) ; les taux sont en ajout seul.
RG-12	Un prix converti est arrondi une seule fois, à la précision de la devise de la vente (arrondi commercial, demi vers le haut) ; ensuite les calculs sont exacts.
RG-13	Bénéfice d'une ligne = (prix unitaire − coût unitaire) × quantité ; seules les ventes COMPLETED comptent dans les statistiques.
RG-14	Les statistiques consolidées convertissent chaque vente avec son propre taux vers la devise de référence de la boutique.

ID	Règle
RG-15	Le reçu est une représentation de la vente (aucune entité propre) ; son numéro = date (AAMMJJ) + code court dérivé de l'UUID.
RG-16	Toute donnée créée hors ligne reçoit un UUID v4 côté client ; la synchronisation est idempotente (rejouer un envoi ne duplique rien).
RG-17	Un enregistrement local n'est jamais supprimé avant confirmation de sa synchronisation par le serveur.
RG-18	Une alerte de stock faible est affichée quand stock courant ≤ seuil d'alerte du produit.
RG-19	Le stock n'est jamais synchronisé comme une valeur mais comme des variations (mouvements) ; le serveur recalcule le stock courant.
RG-20	Les taux de change et les ventes ne sont jamais modifiés après coup ; une correction se fait par un nouvel enregistrement (nouveau taux, annulation).

Tableau 2.3 : Règles de gestion

2.5 Informations manquantes et hypothèses

La description initiale ne précise pas tous les points nécessaires à une modélisation exploitable. Plutôt que de les inventer en silence, chaque trou est listé avec l'hypothèse retenue et son impact. **Le porteur du projet est invité à valider ou corriger chaque ligne** ; les plus structurantes sont H-01, H-02, H-04, H-06 et H-11.

ID	Information manquante	Hypothèse retenue	Impact sur la modélisation
H-01	**Identifiant de connexion**	Le commerçant se connecte avec son numéro de téléphone (format E.164) et un mot de passe.	app_user.phone UNIQUE ; email facultatif. Le téléphone est aussi le nom d'utilisateur Django (USERNAME_FIELD).
H-02	**Nombre d'utilisateurs par boutique**	Un seul utilisateur (le propriétaire) par boutique ; pas d'employés ni de rôles en MVP.	Pas de table de rôles ; autorisation = « propriétaire de la boutique ». Évolution : table membre(boutique, utilisateur, rôle).
H-03	**Nombre de boutiques**	Un commerçant possède une seule boutique.	shop.owner_id UNIQUE. Lever la contrainte suffit à passer en multi-boutiques (toutes les données portent déjà shop_id).
H-04	**Unité de vente**	Les produits sont vendus à l'unité (quantités entières). Pas de vente au poids ou au volume.	quantity INTEGER. Évolution : DECIMAL(12,3) + champ unité sur le produit.
H-05	**Vente avec stock insuffisant**	La caisse bloque une ligne dont la quantité dépasse le stock local ; le serveur, lui, n'annule jamais une vente déjà réalisée et signale un stock négatif.	Pas de CHECK current_stock >= 0 en base. Alerte « stock à vérifier » + ajustement manuel.
H-06	**Crédit client et paiements partiels**	Aucune vente à crédit, aucun paiement partiel : une vente validée est encaissée en totalité, en une seule devise.	Pas d'entité Paiement ni Client en MVP. Évolution : Paiement (1,1)–(0,N) Vente ; une ligne de paiement par vente existante à la migration.
H-07	**Mode de paiement**	Non distingué (espèces et mobile money confondus).	Aucun attribut « mode » ; ajout simple ultérieur (colonne avec valeur par défaut).
H-08	**Fichier clients**	Pas de gestion de clients. Le numéro WhatsApp est saisi à l'envoi du reçu et n'est pas conservé.	Pas de table customer ; /customers/ non implémenté. Réduit les données personnelles stockées.
H-09	**Taux de change**	Saisi manuellement par le commerçant ; un seul couple USD → CDF en MVP ; pas de source automatique.	Table exchange_rate générique (base, cotée) mais alimentée uniquement en USD/CDF.
H-10	**Précision des montants**	CDF sans décimales (en pratique), USD à 2 décimales.	currency.decimal_places pilote l'arrondi ; montants stockés en DECIMAL(14,2).
H-11	**Coût d'achat**	Le coût d'une vente est le prix d'achat courant du produit au moment de la vente (dernier prix connu). Pas de CMUP ni de FIFO.	Pas de coût par lot. Évolution : coût unitaire sur les mouvements RESTOCK et calcul du coût moyen pondéré.
H-12	**Valeur légale du reçu**	Le reçu Doku est un justificatif commercial, pas une facture fiscale. La conformité fiscale est hors périmètre.	Numérotation non séquentielle (impossible hors ligne sans coordination). À étudier avec un conseil fiscal si nécessaire.

ID	Information manquante	Hypothèse retenue	Impact sur la modélisation
H-13	**Date d'une vente**	La date de vente est l'instant de validation sur l'appareil ; pas de saisie rétroactive en MVP.	sold_at = horloge de l'appareil ; le serveur contrôle la plausibilité et conserve server_updated_at.
H-14	**Terminal cible**	Smartphone Android d'entrée ou milieu de gamme (Chrome) en priorité ; iOS (Safari, PWA installée) en best effort.	PWA + IndexedDB ; pas de dépendance à la Background Sync API ; demande de stockage persistant.
H-15	**Langue de l'interface**	Français.	Libellés externalisés pour faciliter d'autres langues plus tard.
H-16	**Volumétrie**	Moins de 2 000 produits et moins de 500 ventes par jour et par boutique.	Aucune contrainte de partitionnement ; index simples suffisants.
H-17	**Récupération de mot de passe**	Hors MVP. Seul le changement de mot de passe (connecté) est prévu.	Pas de table de jetons de réinitialisation.
H-18	**Hébergement et base de données**	Base PostgreSQL recommandée (contraintes CHECK, index partiels, JSONB). MySQL reste possible avec adaptations.	Le MLD reste neutre ; les écarts MySQL sont listés au chapitre 9.
H-19	**Sens de « vente annulée »**	Deux cas distincts : l'abandon d'un panier avant validation (aucune trace) et l'annulation d'une vente déjà validée.	UC22 ajouté ; sale.status, cancelled_at, cancel_reason ; mouvements SALE_CANCEL. Si seul l'abandon est voulu, UC22 se retire.

Tableau 2.4 : Hypothèses de modélisation à valider

Pourquoi valider maintenant

Une hypothèse fausse coûte cher quand elle est découverte tard (migration de données, refonte d'écrans). Valider les hypothèses H-01 à H-19 **avant** d'écrire les modèles Django est le meilleur investissement du projet.

3. Acteurs

3.1 Définition et méthode d'analyse

En UML, un **acteur** est une entité *extérieure* au système qui interagit avec lui (une personne ou un autre système). Un composant interne (une API, un service de synchronisation, un module d'authentification) n'est pas un acteur : il fait partie du système. Cette règle sert de critère pour décider, acteur par acteur, s'il doit figurer dans la modélisation, sans en ajouter artificiellement.

3.2 Acteur principal : le commerçant

Rôle	Propriétaire de la boutique ; seul utilisateur du système en MVP (H-02).
Objectifs	Vendre vite, ne plus se tromper, connaître son stock et son bénéfice, informer ses clients.
Contexte d'usage	Smartphone, souvent debout derrière son comptoir, connexion instable, interruptions fréquentes.
Compétences	Connaît son commerce ; aisance numérique variable ; attend une interface très simple.
Cas d'utilisation	Tous, sauf les comportements internes (UC17, UC31) déclenchés par le système.

3.3 Acteur secondaire : WhatsApp

WhatsApp est un **système externe** : Doku ne le contrôle pas, il ne peut que lui transmettre un lien ou un contenu à partager. C'est pourquoi il apparaît comme acteur secondaire (à droite du diagramme) pour le cas d'utilisation « Partager un reçu via WhatsApp ». Le client final, destinataire du message, n'est pas un acteur : il ne dialogue jamais avec Doku.

3.4 Analyse des acteurs candidats

Acteur candidat	Décision	Justification
Administrateur	Non retenu en MVP	Aucune fonctionnalité demandée ne l'exige. L'interface /admin/ de Django est un outil interne pour l'équipe technique, pas un acteur fonctionnel. À introduire si un support multi-commerçants, une validation de comptes ou un tableau de bord de plateforme deviennent nécessaires.
Service de synchronisation	Non retenu (composant interne)	Le moteur de synchronisation (client) et l'endpoint /sync/ (serveur) font partie du système. Le déclenchement automatique au retour du réseau est un comportement interne du système, pas un acteur.
Service d'authentification	Non retenu (composant interne)	L'authentification JWT est réalisée par Django (SimpleJWT) à l'intérieur du système. Elle deviendrait un acteur externe seulement avec un fournisseur d'identité tiers (OTP par SMS, connexion Google…), non prévu.
Service WhatsApp	Retenu (secondaire)	Système externe réel, hors de contrôle de Doku. Le MVP n'utilise que le lien de partage ; l'API WhatsApp Business serait un second type d'interaction (hors MVP).
Client final	Non retenu	Il reçoit un message mais n'agit pas sur Doku ; pas de fichier client en MVP (H-08).
Réseau / horloge	Non retenu	Simples événements déclencheurs (retour de connexion, minuterie) ; ils sont modélisés comme conditions d'extension, pas comme acteurs.

Tableau 3.1 : Décision sur chaque acteur candidat

> **Décision**
> Le diagramme de cas d'utilisation comporte donc **deux acteurs** : le **commerçant** (principal) et **WhatsApp** (secondaire, système externe). Cette sobriété est volontaire : un acteur de plus entraînerait des cas d'utilisation, des tables et des routes à maintenir sans valeur pour le MVP.

4. Diagramme de cas d'utilisation

4.1 Liste des cas d'utilisation

Le système compte **31 cas d'utilisation** répartis en sept paquetages. Les 23 cas listés par le porteur du projet sont tous présents ; huit cas ont été ajoutés ou précisés parce qu'ils découlent directement des diagrammes d'activité demandés (réapprovisionnement, alerte de stock faible, annulation, conflit) ou de la gestion bi-devise (taux de change).

ID	Cas d'utilisation	Paquetage	Relations UML	Hors ligne
UC01	Créer un compte	Compte et profil	—	Non
UC02	Se connecter	Compte et profil	—	Partiel
UC03	Se déconnecter	Compte et profil	—	Partiel
UC04	Gérer son profil	Compte et profil	—	Lecture seule
UC05	Gérer le taux de change	Compte et profil	—	Oui
UC06	Ajouter un produit	Catalogue	«include» UC11	Oui
UC07	Modifier un produit	Catalogue	—	Oui
UC08	Supprimer (archiver) un produit	Catalogue	—	Oui
UC09	Consulter les produits	Catalogue	—	Oui
UC10	Gérer les catégories	Catalogue	—	Oui
UC11	Définir le stock initial	Stock	«include» UC17	Oui
UC12	Consulter le stock	Stock	—	Oui
UC13	Consulter les mouvements de stock	Stock	—	Oui
UC14	Réapprovisionner un produit	Stock	«include» UC17	Oui
UC15	Ajuster le stock	Stock	«include» UC17	Oui
UC16	Signaler un stock faible	Stock	«extend» UC12	Oui
UC17	Enregistrer les mouvements de stock	Stock	—	Oui
UC18	Enregistrer une vente	Ventes	«include» UC19, UC20, UC21, UC17, UC27 ; «extend» par UC29	Oui
UC19	Ajouter des produits à la vente	Ventes	—	Oui
UC20	Calculer automatiquement le total	Ventes	—	Oui
UC21	Choisir la devise (CDF ou USD)	Ventes	—	Oui
UC22	Annuler une vente	Ventes	«include» UC17	Oui
UC23	Consulter l'historique des ventes	Ventes	—	Oui
UC24	Consulter les statistiques	Pilotage	Généralise UC25, UC26	Oui
UC25	Consulter le chiffre d'affaires	Pilotage	Spécialise UC24	Oui
UC26	Consulter le bénéfice	Pilotage	Spécialise UC24	Oui
UC27	Générer un reçu	Reçus	—	Oui
UC28	Partager un reçu via WhatsApp	Reçus	«include» UC27	Partiel
UC29	Travailler hors ligne	Hors ligne et synchronisation	«extend» UC18	Oui
UC30	Synchroniser les données	Hors ligne et synchronisation	—	Non (requiert réseau)
UC31	Résoudre un conflit de synchronisation	Hors ligne et synchronisation	«extend» UC30	Non (requiert réseau)

Tableau 4.1 : Catalogue des cas d'utilisation (« Hors ligne » : utilisable sans Internet)

4.2 Relations UML utilisées

Relation	Sens	Exemple dans Doku	Pourquoi
«include»	Le cas de base *inclut toujours* le cas inclus	UC18 Enregistrer une vente → UC21 Choisir la devise ; UC06 → UC11 Définir le stock initial	Comportement obligatoire et réutilisable (le mouvement de stock UC17 est partagé par 5 cas).
«extend»	Le cas étendant ajoute un comportement *optionnel sous condition*	UC29 Travailler hors ligne étend UC18 [pas de connexion] ; UC16 Stock faible étend UC12 [stock ≤ seuil] ; UC31 étend UC30 [conflit]	Comportement conditionnel qui ne fait pas partie du déroulement normal.
Généralisation	Le cas spécialisé hérite du cas général	UC25 Chiffre d'affaires et UC26 Bénéfice spécialisent UC24 Statistiques	Deux façons de consulter la même famille d'indicateurs.

Tableau 4.2 : Relations entre cas d'utilisation

4.3 Cas d'utilisation ajoutés ou précisés

Cas	Origine de l'ajout
UC05 Gérer le taux de change	Conséquence directe de la gestion CDF/USD : sans taux, ni conversion ni consolidation possibles.
UC11 Définir le stock initial	Précisé par l'activité « Gestion du stock » (définition du stock initial).
UC14 Réapprovisionner un produit	Demandé par l'activité « Gestion du stock » (réapprovisionnement).
UC15 Ajuster le stock	Découle de l'exigence « ajustements » et de la traçabilité du stock.
UC16 Signaler un stock faible	Exigé par l'activité « Gestion du stock » (vérification du seuil, alerte de stock faible).
UC17 Enregistrer les mouvements de stock	Comportement partagé extrait des activités 1 et 4 (mise à jour du stock) : une seule règle pour toutes les variations.
UC22 Annuler une vente	Déduit de « Vente annulée » (activité 1) et du principe d'immuabilité des ventes.
UC31 Résoudre un conflit de synchronisation	Demandé par l'activité « Synchronisation » (cas : conflit de données).

Tableau 4.3 : Ajouts justifiés (aucun ajout gratuit)

> **Hypothèse H-19 : sens de « vente annulée »**
> **« Vente annulée » (activité 1)** peut désigner l'abandon d'un panier avant validation *ou* l'annulation d'une vente déjà validée. Les deux sont modélisés : l'abandon ne laisse aucune trace (le panier est un état local volatil) ; l'annulation d'une vente validée (UC22) change son statut et remet le stock, sans jamais supprimer la vente. Si seul l'abandon était voulu, UC22 se retire sans autre impact que le statut CANCELLED.

4.4 Lecture des diagrammes

- **Ellipse** : un cas d'utilisation (un objectif du commerçant). **Silhouette** : un acteur. **Rectangle** : la frontière du système Doku.
- **Trait plein** acteur → cas : l'acteur déclenche le cas. **Flèche pointillée «include»** : va du cas de base vers le cas inclus. **Flèche pointillée «extend»** : va du cas qui étend vers le cas de base. **Flèche à pointe creuse** : généralisation.
- Précondition commune : **session authentifiée** pour tous les cas sauf UC01 et UC02 ; elle n'est pas dessinée pour ne pas surcharger les figures.

Le diagramme est présenté en quatre figures : une **vue d'ensemble** qui regroupe les 31 cas par paquetage, puis trois **vues détaillées** qui montrent les relations include, extend et de généralisation.

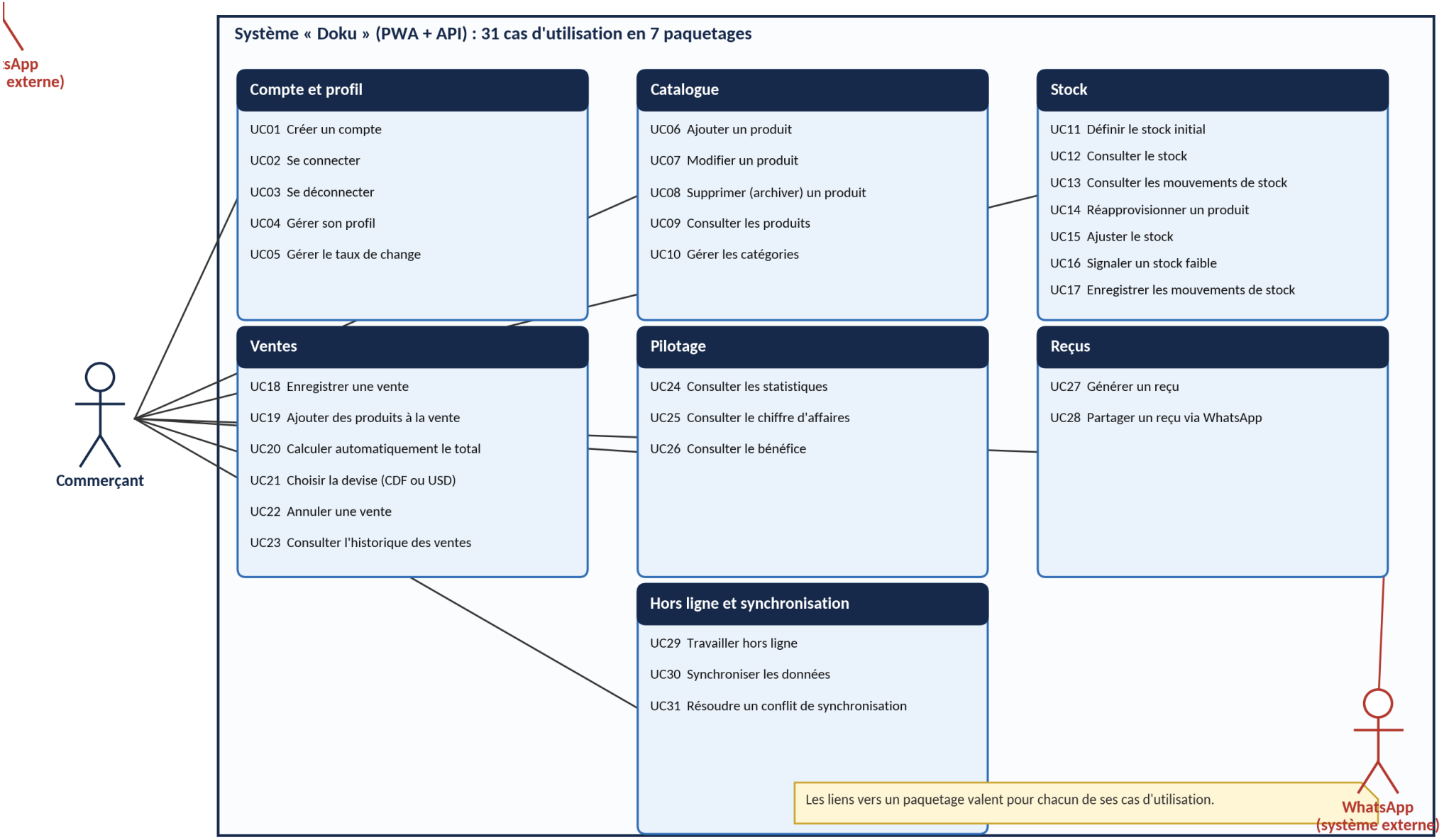


Figure 4.1 : Vue d'ensemble : 31 cas d'utilisation, 7 paquetages, 2 acteurs

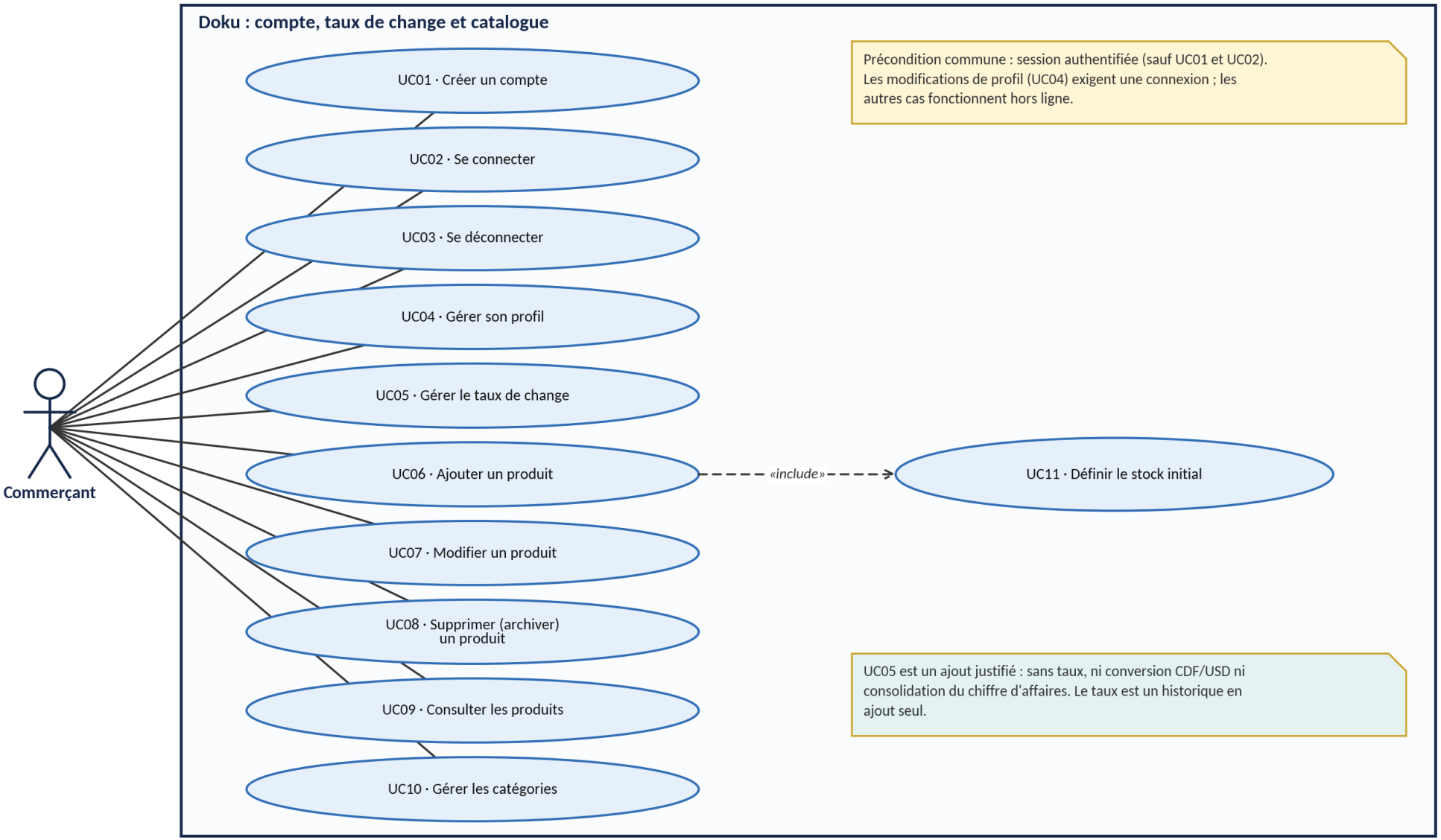


Figure 4.2 : Vue A : compte, taux de change et catalogue

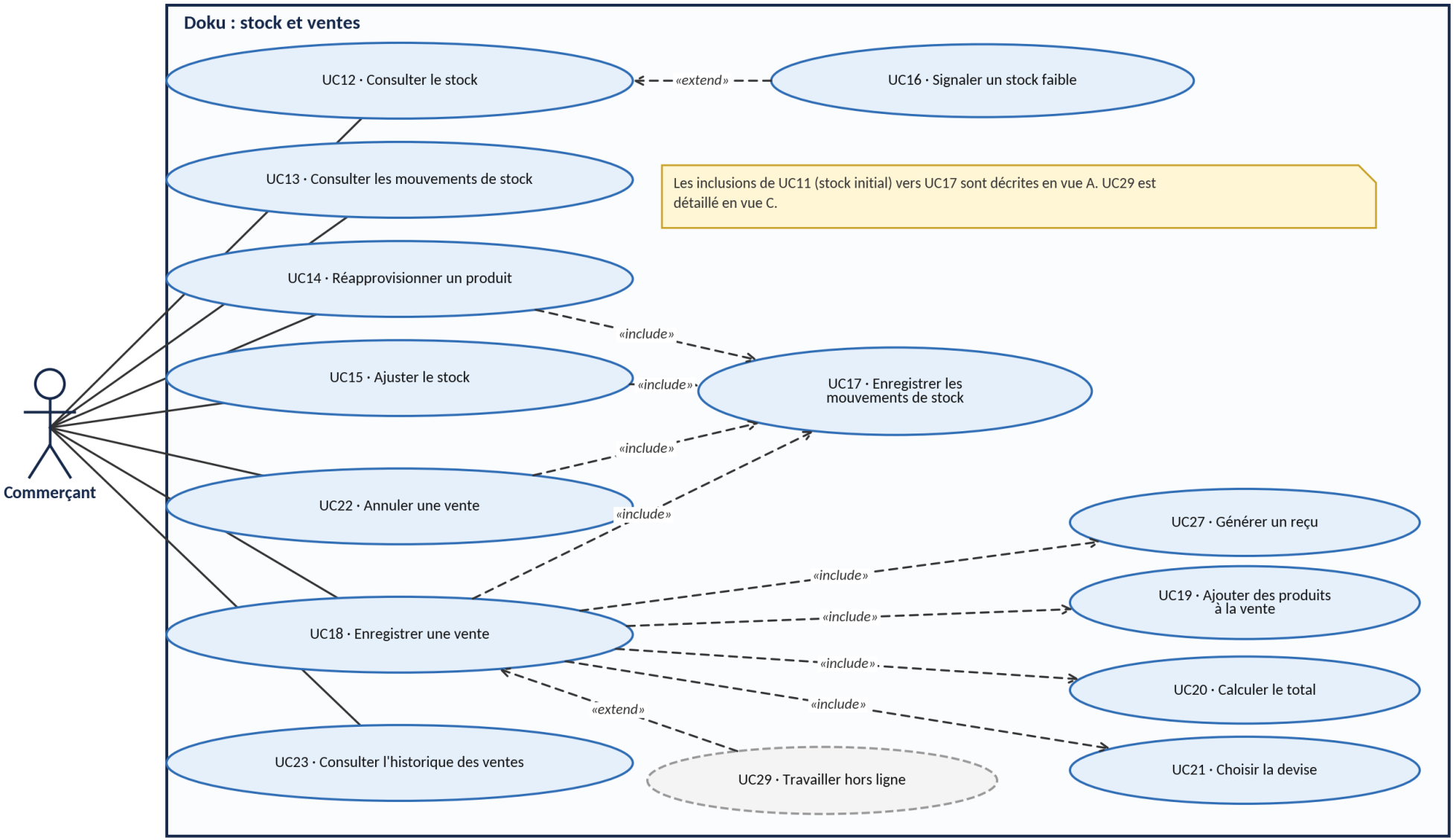


Figure 4.3 : Vue B : stock et ventes (include et extend)

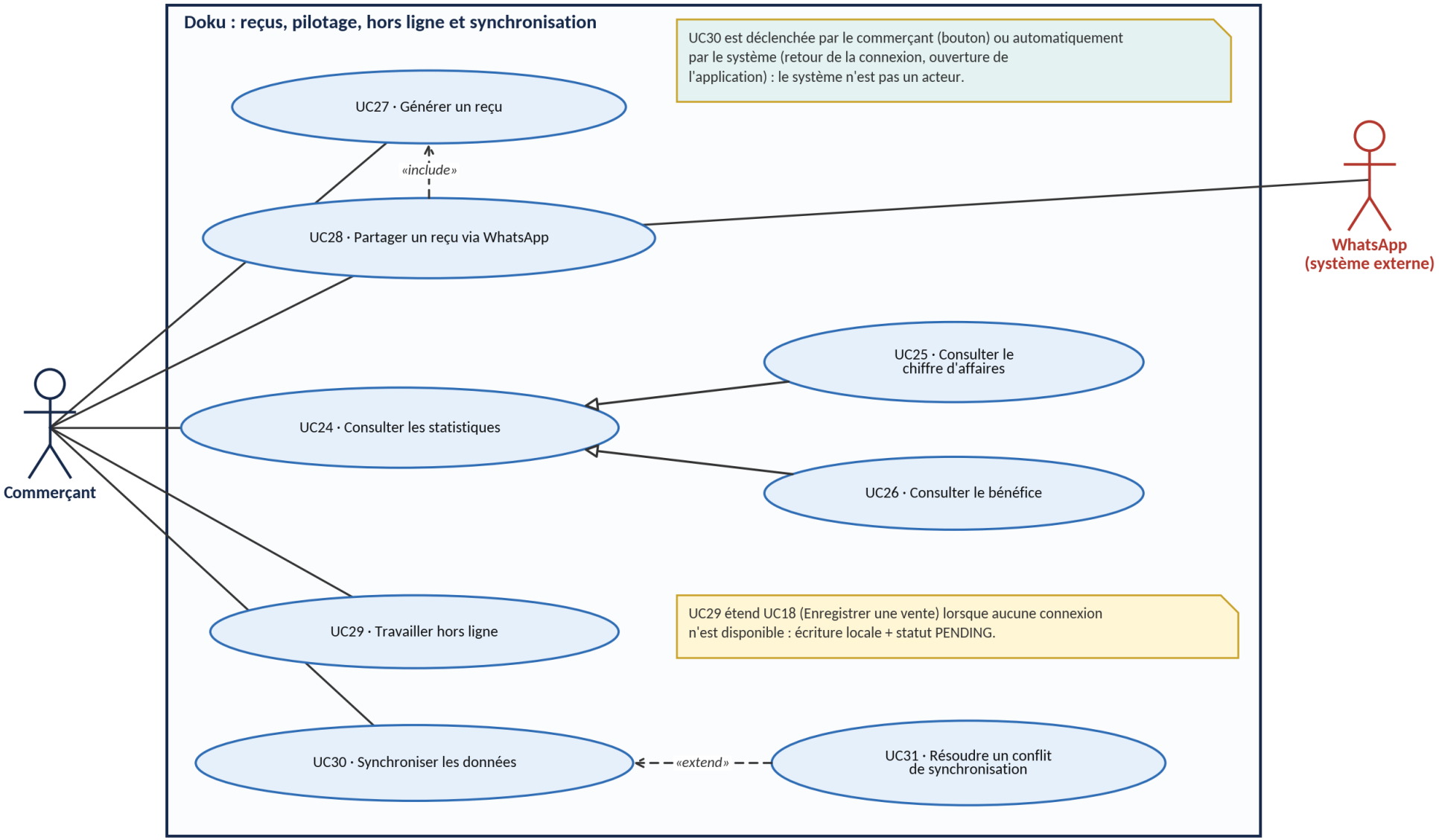


Figure 4.4 : Vue C : reçus, pilotage, hors ligne et synchronisation (généralisation)

4.5 Explication des vues

- **Vue A** : dix cas se déclenchent directement par le commerçant. UC06 inclut UC11 car la création d'un produit demande toujours un stock initial (éventuellement 0). UC05 (taux de change) est le prérequis de toute conversion.
- **Vue B** : UC18 est le cœur du produit. Il inclut le panier (UC19), le calcul du total (UC20), le choix de la devise (UC21), les mouvements de stock (UC17) et le reçu (UC27). UC17 est partagé avec UC14, UC15 et UC22 : toute variation de stock passe par la même règle. UC16 étend UC12 sous condition de seuil.
- **Vue C** : UC28 inclut UC27 (un partage suppose un reçu) et fait intervenir WhatsApp. UC24 est la généralisation de UC25 et UC26. UC30 est déclenchée par le commerçant ou automatiquement ; UC31 l'étend lorsqu'un conflit survient.

5. Diagrammes d'activités

5.1 Rôle des diagrammes et conventions de lecture

Un **diagramme d'activité** décrit le déroulement d'un processus : les actions, les décisions et les boucles, du point de départ jusqu'à la fin. Il répond à la question « que se passe-t-il, dans quel ordre, et que fait-on quand quelque chose ne va pas ? ». Les cinq processus ci-dessous sont ceux qui portent la valeur de Doku.

Symbole	Signification
Disque noir	Début du processus
Disque cerclé	Fin du processus
Rectangle arrondi bleu	Action du commerçant
Rectangle arrondi gris	Action du système (application ou service local)
Rectangle arrondi vert	Action du serveur ou état nominal
Rectangle arrondi rouge	Erreur, alerte ou cas d'échec
Losange jaune	Décision : chaque sortie porte une condition (oui / non…)
Flèche en pointillé	Chemin d'échec ou de reprise

Tableau 5.1 : Légende commune des diagrammes d'activité

5.2 Activité 1 : enregistrement d'une vente

Objectif	Enregistrer une vente multi-produits, dans la devise choisie, avec mise à jour du stock et reçu.
Déclencheur	Le commerçant ouvre la caisse (écran principal de Doku).
Préconditions	Session ouverte ; au moins un produit actif ; un taux de change défini (H-09).
Postconditions	Une vente COMPLETED avec ses lignes et ses mouvements SALE, stock courant diminué, reçu généré. En cas d'abandon : aucune donnée enregistrée.
Cas d'erreur traités	Produit inexistant, produit indisponible (archivé), quantité invalide, stock insuffisant, vente annulée.
Règles liées	RG-07, RG-08, RG-09, RG-11, RG-12, RG-18
Cas d'utilisation	UC18, UC19, UC20, UC21, UC17, UC16, UC27

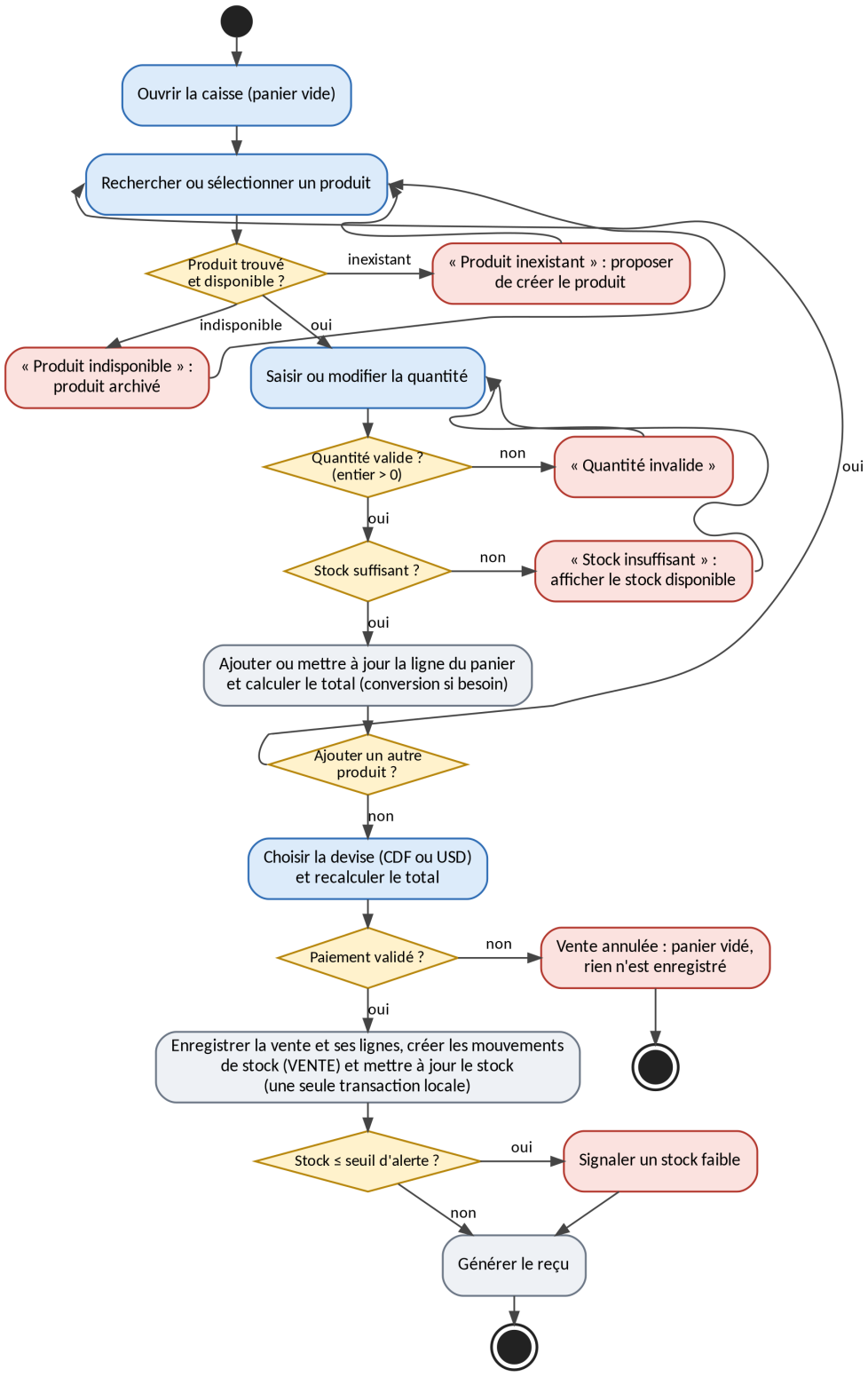


Figure 5.1 : Activité 1 : enregistrement d'une vente

Comment lire le diagramme

- La **colonne centrale** est le déroulement nominal : sélection du produit, quantité, panier, devise, paiement, enregistrement, reçu.
- Les **boîtes rouges à droite** sont les quatre erreurs de saisie ; chacune **reboucle** vers l'étape qui permet de corriger (recherche ou quantité) : le commerçant n'est jamais bloqué.

- Le losange « Ajouter un autre produit ? » reboucle vers la recherche : c'est ce qui permet un panier multi-produits.
- « Vente annulée » mène à une fin sans enregistrement : **l'abandon d'un panier ne laisse aucune trace** (état local volatil).

Atomicité
L'enregistrement de la vente, des lignes et des mouvements de stock se fait dans **une seule transaction locale** (IndexedDB). Soit tout est écrit, soit rien : il est impossible d'avoir une vente sans sa sortie de stock.

Erreur fréquente
Vérifier le stock *uniquement* à la validation finale. Le contrôle doit se faire dès la saisie de la quantité (étape « Stock suffisant ? ») *et* être rejoué dans la transaction d'enregistrement, car un autre appareil a pu vendre entre-temps.

5.3 Activité 2 : fonctionnement hors ligne

Objectif	Garantir qu'une vente se fait et se confirme sans Internet, puis atteint le serveur au retour du réseau.
Déclencheur	Le commerçant valide une vente, avec ou sans connexion.
Principe	**Écriture locale d'abord, réseau ensuite.** Le chemin « en ligne » écrit lui aussi localement avant de synchroniser : il n'existe donc qu'un seul chemin de code.
Postconditions	Vente et mouvements en base locale avec le statut PENDING puis SYNCED après confirmation du serveur.
Règles liées	RG-16, RG-17, RG-19
Cas d'utilisation	UC29, UC18, UC30

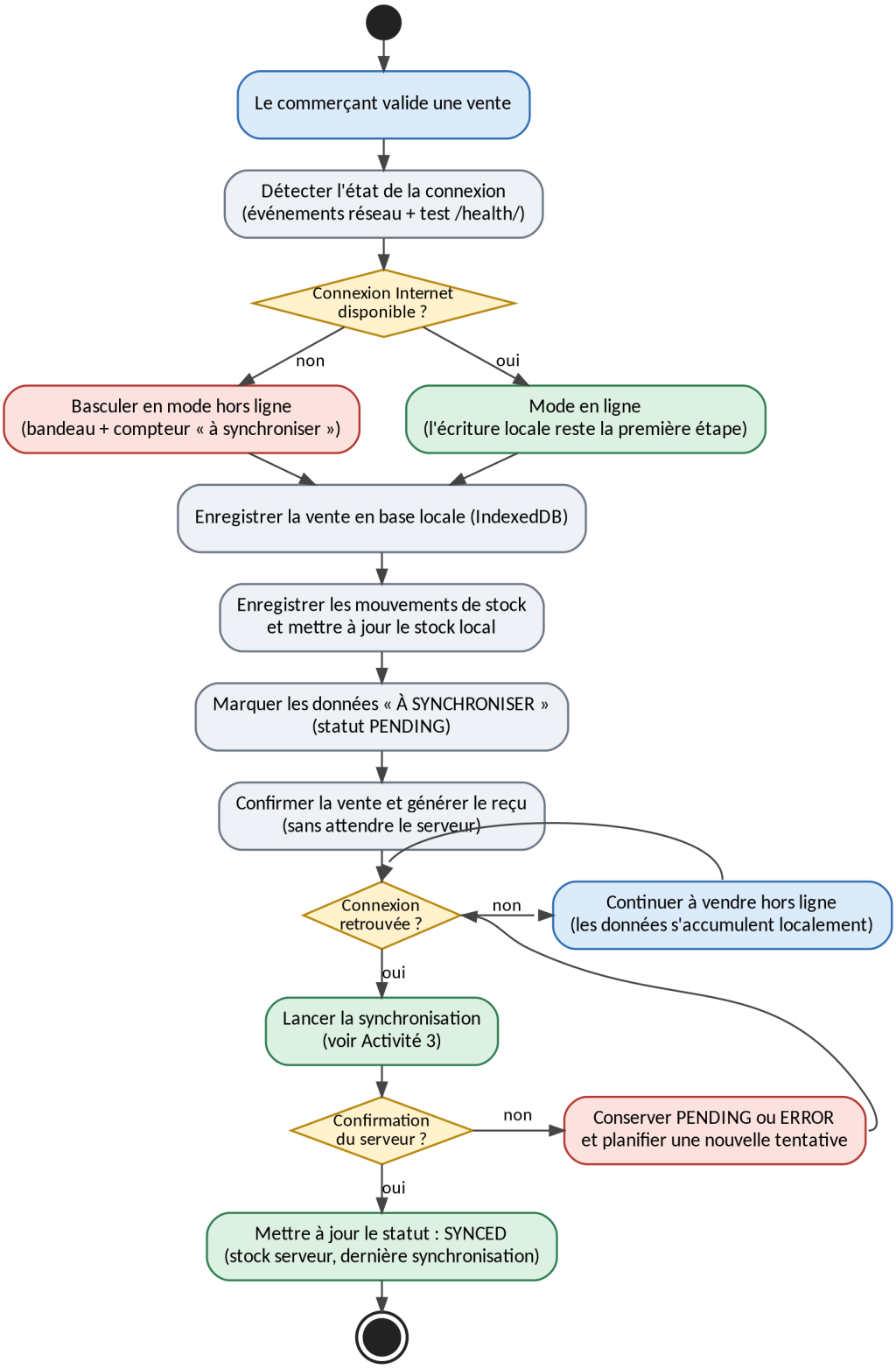


Figure 5.2 : Activité 2 : fonctionnement hors ligne

Comment lire le diagramme

- La détection de connexion combine les **événements du navigateur** et un **test réel** (appel léger à /health/) : « connecté au Wi-Fi » ne veut pas dire « Internet disponible ».
- Les deux branches (hors ligne / en ligne) **fusionnent** sur la même écriture locale ; seule la suite diffère (attente ou synchronisation immédiate).

- La boucle « Connexion retrouvée ? » représente l'accumulation de ventes hors ligne : elle peut durer des jours sans dégrader l'usage.
- En cas de refus ou d'absence de confirmation, la donnée **reste PENDING ou ERROR** et une nouvelle tentative est planifiée : rien n'est jamais supprimé avant confirmation (RG-17).

5.4 Activité 3 : synchronisation

Objectif	Envoyer les données locales en attente, recevoir les changements du serveur et réconcilier les deux sans perte ni doublon.
Déclencheurs	Retour de la connexion, ouverture de l'application, minuterie périodique pendant l'usage, bouton « Synchroniser maintenant ».
Préconditions	Connexion Internet avérée (test /health/) ; session valide ou renouvelable.
Cas traités	Échec réseau, conflit de données, donnée déjà synchronisée, erreur serveur, nouvelle tentative, session expirée.
Règles liées	RG-16, RG-17, RG-19, RG-20
Cas d'utilisation	UC30, UC31

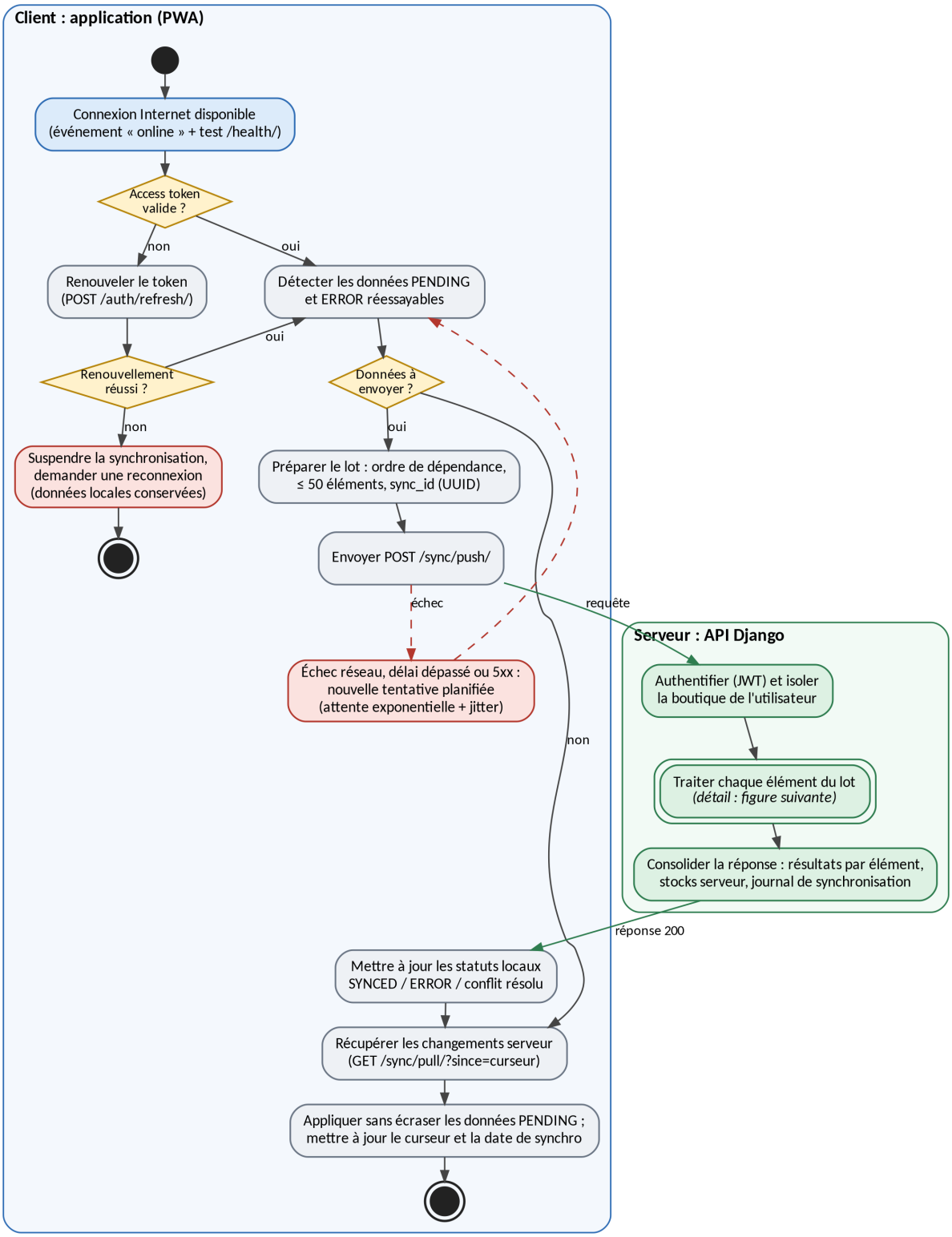


Figure 5.3 : Activité 3 : synchronisation (vue d'ensemble, couloirs client et serveur)

Comment lire le diagramme

- Deux **couloirs** : le client (à gauche, en bleu) et le serveur (à droite, en vert). Les flèches qui traversent la frontière sont des échanges HTTP.
- Avant d'envoyer quoi que ce soit, le client **vérifie sa session** ; si le renouvellement échoue, il suspend la synchronisation et demande une reconnexion **sans effacer les données locales**.

- L'échec réseau (flèche pointillée rouge) ne perd rien : les éléments restent PENDING et sont réessayés avec une attente croissante.
- Le client termine par la phase de **réception (pull)** : sans elle, un second appareil ne verrait jamais les ventes du premier.

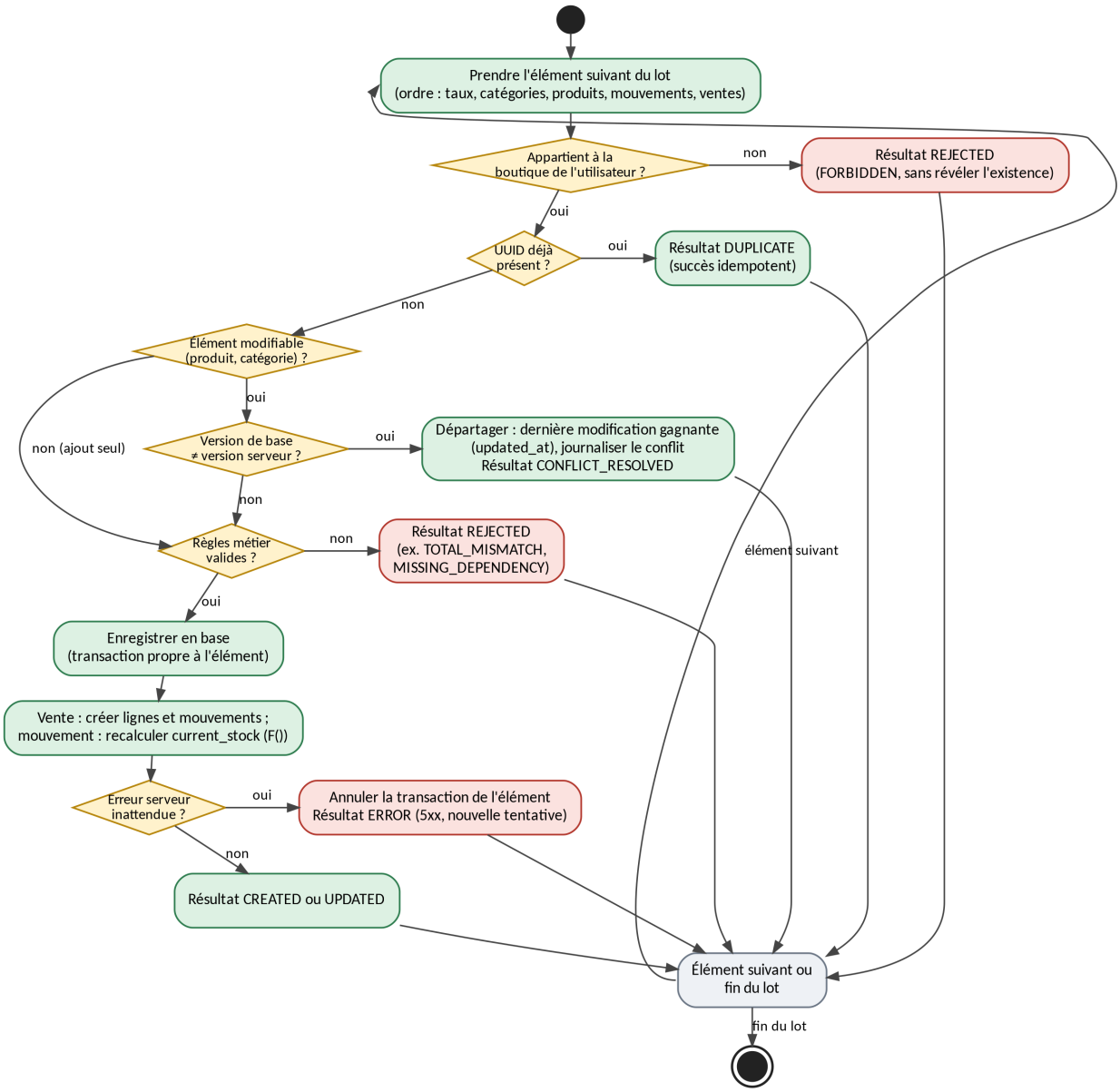


Figure 5.4 : Activité 3 (détail) : traitement serveur de chaque élément du lot

Points clés du traitement serveur

- **Isolation d'abord** : un élément qui n'appartient pas à la boutique de l'utilisateur est rejeté sans révéler s'il existe (protection contre l'énumération d'UUID).
- **Idempotence** : si l'UUID existe déjà, le serveur répond DUPLICATE (un succès) et ne modifie rien. Rejouer un lot est donc toujours sans danger.
- **Conflit** : seuls les éléments modifiables (produit, catégorie) peuvent entrer en conflit de version ; les éléments en ajout seul ne conflictent jamais.
- **Une transaction par élément** : une vente rejetée ne bloque pas les dix suivantes ; le lot peut donc être partiellement accepté (statut PARTIAL).

5.5 Activité 4 : gestion du stock

Objectif	Tenir un stock exact et traçable, de la création du produit au réapprovisionnement.
Principe	Le stock n'est jamais « saisi » : il résulte de mouvements ajoutés au journal (INITIAL, SALE, RESTOCK, ADJUSTMENT, SALE_CANCEL).
Déclencheurs	Création d'un produit, vente, achat reçu, inventaire ou casse, annulation de vente.
Postconditions	Chaque variation a une ligne de journal ; le stock courant égale la somme des mouvements ; une alerte est affichée sous le seuil.
Règles liées	RG-05, RG-06, RG-18, RG-19
Cas d'utilisation	UC06, UC11, UC12, UC14, UC15, UC16, UC17

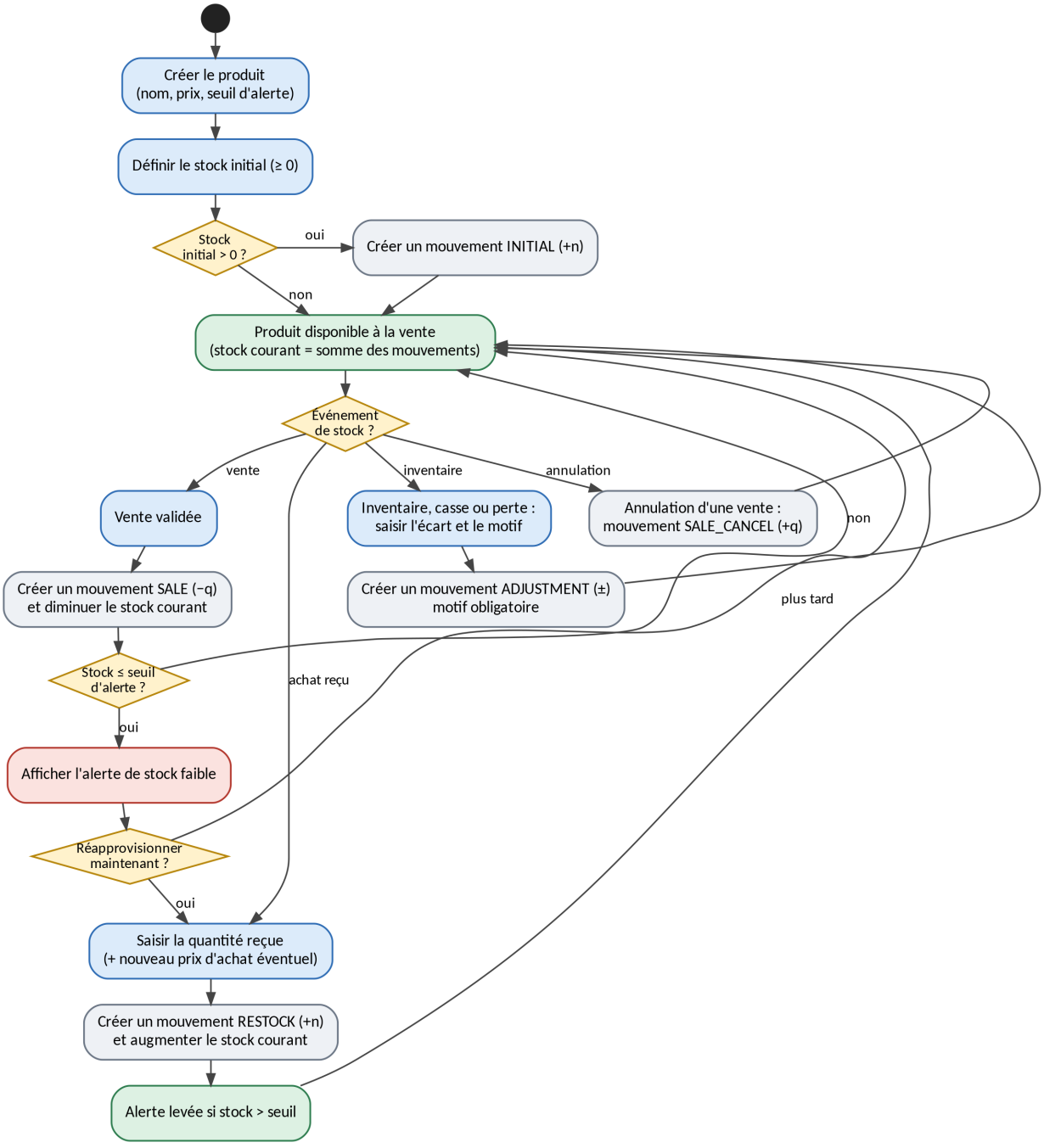


Figure 5.5 : Activité 4 : gestion du stock

Comment lire le diagramme

- Le nœud central « Produit disponible à la vente » est un **état d'attente** : le cycle de vie du stock est une boucle autour de lui.
- Chaque événement (vente, inventaire, achat reçu, annulation) crée **exactement un mouvement** ; la branche d'alerte n'en crée pas, elle se contente d'informer.
- L'ajustement impose un **motif** : c'est ce qui permet de répondre plus tard à « pourquoi le stock a-t-il changé ? ».

5.6 Activité 5 : envoi du reçu

Objectif	Remettre au client un justificatif de sa vente via WhatsApp, simplement.
Déclencheur	Vente validée, ou ouverture d'une vente depuis l'historique.
Ce que fait Doku	Génère le reçu, prépare un message, ouvre WhatsApp. **Le commerçant envoie lui-même** : c'est un partage, pas un envoi automatique.
Postconditions	Message prêt dans WhatsApp ; aucun accusé d'envoi côté Doku ; le numéro du client n'est pas conservé (H-08).
Règles liées	RG-15
Cas d'utilisation	UC27, UC28

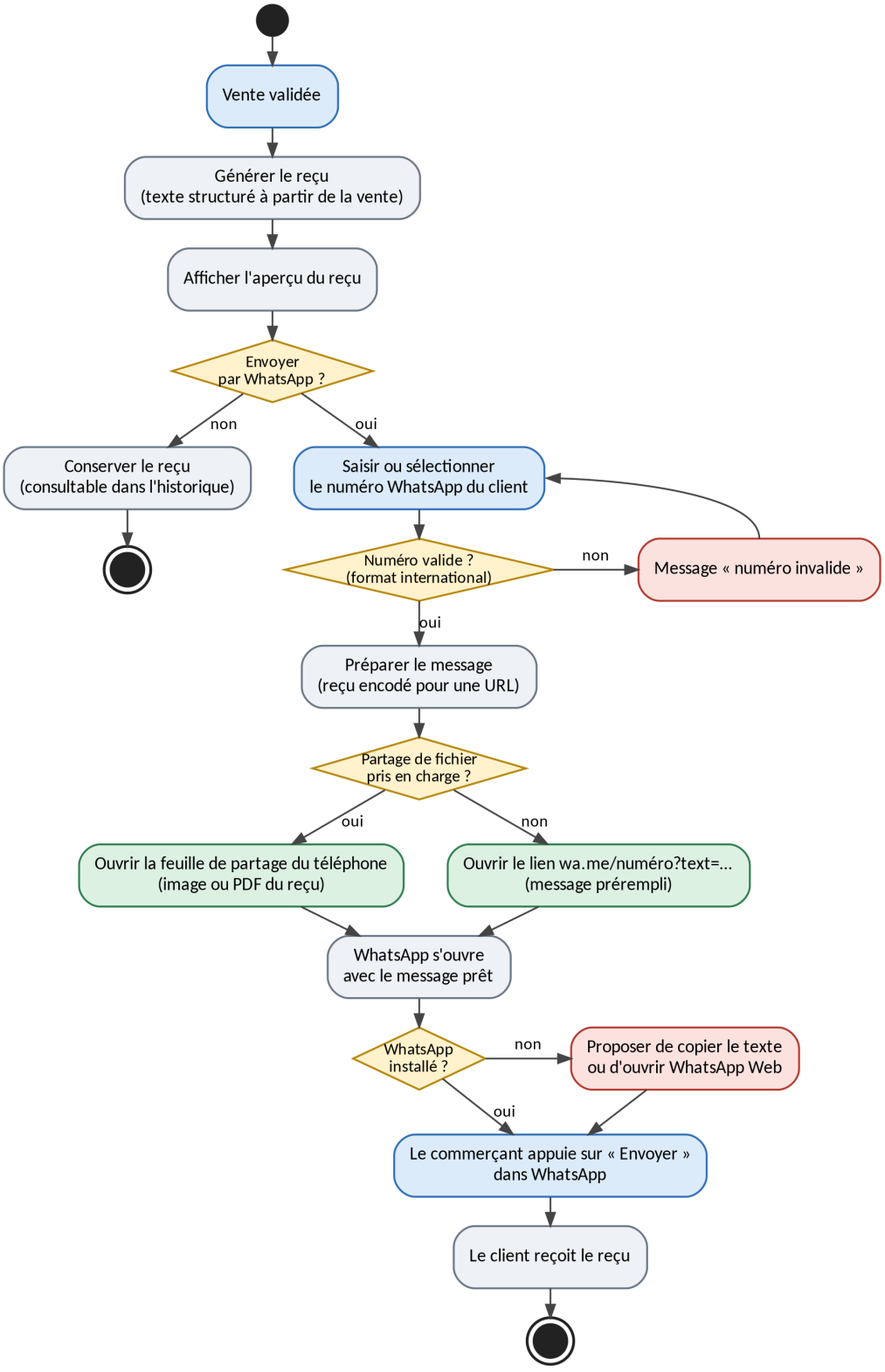


Figure 5.6 : Activité 5 : envoi du reçu par WhatsApp

Comment lire le diagramme

- La décision « Partage de fichier pris en charge ? » sépare deux techniques : la **feuille de partage** du téléphone (image ou PDF du reçu) et le **lien wa.me** (texte prérempli).
- Un numéro invalide reboucle vers la saisie ; l'absence de WhatsApp propose une solution de repli (copier le texte, WhatsApp Web).

- Le dernier geste est **humain** : tant que Doku n'utilise pas l'API officielle WhatsApp Business, il ne peut pas envoyer le message à la place du commerçant.

6. Diagrammes de séquence

6.1 Rôle des diagrammes et conventions de lecture

Un **diagramme de séquence** montre qui parle à qui, dans quel ordre, au cours d'un scénario. Là où l'activité décrit la logique d'un processus, la séquence décrit **les échanges entre composants** : c'est le document qui sert directement à écrire le code des services et des appels d'API.

- Chaque **colonne** est un participant (acteur, composant, base de données) ; sa **ligne pointillée verticale** est sa ligne de vie, le temps s'écoule vers le bas.
- **Flèche pleine** : un appel. **Flèche pointillée italique** : la réponse. **Boucle sur un participant** : un traitement interne.
- Les cadres **alt** (alternative : une seule branche s'exécute), **opt** (optionnel) et **loop** (répétition) portent leur condition entre crochets.
- Les messages sont **numérotés** ; les explications sous chaque figure y font référence.

6.2 Séquence 1 : enregistrer une vente

Participants	Commerçant → Interface Doku → Panier → Service de vente → Base locale / API → Service de stock → Service de reçu
Scénario	Ajout d'un produit au panier, choix de la devise, validation du paiement, enregistrement et reçu.
Idée forte	La confirmation vient de la **base locale**, jamais du serveur : la vente est immédiate même sans réseau.

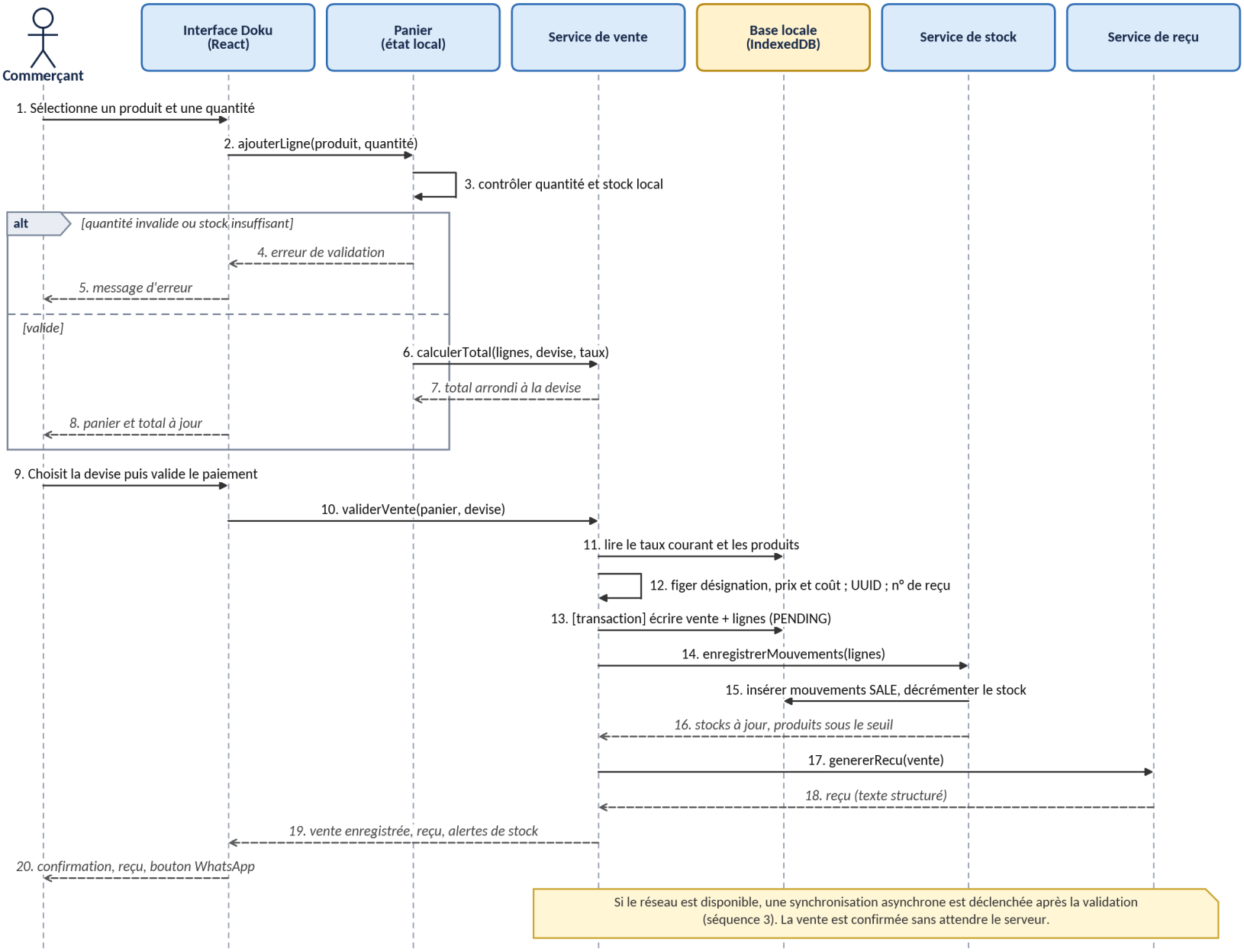


Figure 6.1 : Séquence 1 : enregistrer une vente (mode nominal et cas d'erreur de saisie)

Points clés

- **Messages 3 à 5** : le contrôle de quantité et de stock est fait *avant* tout calcul ; en cas d'erreur, le panier n'est pas modifié.
- **Message 12** : le service de vente fige les instantanés (désignation, prix converti, coût converti), génère l'UUID de la vente et le numéro de reçu : ces valeurs ne seront plus jamais recalculées (RG-09).
- **Messages 13 à 15** : vente, lignes et mouvements de stock sont écrits dans la même transaction ; le service de stock décrémente *current_stock* localement.
- **Message 16** : le retour contient les produits passés sous leur seuil, ce qui déclenche l'alerte de stock faible sans requête supplémentaire.
- La note finale rappelle que la synchronisation éventuelle est asynchrone et ne retarde jamais la confirmation.

6.3 Séquence 2 : vente hors ligne

Participants	Commerçant → Frontend → Stockage local → Service de synchronisation (+ Moniteur réseau et API Django)
Scénario	Une vente est faite sans Internet, puis la connexion revient et la vente est envoyée.
Idée forte	Le moniteur réseau ne décide rien : il **informe**. Le service de synchronisation est le seul à interroger l'API.

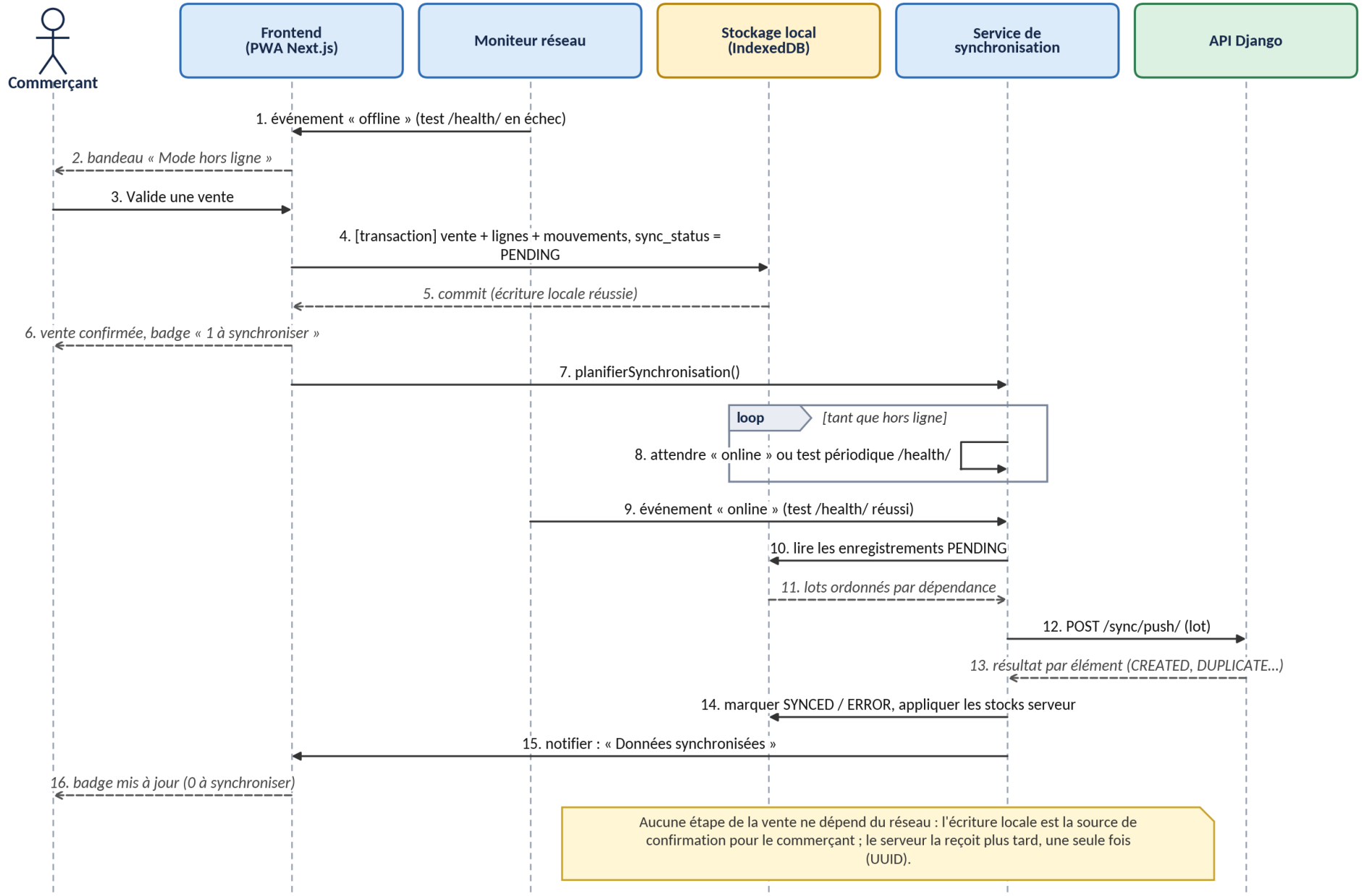


Figure 6.2 : Séquence 2 : vente hors ligne puis synchronisation au retour du réseau

Points clés

- **Messages 1 et 2** : l'application informe le commerçant (bandeau) mais ne bloque aucune action.
- **Message 4** : la vente est écrite avec **sync_status = PENDING**. C'est ce statut local, absent de la base serveur, qui matérialise le « à synchroniser ».
- **Cadre loop** : le service attend l'événement « online » *ou* un test périodique ; on ne dépend jamais d'un seul signal (les événements du navigateur sont parfois faux).
- **Messages 12 à 14** : l'API répond élément par élément ; le service met à jour chaque statut (SYNCED ou ERROR) et applique les stocks recalculés par le serveur.

6.4 Séquence 3 : synchronisation

Participants	Application → Service de synchronisation → Base locale → API Django → Base de données → Réponse → Application
Scénario	Envoi d'un lot (push) puis réception des changements (pull), avec renouvellement de jeton et gestion d'échec.
Idée forte	Chaque élément est traité de façon **idempotente** : le même lot rejoué donne le même état serveur.

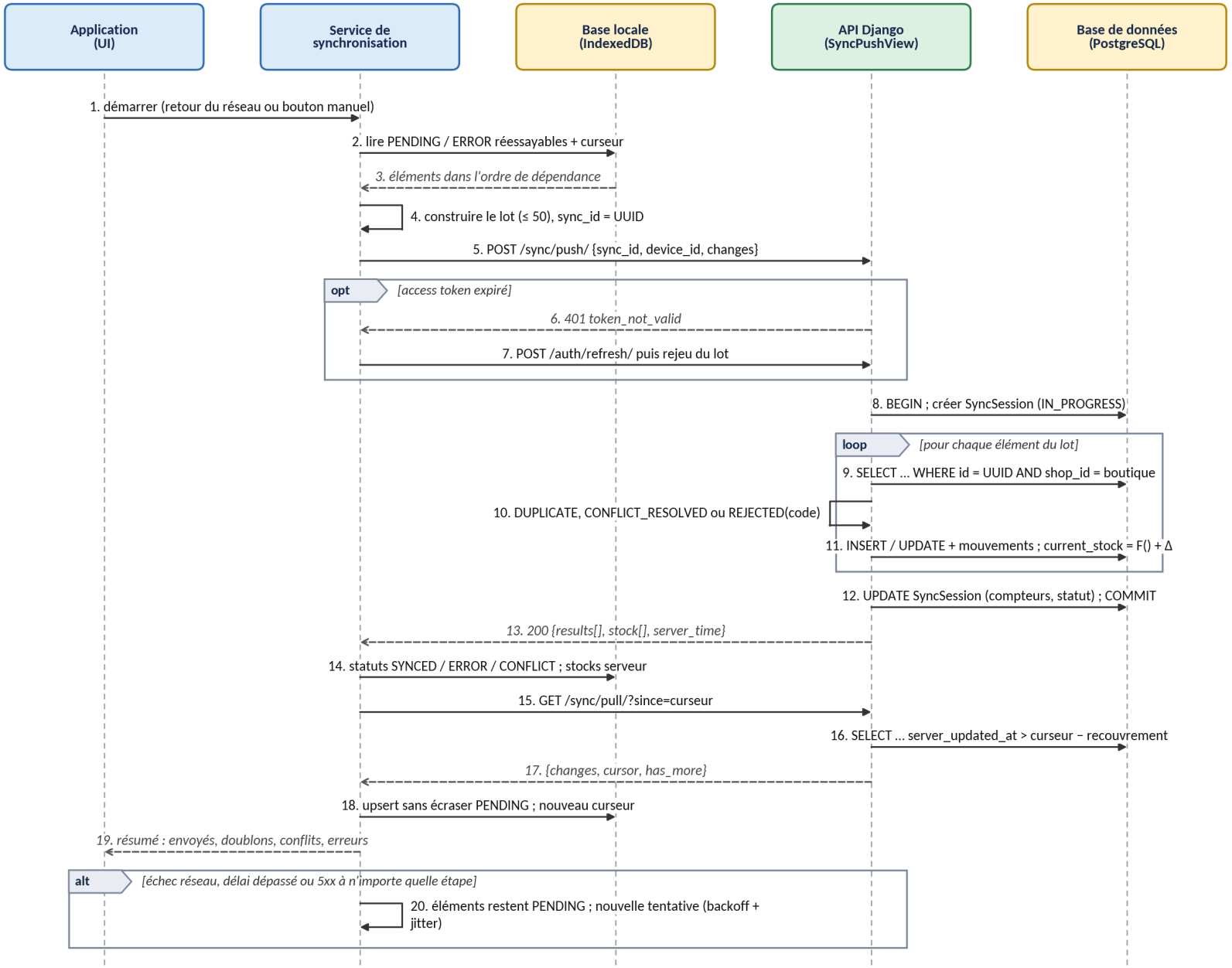


Figure 6.3 : Séquence 3 : synchronisation (push, traitement serveur, pull, échec réseau)

Points clés

- **Message 5** : le lot porte un *sync_id* (UUID) et un *device_id* ; il est limité à 50 éléments et ordonné par dépendance (taux, catégories, produits, mouvements, ventes).
- **Cadre opt** : un jeton expiré provoque un *401* ; le service renouvelle le jeton et **rejoue le même lot**, sans intervention de l'utilisateur.
- **Cadre loop** : pour chaque élément, le serveur vérifie l'UUID dans la boutique de l'utilisateur, applique la règle (DUPLICATE, CONFLICT_RESOLVED, REJECTED) puis écrit ; *current_stock* est modifié par une expression atomique (F() + variation), jamais par lecture puis écriture.
- **Messages 14 à 17** : le pull utilise un curseur basé sur l'**horloge du serveur** (jamais celle du téléphone), avec une marge de recouvrement : appliquer deux fois un même changement est sans effet.
- **Cadre alt final** : tout échec réseau laisse les éléments PENDING ; la reprise suit une attente exponentielle avec « jitter ».

6.5 Séquence 4 : authentification

Participants	Utilisateur → Frontend → API Django → Service JWT → Base de données
Scénario	Connexion, accès à une ressource protégée, expiration et renouvellement du jeton, déconnexion.
Choix	**Access token** court (15 min) conservé en mémoire ; **refresh token** rotatif en cookie **HttpOnly**, mis en liste noire à la rotation et à la déconnexion (chapitre 17).

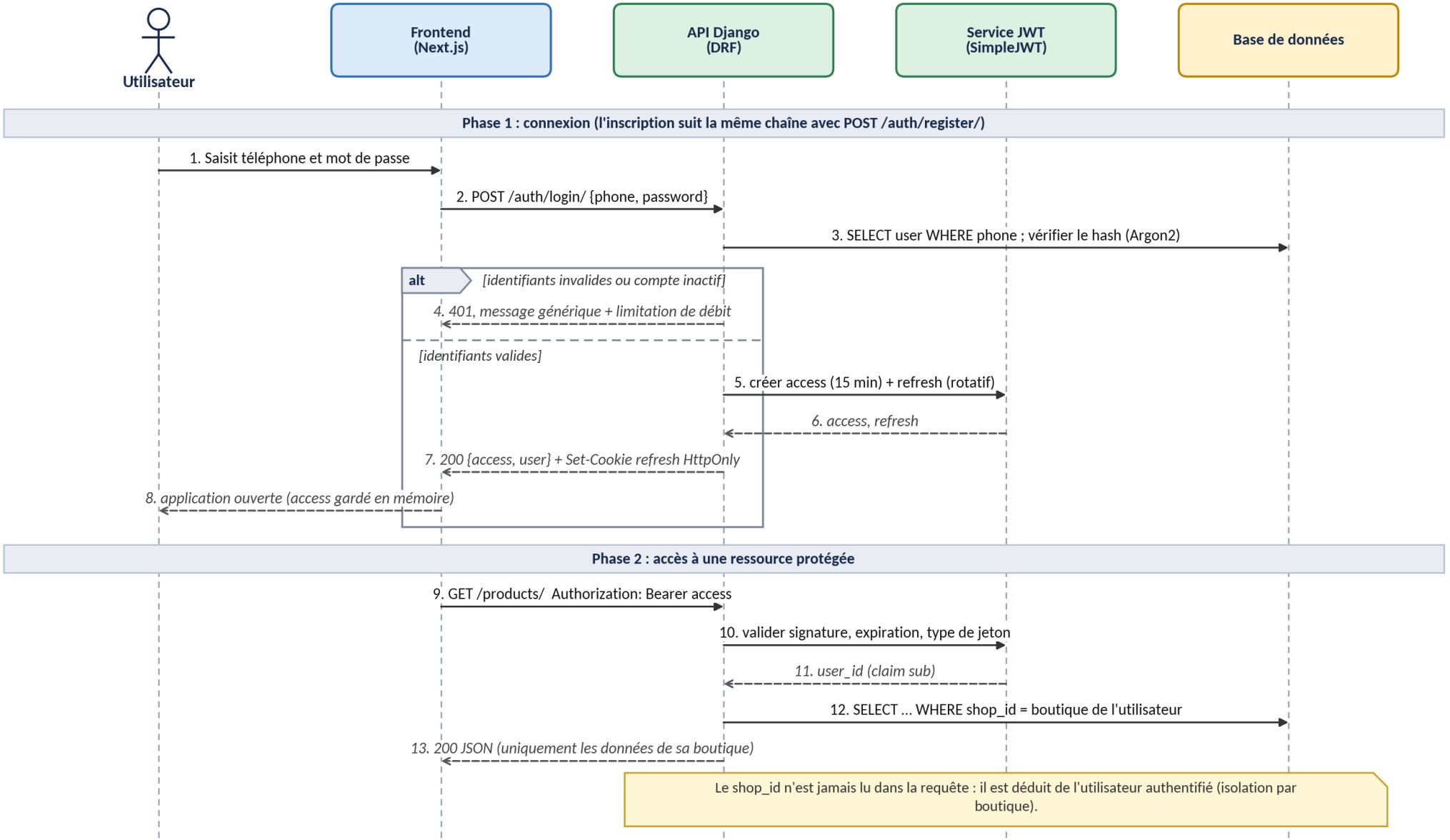


Figure 6.4 : Séquence 4 (1/2) : connexion et accès à une ressource protégée

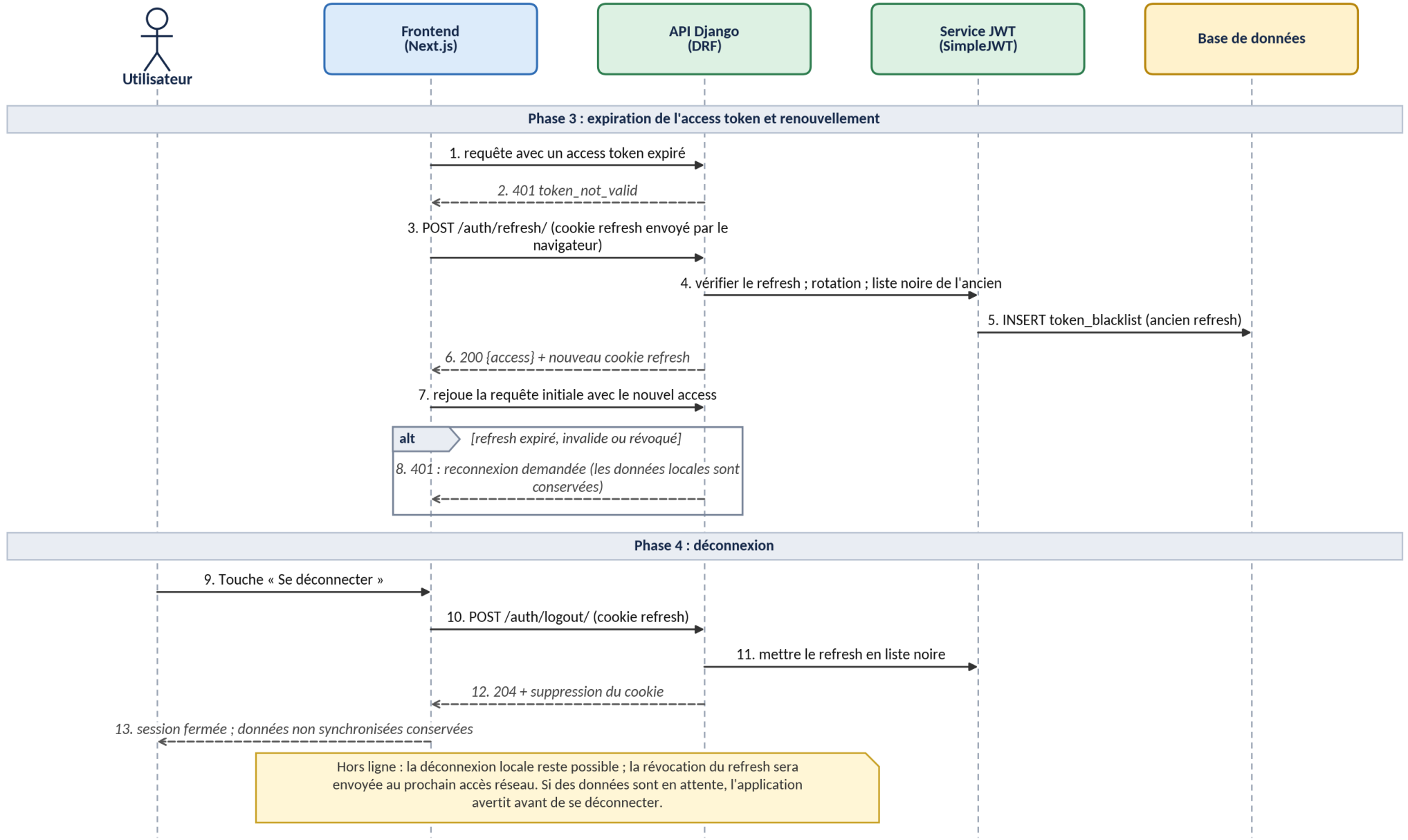


Figure 6.5 : Séquence 4 (2/2) : renouvellement du jeton et déconnexion

Points clés

- **Message d'échec de connexion** : volontairement générique (« identifiants invalides ») pour ne pas révéler si un numéro existe ; la limitation de débit freine les essais répétés.
- **Ressource protégée** : le *shop_id* n'est jamais lu dans la requête ; il est déduit de l'utilisateur authentifié. C'est la pierre angulaire de l'isolation entre boutiques.
- **Refresh rotatif** : à chaque renouvellement, l'ancien refresh est invalidé ; la réutilisation d'un refresh déjà consommé signale un vol possible.
- **Déconnexion** : si des données sont en attente, l'application prévient ; elle ne les supprime jamais. Hors ligne, la déconnexion locale reste possible et la révocation partira plus tard.

> **À retenir**
> L'inscription (UC01) suit la même chaîne que la connexion avec *POST /auth/register/* : le serveur crée l'utilisateur **et sa boutique dans une même transaction**, puis émet les jetons.

6.6 Séquence 5 : envoi d'un reçu via WhatsApp

Participants	Commerçant → Application Doku → Service de reçu → WhatsApp → Client
Scénario	Préparation du reçu et ouverture de WhatsApp avec le message prérempli.
Limite technique	Un lien wa.me ne permet **ni l'envoi automatique ni la confirmation de livraison** ; l'envoi final reste un geste du commerçant.

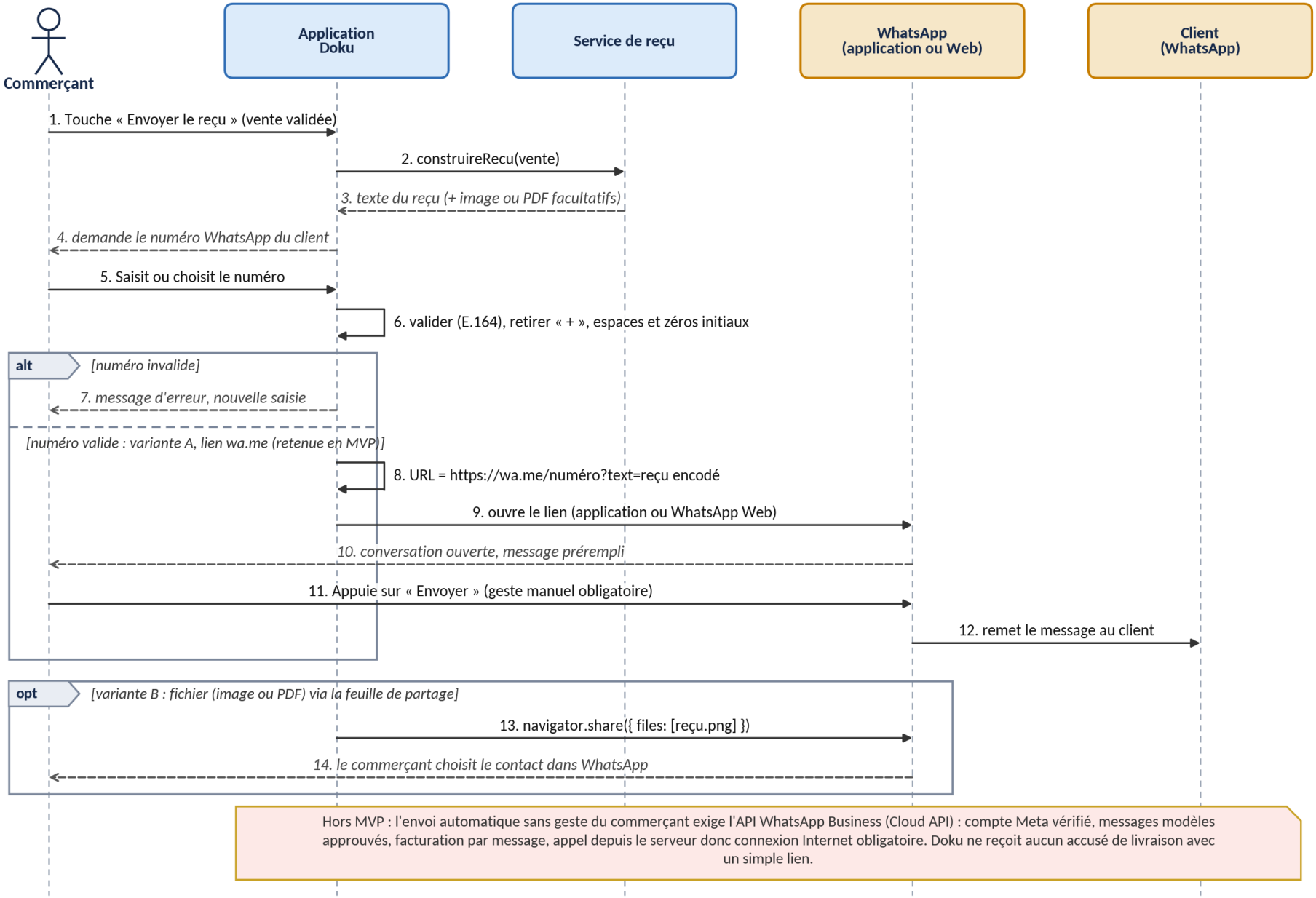


Figure 6.6 : Séquence 5 : envoi d'un reçu par WhatsApp (lien wa.me, partage de fichier, évolution API)

Ce qui est réalisé en MVP et ce qui exigerait l'API WhatsApp Business

	Partage par lien / feuille de partage (MVP)	API WhatsApp Business, Cloud API (hors MVP)
Mécanisme	Lien *wa.me/numéro?text=…* ou partage de fichier du téléphone	Appel HTTP depuis le serveur Doku vers la plateforme WhatsApp Business
Envoi	Le commerçant appuie sur « Envoyer » dans WhatsApp	Automatique, sans geste du commerçant
Réseau	Lien généré localement ; WhatsApp a sa propre connexion	Internet obligatoire côté serveur ; incompatible avec une vente hors ligne sans file d'attente dédiée
Prérequis	Aucun compte à créer	Compte Meta Business vérifié, numéro dédié, messages « modèle » approuvés pour écrire hors fenêtre de 24 h
Coût	Gratuit	Facturation par message selon la grille de Meta (à consulter à la date du projet)
Suivi	Aucun accusé de livraison	Statuts de livraison et de lecture disponibles
Données	Numéro du client non conservé	Numéros à stocker et consentement à gérer (impact sur le modèle : table client et envois)

Tableau 6.1 : Deux façons d'envoyer un reçu : ce qui change

Décision D10
Le MVP retient le **lien wa.me** : zéro coût, zéro dépendance, fonctionne partout. Le format du lien est *https://wa.me/<numéro>?text=<texte encodé>*, avec le numéro au format international sans « + », sans espaces ni zéros initiaux ; le texte doit être encodé pour une URL (encodeURIComponent).

7. Diagramme de classes UML

7.1 Rôle et conventions

Le **diagramme de classes** décrit les concepts du domaine (classes), leurs données (attributs), leurs comportements (opérations) et leurs liens (associations avec multiplicités). Il se situe entre le MCD (point de vue données) et le code : chaque classe devient un modèle Django, et les opérations deviennent des méthodes de modèle ou des fonctions de service.

- Une **boîte à trois compartiments** : nom (avec stéréotype), attributs, opérations. Les attributs facultatifs sont notés *[0..1]*.
- Une **ligne** est une association ; les chiffres aux extrémités sont les **multiplicités** (« 1 », « 0..* », « 1..* »). Le **losange plein** de Sale vers SaleItem signale une **composition** : une ligne n'existe pas sans sa vente.
- Une **flèche pointillée** relie une classe à une énumération. Les stéréotypes *syncable / mutable / append-only* indiquent le comportement de synchronisation de la classe (chapitre 15).
- Couleurs : bleu = compte et catalogue, vert = ventes et stock, sable = devises et taux, violet = journal technique.

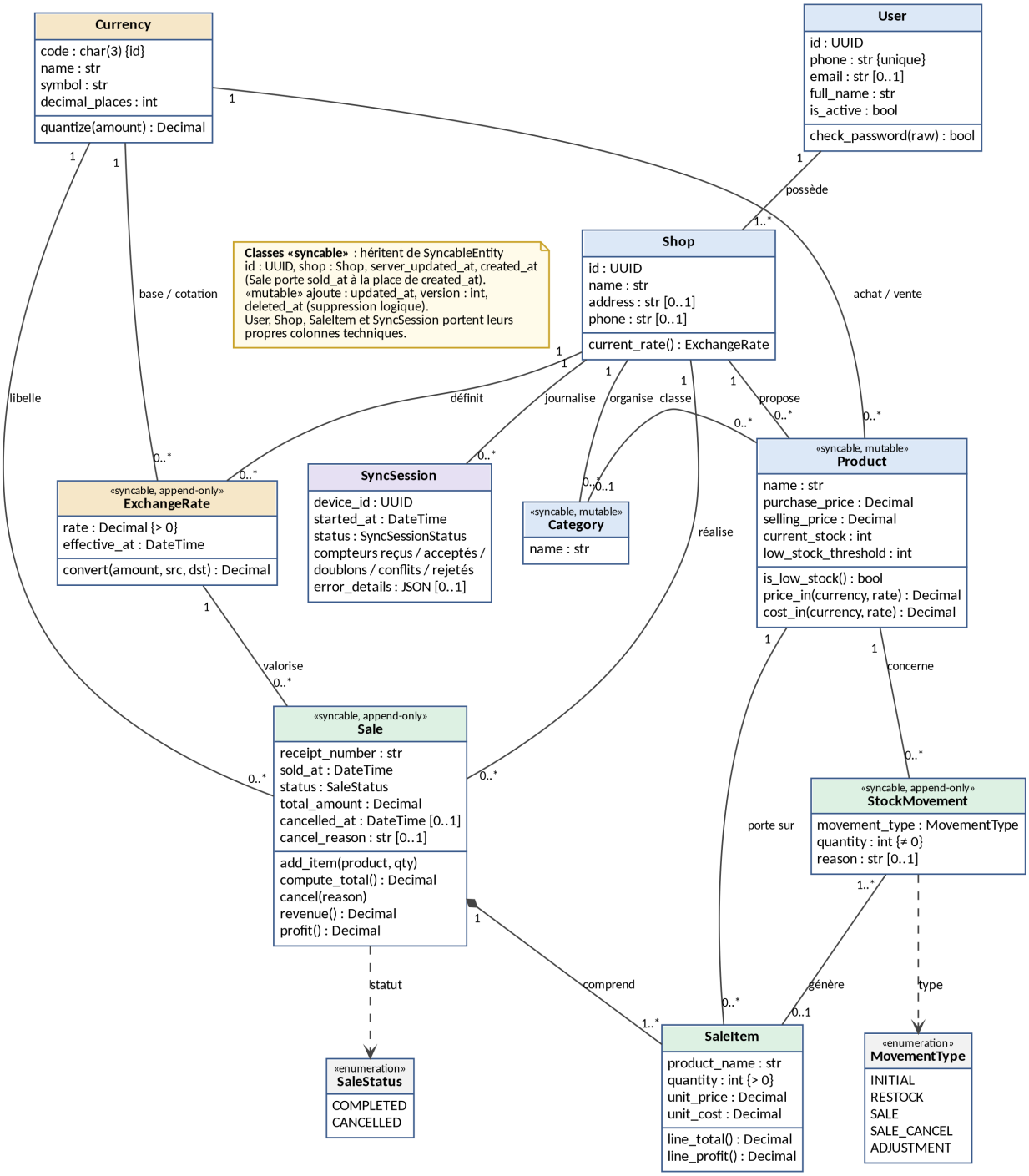


Figure 7.1 : Diagramme de classes du domaine Doku

7.2 Lecture et choix de conception

- **Hiérarchie de synchronisation.** Les classes qui voyagent entre l'appareil et le serveur héritent d'une classe de base abstraite (id UUID, boutique, dates). Celles qui peuvent être modifiées ajoutent version et suppression logique ; celles en ajout seul (taux, mouvements, ventes) n'en ont pas besoin. En Django : deux classes abstraites.
- **Sale compose SaleItem.** La vente est la racine d'agrégat : on n'écrit jamais une ligne sans sa vente, et les totaux se calculent sur la vente.
- **StockMovement ← SaleItem (0..1 – 1..*).** Chaque ligne de vente engendre au moins un mouvement (la sortie) et un second si la vente est annulée ; les mouvements manuels (réapprovisionnement, ajustement) n'ont pas de ligne.

- **Product.current_stock** figure dans la classe pour l'usage, mais il est *dérivé* : sa source de vérité est la somme des mouvements (chapitre 11).
- **ExchangeRate.convert()** et **Currency.quantize()** portent les règles de conversion et d'arrondi (RG-11, RG-12) : une seule implémentation, testée, utilisée par la caisse et par les statistiques.
- **SyncSession** est une classe technique, non synchronisée vers le client.

7.3 Opérations principales et services applicatifs

Les opérations « de domaine » (is_low_stock, line_total, profit…) sont de petites méthodes pures, faciles à tester. Les opérations qui touchent plusieurs objets ou la base (créer une vente, appliquer un mouvement, synchroniser) vivent dans des **services applicatifs**, pour éviter de mettre la logique métier dans les vues ou les serializers.

Service	Responsabilités	Côté serveur (Django)	Côté client (Next.js)
SaleService	Créer une vente (instantanés, total, mouvements) et l'annuler	sales/services.py	lib/services/sale.ts
StockService	Appliquer un mouvement, recalculer et réconcilier current_stock	inventory/services.py	lib/services/stock.ts
PricingService	Convertir un prix, arrondir à la devise, calculer coût et total	core/pricing.py	lib/pricing.ts (mêmes tests)
ReceiptService	Construire le reçu et le message WhatsApp	sales/receipts.py (JSON)	lib/services/receipt.ts
SyncService	Traiter un lot (push) et servir les changements (pull)	sync/services.py	lib/sync/engine.ts
AuthService	Inscription, connexion, renouvellement, déconnexion	accounts/services.py + SimpleJWT	lib/auth.ts (Axios)
StatsService	Chiffre d'affaires, coût, bénéfice, marge, séries	sales/stats.py	lib/services/stats.ts (IndexedDB)

Tableau 7.1 : Services applicatifs et leur emplacement dans le code

> **Bonne pratique**
> Écrire **PricingService** d'abord, avec un tableau de cas de test (prix, devise, taux, résultat attendu) partagé entre Python et TypeScript. C'est la meilleure protection contre un bénéfice qui diffère entre l'écran du téléphone et le rapport du serveur.

8. MCD Merise : modèle conceptuel de données

8.1 Démarche et lecture

Le **MCD** décrit les données du domaine indépendamment de toute technologie. Il se compose d'**entités** (objets du métier, chacun avec un identifiant souligné et des attributs), d'**associations** (liens nommés par un verbe) et de **cardinalités** (min,max) notées à chaque extrémité.

> **Comment lire**
> **Lire une cardinalité.** Pour l'association PROPOSE entre BOUTIQUE (0,N) et PRODUIT (1,1) : « une boutique propose de 0 à N produits » et « un produit est proposé par exactement une boutique ». Le premier chiffre dit si la participation est facultative (0) ou obligatoire (1) ; le second si elle est unique (1) ou multiple (N).

Méthode suivie : (1) extraire les candidats entités des exigences ; (2) éliminer ou fusionner ceux qui n'ont pas de justification propre ; (3) définir identifiants et attributs conceptuels (sans colonnes techniques ni valeurs calculables) ; (4) poser les associations et leurs cardinalités ; (5) contrôler le résultat en 3e forme normale.

8.2 Analyse des entités candidates

Le porteur du projet a proposé une liste d'entités possibles en demandant de **ne pas toutes les créer automatiquement**. Voici le tri, entité par entité : sur quinze candidates, dix sont retenues.

Entité candidate	Décision	Justification
Utilisateur	Retenue	Compte de connexion ; porte l'authentification.
Commerçant	Fusionnée avec Utilisateur	Un seul rôle en MVP (H-02) : l'association serait 1-1 et sans attribut propre. Séparer les deux créerait une table de plus sans gain. Un futur rôle se modélisera par une table d'appartenance.
Boutique	Retenue	Racine d'isolation des données (tenant), porte la devise de référence et l'identité du reçu. Rend le multi-boutique possible sans refonte (H-03).
Produit	Retenue	Cœur du catalogue ; porte prix et seuil.
Catégorie	Retenue	Demandée (« gérer les catégories »).
Stock	Non retenue : dérivée	Un stock par produit = relation 1-1 avec Produit, sans attribut propre. Le stock courant est une valeur *calculable* à partir des mouvements ; il est matérialisé dans product.current_stock au niveau logique (redondance contrôlée).
Mouvement de stock	Retenue	Indispensable à la traçabilité (« pourquoi le stock a changé ») et à la synchronisation par variations.
Vente	Retenue	Cœur de la caisse.
Ligne de vente	Retenue (entité promue)	Porte un identifiant propre référencé par les mouvements de stock. En Merise, une association ne peut pas être reliée à une autre association : « Vente contient Produit » est donc promue en entité.
Paiement	Non retenue en MVP	Vente encaissée en totalité, en une devise (H-06) : le paiement serait une copie du total. À introduire pour les paiements mixtes, partiels ou mobile money : Paiement (1,1)–(0,N) Vente.
Devise	Retenue (référence)	Possède ses propres propriétés (symbole, précision décimale) et est référencée par six associations : c'est une entité, non un simple attribut.
Taux de change	Retenue (ajout)	Historique daté en ajout seul ; garantit que les anciennes ventes ne changent pas quand le taux bouge.

Entité candidate	Décision	Justification
Client	**Non retenue en MVP**	Pas de fichier client (H-08) : le numéro WhatsApp est saisi à l'envoi et non conservé (minimisation des données personnelles). Évolution : Client + lien optionnel avec Vente.
Reçu	**Non retenue : document dérivé**	Un reçu est une *représentation* d'une vente : toutes ses données existent déjà (vente, lignes, boutique). Son numéro devient un attribut de Vente. Une table de reçus ne se justifierait que pour suivre des envois (API WhatsApp Business).
Synchronisation	**Retenue (technique)**	Journal des lots reçus par le serveur : support, supervision, preuve. N'est pas synchronisée vers le client.

Tableau 8.1 : Tri des entités candidates

8.3 Entités retenues

Pour chaque entité : identifiant, attributs conceptuels (avec la colonne du MLD qui les porte), puis relations et cardinalités. Les colonnes techniques (dates de synchronisation, version, suppression logique, boutique) n'apparaissent pas ici : elles sont ajoutées au niveau logique (chapitre 9).

8.3.1 UTILISATEUR

Personne qui se connecte à Doku. En MVP, un utilisateur est toujours le commerçant propriétaire de sa boutique.

Attribut	Type	Description	Colonne MLD
id_utilisateur (identifiant)	UUID	Identifiant unique	app_user.id
nom_complet	Texte	Nom du commerçant	app_user.full_name
telephone	Texte	Identifiant de connexion (E.164)	app_user.phone
email	Texte	Adresse e-mail facultative	app_user.email
mot_de_passe	Texte	Empreinte du mot de passe	app_user.password
actif	Booléen	Compte actif	app_user.is_active

Relations : POSSEDE : UTILISATEUR **(1,N)** — BOUTIQUE **(1,1)**

8.3.2 BOUTIQUE

Entité commerciale ; racine d'isolation de toutes les données métier (tenant).

Attribut	Type	Description	Colonne MLD
id_boutique (identifiant)	UUID	Identifiant unique	shop.id
nom	Texte	Nom commercial	shop.name
adresse	Texte	Adresse / localisation	shop.address
telephone	Texte	Téléphone de la boutique	shop.phone

Relations : POSSEDE : BOUTIQUE **(1,1)** — UTILISATEUR **(1,N)** · **PROPOSE** : BOUTIQUE **(0,N)** — PRODUIT **(1,1)** · **ORGANISE** : BOUTIQUE **(0,N)** — CATEGORIE **(1,1)** · **REALISE** : BOUTIQUE **(0,N)** — VENTE **(1,1)** · **DEFINIT** : BOUTIQUE **(0,N)** — TAUX_CHANGE **(1,1)** · **REFERENCE** : BOUTIQUE **(1,1)** — DEVISE **(0,N)** · **SYNCHRONISE** : BOUTIQUE **(0,N)** — SYNCHRONISATION **(1,1)**

8.3.3 CATEGORIE

Regroupement libre de produits défini par le commerçant.

Attribut	Type	Description	Colonne MLD
id_categorie (identifiant)	UUID	Identifiant unique	category.id
nom	Texte	Nom de la catégorie	category.name

Relations : ORGANISE : CATEGORIE **(1,1)** — BOUTIQUE **(0,N)** · **CLASSE** : CATEGORIE **(0,N)** — PRODUIT **(0,1)**

8.3.4 PRODUIT

Article proposé à la vente. Le stock courant n'apparaît pas ici : il est calculable à partir des mouvements.

Attribut	Type	Description	Colonne MLD
id_produit (identifiant)	UUID	Identifiant unique	product.id
nom	Texte	Désignation	product.name
prix_achat	Décimal	Prix d'achat unitaire courant	product.purchase_price
prix_vente	Décimal	Prix de vente unitaire courant	product.selling_price
seuil_alerte	Entier	Seuil de stock faible	product.low_stock_threshold

Relations : PROPOSE : PRODUIT **(1,1)** — BOUTIQUE **(0,N)** · **CLASSE** : PRODUIT **(0,1)** — CATEGORIE **(0,N)** · **CONCERNE** : PRODUIT **(0,N)** — MOUVEMENT_STOCK **(1,1)** · **PORTE_SUR** : PRODUIT **(0,N)** — LIGNE_VENTE **(1,1)** · **ACHETE_EN** : PRODUIT **(1,1)** — DEVISE **(0,N)** · **VEND_EN** : PRODUIT **(1,1)** — DEVISE **(0,N)**

8.3.5 MOUVEMENT_STOCK

Une variation de stock tracée et datée. Permet de retrouver pourquoi le stock a changé.

Attribut	Type	Description	Colonne MLD
id_mouvement (identifiant)	UUID	Identifiant unique	stock_movement.id
type_mouvement	Énumération	INITIAL, RESTOCK, SALE, SALE_CANCEL, ADJUSTMENT	stock_movement.movement_type
quantite	Entier	Variation signée (+ entrée, − sortie)	stock_movement.quantity
motif	Texte	Cause (obligatoire pour un ajustement)	stock_movement.reason
date_mouvement	Date/heure	Instant du mouvement	stock_movement.created_at

Relations : CONCERNE : MOUVEMENT_STOCK **(1,1)** — PRODUIT **(0,N)** · **GENERE** : MOUVEMENT_STOCK **(0,1)** — LIGNE_VENTE **(1,N)**

8.3.6 VENTE

Transaction validée entre la boutique et un client anonyme. Le total est calculable à partir des lignes.

Attribut	Type	Description	Colonne MLD
id_vente (identifiant)	UUID	Identifiant unique	sale.id
numero_recu	Texte	Numéro de reçu lisible	sale.receipt_number
date_vente	Date/heure	Instant de validation	sale.sold_at
statut	Énumération	COMPLETED ou CANCELLED	sale.status
date_annulation	Date/heure	Instant d'annulation	sale.cancelled_at
motif_annulation	Texte	Motif d'annulation	sale.cancel_reason

Relations : REALISE : VENTE **(1,1)** — BOUTIQUE **(0,N)** · **COMPREND** : VENTE **(1,N)** — LIGNE_VENTE **(1,1)** · **VALORISE** : VENTE **(1,1)** — TAUX_CHANGE **(0,N)** · **LIBELLE** : VENTE **(1,1)** — DEVISE **(0,N)**

8.3.7 LIGNE_VENTE

Ligne d'une vente. Promue en entité (et non en simple association) car elle porte un identifiant propre référencé par les mouvements de stock.

Attribut	Type	Description	Colonne MLD
id_ligne (identifiant)	UUID	Identifiant unique	sale_item.id
designation	Texte	Désignation figée	sale_item.product_name
quantite	Entier	Quantité vendue	sale_item.quantity
prix_unitaire	Décimal	Prix unitaire figé (devise de la vente)	sale_item.unit_price
cout_unitaire	Décimal	Coût unitaire figé (devise de la vente)	sale_item.unit_cost

Relations : COMPREND : LIGNE_VENTE **(1,1)** — VENTE **(1,N)** · **PORTE_SUR** : LIGNE_VENTE **(1,1)** — PRODUIT **(0,N)** · **GENERE** : LIGNE_VENTE **(1,N)** — MOUVEMENT_STOCK **(0,1)**

8.3.8 TAUX_CHANGE

Taux saisi par le commerçant à une date donnée. Historique en ajout seul.

Attribut	Type	Description	Colonne MLD
id_taux (identifiant)	UUID	Identifiant unique	exchange_rate.id
taux	Décimal	Unités de la devise cotée pour 1 unité de la devise de base	exchange_rate.rate
date_effet	Date/heure	Entrée en vigueur	exchange_rate.effective_at

Relations : DEFINIT : TAUX_CHANGE **(1,1)** — BOUTIQUE **(0,N)** · **VALORISE** : TAUX_CHANGE **(0,N)** — VENTE **(1,1)** · **A_POUR_BASE** : TAUX_CHANGE **(1,1)** — DEVISE **(0,N)** · **A_POUR_COTATION** : TAUX_CHANGE **(1,1)** — DEVISE **(0,N)**

8.3.9 DEVISE

Devise gérée par Doku (CDF, USD). Entité de référence.

Attribut	Type	Description	Colonne MLD
code_devise (identifiant)	Texte(3)	Identifiant unique	currency.code
libelle	Texte	Nom de la devise	currency.name
symbole	Texte	Symbole	currency.symbol
decimales	Entier	Précision décimale	currency.decimal_places

Relations : ACHETE_EN : DEVISE **(0,N)** — PRODUIT **(1,1)** · **VEND_EN** : DEVISE **(0,N)** — PRODUIT **(1,1)** · **LIBELLE** : DEVISE **(0,N)** — VENTE **(1,1)** · **REFERENCE** : DEVISE **(0,N)** — BOUTIQUE **(1,1)** · **A_POUR_BASE** : DEVISE **(0,N)** — TAUX_CHANGE **(1,1)** · **A_POUR_COTATION** : DEVISE **(0,N)** — TAUX_CHANGE **(1,1)**

8.3.10 SYNCHRONISATION

Lot de synchronisation journalisé côté serveur (technique).

Attribut	Type	Description	Colonne MLD
id_synchro (identifiant)	UUID	Identifiant unique	sync_session.id
id_appareil	UUID	Appareil émetteur	sync_session.device_id
date_debut	Date/heure	Début	sync_session.started_at
date_fin	Date/heure	Fin	sync_session.finished_at
statut	Énumération	Statut du lot	sync_session.status
nb_recus	Entier	Éléments reçus	sync_session.received_count
nb_acceptes	Entier	Éléments acceptés	sync_session.accepted_count
nb_doublons	Entier	Doublons	sync_session.duplicate_count
nb_conflits	Entier	Conflits résolus	sync_session.conflict_count
nb_rejetes	Entier	Éléments rejetés	sync_session.rejected_count
erreurs	JSON	Détails des anomalies	sync_session.error_details

Relations : SYNCHRONISE : SYNCHRONISATION **(1,1)** — BOUTIQUE **(0,N)**

8.4 Associations et cardinalités

Association	Entité 1	Card.	Entité 2	Card.	Lecture
POSSEDE	UTILISATEUR	1,N	BOUTIQUE	1,1	Un utilisateur possède au moins une boutique (exactement une en MVP) ; une boutique appartient à un seul utilisateur.
PROPOSE	BOUTIQUE	0,N	PRODUIT	1,1	Une boutique propose 0 à N produits ; un produit appartient à une seule boutique.
ORGANISE	BOUTIQUE	0,N	CATEGORIE	1,1	Une boutique définit 0 à N catégories ; une catégorie appartient à une boutique.
CLASSE	CATEGORIE	0,N	PRODUIT	0,1	Un produit a 0 ou 1 catégorie ; une catégorie regroupe 0 à N produits.
CONCERNE	PRODUIT	0,N	MOUVEMENT_STOCK	1,1	Un mouvement concerne exactement un produit ; un produit a 0 à N mouvements.
REALISE	BOUTIQUE	0,N	VENTE	1,1	Une boutique réalise 0 à N ventes ; une vente est réalisée par une boutique.
COMPREND	VENTE	1,N	LIGNE_VENTE	1,1	Une vente comprend au moins une ligne ; une ligne appartient à une seule vente.
PORTE_SUR	PRODUIT	0,N	LIGNE_VENTE	1,1	Une ligne porte sur exactement un produit ; un produit figure dans 0 à N lignes.
GENERE	LIGNE_VENTE	1,N	MOUVEMENT_STOCK	0,1	Une ligne génère au moins un mouvement (la sortie) et un second en cas d'annulation ; un mouvement provient de 0 ou 1 ligne.
DEFINIT	BOUTIQUE	0,N	TAUX_CHANGE	1,1	Une boutique définit 0 à N taux ; un taux appartient à une boutique.
VALORISE	TAUX_CHANGE	0,N	VENTE	1,1	Chaque vente est valorisée avec exactement un taux ; un taux valorise 0 à N ventes.
ACHETE_EN	DEVISE	0,N	PRODUIT	1,1	Le prix d'achat d'un produit est exprimé dans exactement une devise.
VEND_EN	DEVISE	0,N	PRODUIT	1,1	Le prix de vente d'un produit est exprimé dans exactement une devise.
LIBELLE	DEVISE	0,N	VENTE	1,1	Une vente est libellée dans exactement une devise.
REFERENCE	DEVISE	0,N	BOUTIQUE	1,1	Une boutique a exactement une devise de référence.
A_POUR_BASE	DEVISE	0,N	TAUX_CHANGE	1,1	Un taux a exactement une devise de base.
A_POUR_COTATION	DEVISE	0,N	TAUX_CHANGE	1,1	Un taux a exactement une devise cotée.
SYNCHRONISE	BOUTIQUE	0,N	SYNCHRONISATION	1,1	Une boutique a 0 à N lots de synchronisation journalisés.

Tableau 8.2 : Les 18 associations du MCD

Pourquoi six associations vers Devise

Les associations vers DEVISE sont nombreuses (six) mais toutes de même nature : « un montant est exprimé dans une devise ». Elles se traduiront par six clés étrangères de type CHAR(3). C'est le prix d'une modélisation sans redondance et sans « devise » codée en dur.

8.5 Données calculables exclues du MCD

Donnée	Calcul	Traitement
Stock courant d'un produit	Σ mouvement.quantité	Exclue du MCD ; colonne dénormalisée product.current_stock au niveau logique (performance, hors ligne)
Stock initial	Premier mouvement de type INITIAL	Pas un attribut : une ligne du journal
Total d'une vente	Σ (quantité × prix unitaire)	Exclue du MCD ; colonne sale.total_amount au niveau logique (redondance contrôlée)
Montant converti	montant × taux de la vente	Jamais stocké : dérivable de façon déterministe
Chiffre d'affaires, coût, bénéfice, marge	Agrégats sur les lignes COMPLETED	Jamais stockés ; calculés à la demande (client et serveur)

Tableau 8.3 : Propriétés calculables : pourquoi elles ne sont pas dans le MCD

8.6 Diagrammes MCD

Quatre figures : la **vue globale** montre les dix entités et les dix-huit associations ; trois **vues détaillées** montrent les attributs. Les rectangles à bandeau sombre sont les entités (identifiant souligné), les ellipses vertes sont les associations, les nombres rouges sont les cardinalités.

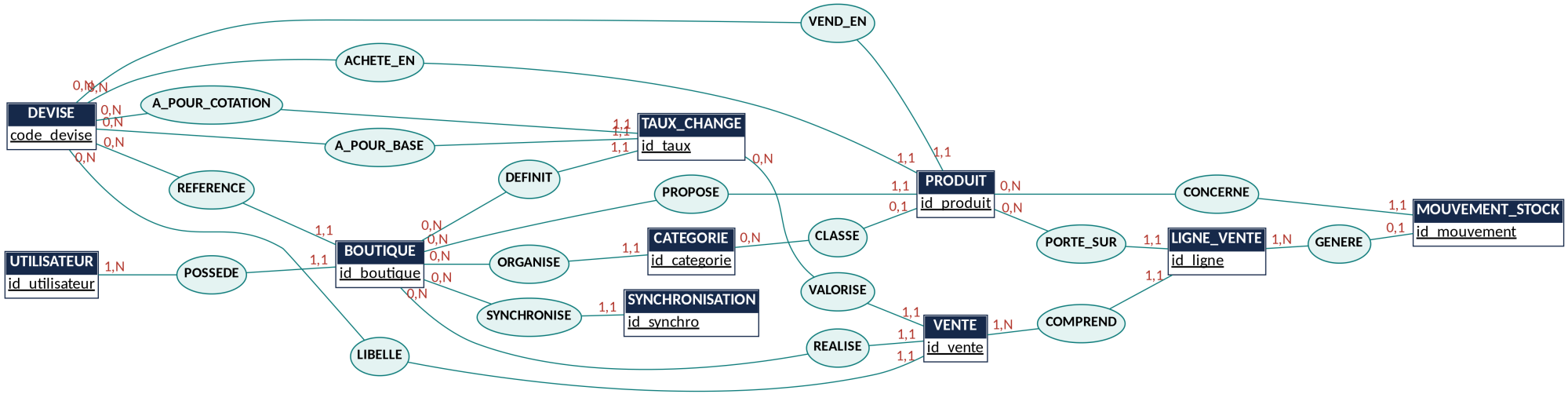


Figure 8.1 : MCD global : 10 entités, 18 associations (identifiants seuls)

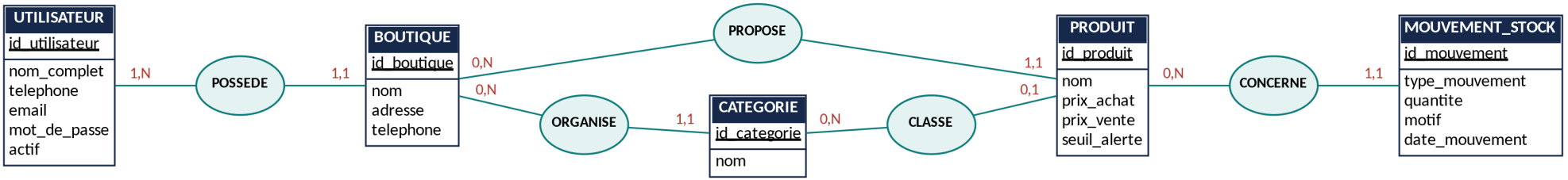


Figure 8.2 : MCD détaillé (1/3) : compte, boutique, catalogue et mouvements

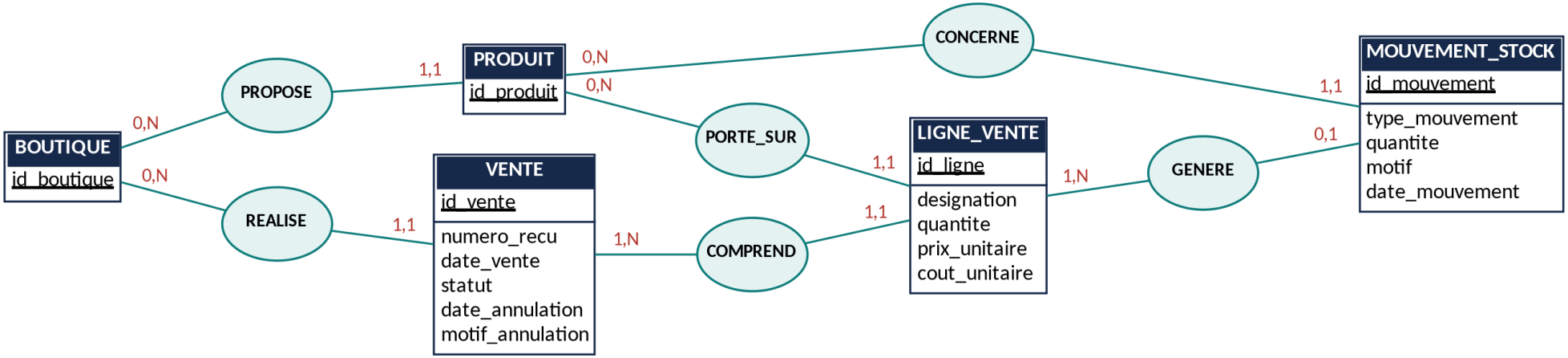


Figure 8.3 : MCD détaillé (2/3) : ventes, lignes et mouvements de stock

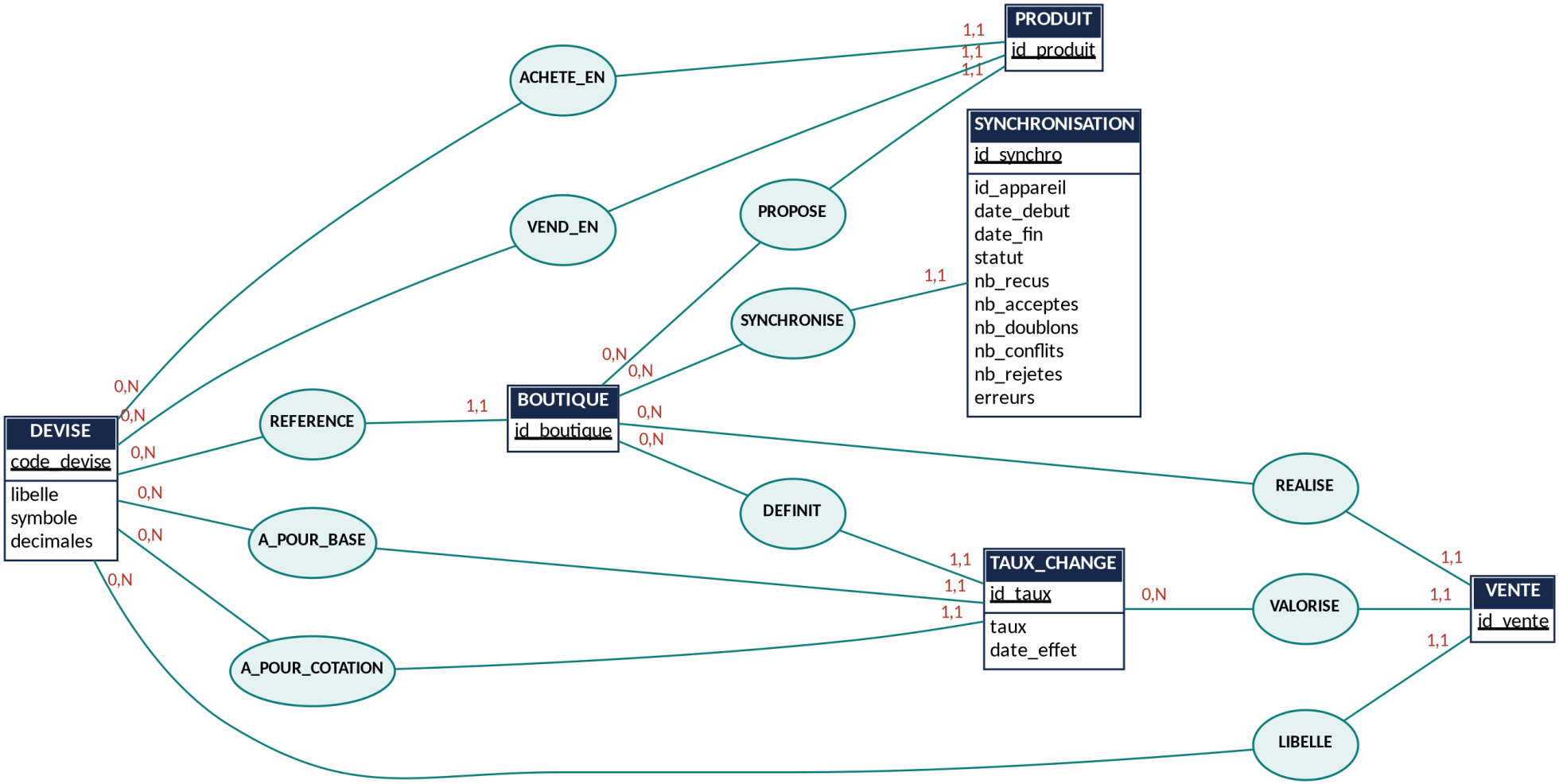


Figure 8.4 : MCD détaillé (3/3) : devises, taux de change et synchronisation

8.7 Explication du modèle

- **Autour de la boutique** : utilisateur, catégories, produits, ventes, taux et lots de synchronisation sont tous rattachés à BOUTIQUE (cardinalité 1,1 de leur côté) : c'est l'isolation des données.
- **Chaîne de la vente** : VENTE (1,N) COMPREND LIGNE_VENTE (1,1) ; une vente a toujours au moins une ligne. LIGNE_VENTE PORTE_SUR un PRODUIT et GENERE au moins un MOUVEMENT_STOCK.
- **Traçabilité du stock** : tout MOUVEMENT_STOCK CONCERNE exactement un produit ; il provient d'une ligne de vente (0,1) ou d'une opération manuelle (aucune ligne).
- **Cohérence financière** : VENTE est VALORISEE par exactement un TAUX_CHANGE ; le taux étant en ajout seul, la valeur historique ne peut plus changer.

9. MLD Merise : modèle logique de données

9.1 Règles de transformation MCD → MLD

Règle	Application dans Doku
Chaque entité devient une table ; son identifiant devient la clé primaire (PK).	10 entités → 10 tables ; PK = UUID, sauf currency (PK = code ISO).
Une association dont une patte est (x,1) disparaît : la clé de l'autre entité devient clé étrangère (FK) dans la table du côté (x,1).	17 associations sur 18 produisent une FK (ex. PROPOSE → product.shop_id). L'association GENERE (0,1) produit stock_movement.sale_item_id, nullable.
Une cardinalité minimale 0 côté (x,1) rend la FK facultative (NULL autorisé) ; 1 la rend obligatoire (NOT NULL).	CLASSE (0,1) → product.category_id NULL ; toutes les autres FK sont NOT NULL.
Une association (N,N) devient une table d'association à clé composée.	Aucune association (N,N) : LIGNE_VENTE a été promue en entité.
On ajoute au niveau logique les colonnes techniques et les redondances contrôlées.	Section 9.7 et 9.8.

Tableau 9.1 : Règles de passage appliquées

9.2 Schéma relationnel (notation Merise)

> **Nom de table**
> La table des utilisateurs se nomme **app_user** et non « user » : *user* est un mot réservé de PostgreSQL (il oblige à mettre des guillemets dans toutes les requêtes SQL manuelles). Django met les guillemets automatiquement, mais un script SQL, un outil de reporting ou une requête brute les oublieront.

Notation : **clé primaire** soulignée en gras, **#clé_étrangère** précédée de #. Les noms sont ceux de la base, de Django et de l'API.

CURRENCY (**code**, name, symbol, decimal_places)

APP_USER (**id**, phone, email, password, full_name, is_active, is_staff, is_superuser, last_login, created_at, updated_at)

SHOP (**id**, #owner_id, name, address, phone, #reference_currency_code, created_at, updated_at)

CATEGORY (**id**, #shop_id, name, created_at, updated_at, server_updated_at, version, deleted_at)

PRODUCT (**id**, #shop_id, #category_id, name, purchase_price, #purchase_currency_code, selling_price, #selling_currency_code, current_stock, low_stock_threshold, created_at, updated_at, server_updated_at, version, deleted_at)

EXCHANGE_RATE (**id**, #shop_id, #base_currency_code, #quote_currency_code, rate, effective_at, created_at, server_updated_at)

SALE (**id**, #shop_id, #exchange_rate_id, #currency_code, receipt_number, sold_at, status, total_amount, cancelled_at, cancel_reason, server_updated_at)

SALE_ITEM (**id**, #sale_id, #product_id, product_name, quantity, unit_price, unit_cost)

STOCK_MOVEMENT (**id**, #shop_id, #product_id, #sale_item_id, movement_type, quantity, reason, created_at, server_updated_at)

SYNC_SESSION (**id**, #shop_id, device_id, started_at, finished_at, status, received_count, accepted_count, duplicate_count, conflict_count, rejected_count, error_details)

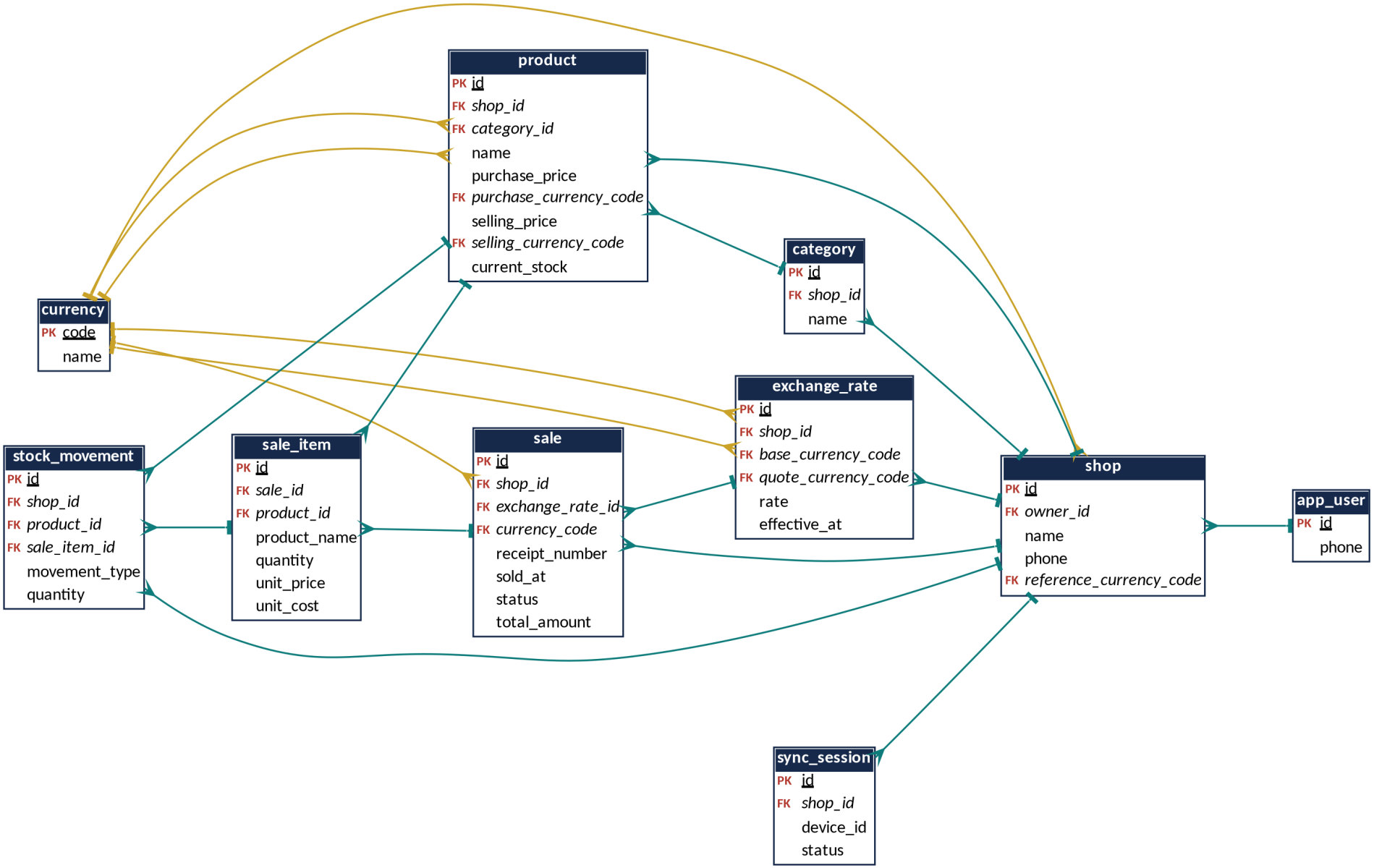


Figure 9.1 : Schéma relationnel : clés primaires (PK) et étrangères (FK), traits verts = liens métier, traits dorés = devises

9.3 Spécification des tables

Pour chaque table : rôle, nature de synchronisation, clé primaire, clés étrangères avec leur règle de suppression, contraintes et index. Les colonnes détaillées (type, taille, exemple) sont dans le dictionnaire de données (chapitre 10).

9.3.1 currency (Devise)

Rôle	Référentiel des devises gérées (CDF, USD). Porte la précision décimale utilisée pour arrondir et afficher les montants. Alimentée par une migration de données ; non modifiable par l'utilisateur ; embarquée en dur dans le client.
Synchronisation	Référence
Clé primaire	code (code ISO 4217)
Clés étrangères	—
Contraintes	CHECK (decimal_places BETWEEN 0 AND 3)
Index	—

9.3.2 app_user (Utilisateur)

Rôle	Compte de connexion du commerçant. Fusionne les notions « utilisateur » et « commerçant » (un seul rôle en MVP). Créé en ligne uniquement ; le client n'en conserve qu'une copie en lecture.
Synchronisation	Serveur (pull)
Clé primaire	id (UUID)
Clés étrangères	—
Contraintes	UNIQUE (phone) UNIQUE (email) — plusieurs NULL autorisés
Index	—

9.3.3 shop (Boutique)

Rôle	Entité commerciale du commerçant : c'est la racine d'isolation (tenant) de toutes les données métier. En MVP : une boutique par compte (contrainte d'unicité sur owner_id, levable plus tard).
Synchronisation	Serveur (pull)
Clé primaire	id (UUID)
Clés étrangères	owner_id → app_user ON DELETE PROTECT reference_currency_code → currency ON DELETE PROTECT
Contraintes	UNIQUE (owner_id) -- règle MVP « une boutique par commerçant »
Index	—

9.3.4 category (Catégorie)

Rôle	Regroupement libre des produits d'une boutique (Boissons, Hygiène…). Créée et modifiée hors ligne ; supprimée logiquement (les produits perdent simplement leur catégorie).
Synchronisation	Modifiable
Clé primaire	id (UUID)
Clés étrangères	shop_id → shop ON DELETE PROTECT
Contraintes	CHECK (version >= 1)
Index	(shop_id, server_updated_at) (shop_id, deleted_at)

9.3.5 product (Produit)

Rôle	Article vendu par la boutique. Porte les prix catalogue (achat et vente, chacun dans sa devise) et le stock courant (colonne dénormalisée, calculée par le serveur à partir des mouvements). Archivé et non supprimé physiquement.
Synchronisation	Modifiable
Clé primaire	id (UUID)
Clés étrangères	shop_id → shop ON DELETE PROTECT category_id → category ON DELETE SET NULL purchase_currency_code → currency ON DELETE PROTECT selling_currency_code → currency ON DELETE PROTECT
Contraintes	CHECK (purchase_price >= 0 AND selling_price >= 0) CHECK (low_stock_threshold >= 0) CHECK (version >= 1)
Index	(shop_id, deleted_at, name) (shop_id, category_id) (shop_id, server_updated_at)

9.3.6 exchange_rate (Taux de change)

Rôle	Historique des taux saisis par le commerçant (ex. 1 USD = 2 800 CDF). Table en ajout seul : un nouveau taux est une nouvelle ligne ; une ligne n'est jamais modifiée, ce qui fige l'historique des ventes qui la référencent.
Synchronisation	Ajout seul
Clé primaire	id (UUID)
Clés étrangères	shop_id → shop ON DELETE PROTECT base_currency_code → currency ON DELETE PROTECT quote_currency_code → currency ON DELETE PROTECT
Contraintes	CHECK (rate > 0) CHECK (base_currency_code <> quote_currency_code)
Index	(shop_id, base_currency_code, quote_currency_code, effective_at DESC) (shop_id, server_updated_at)

9.3.7 sale (Vente)

Rôle	En-tête d'une vente validée : devise, taux utilisé, total, statut. Immuable après validation sauf le passage unique COMPLETED → CANCELLED. Jamais supprimée physiquement.
Synchronisation	Ajout seul*
Clé primaire	id (UUID)
Clés étrangères	shop_id → shop ON DELETE PROTECT exchange_rate_id → exchange_rate ON DELETE PROTECT currency_code → currency ON DELETE PROTECT
Contraintes	CHECK (status IN ('COMPLETED','CANCELLED')) CHECK (total_amount >= 0) CHECK ((status = 'CANCELLED') = (cancelled_at IS NOT NULL))
Index	(shop_id, sold_at DESC) (shop_id, receipt_number) (shop_id, server_updated_at)

9.3.8 sale_item (Ligne de vente)

Rôle	Ligne d'une vente. Fige l'état commercial au moment de l'achat : désignation, prix unitaire et coût unitaire, exprimés dans la devise de la vente. C'est cette table qui garantit la cohérence des anciens bénéfices.
Synchronisation	Ajout seul
Clé primaire	id (UUID)
Clés étrangères	sale_id → sale ON DELETE CASCADE product_id → product ON DELETE PROTECT
Contraintes	CHECK (quantity > 0) CHECK (unit_price >= 0 AND unit_cost >= 0)
Index	(sale_id) (product_id)

9.3.9 stock_movement (Mouvement de stock)

Rôle	Journal (ledger) du stock : une ligne par variation de quantité, avec son type et sa cause. Le stock courant d'un produit est la somme de ses mouvements. Jamais modifié ni supprimé : une erreur se corrige par un mouvement compensatoire.
Synchronisation	Ajout seul
Clé primaire	id (UUID)
Clés étrangères	shop_id → shop ON DELETE PROTECT product_id → product ON DELETE PROTECT sale_item_id → sale_item ON DELETE PROTECT
Contraintes	CHECK (movement_type IN ('INITIAL','RESTOCK','SALE','SALE_CANCEL','ADJUSTMENT')) CHECK (quantity <> 0) CHECK (movement_type <> 'SALE' OR quantity < 0) CHECK (movement_type NOT IN ('INITIAL','RESTOCK','SALE_CANCEL') OR quantity > 0) CHECK (movement_type <> 'ADJUSTMENT' OR reason IS NOT NULL) UNIQUE (sale_item_id, movement_type) -- garde-fou contre le double décompte
Index	(product_id, created_at) (shop_id, server_updated_at)

9.3.10 sync_session (Synchronisation)

Rôle	Journal technique d'un lot de synchronisation (push) : appareil, compteurs, statut et anomalies. N'est pas synchronisé vers le client. Sert au support (« pourquoi ma vente manque-t-elle ? ») et à la supervision.
Synchronisation	Journal serveur
Clé primaire	id (UUID)
Clés étrangères	shop_id → shop ON DELETE PROTECT
Contraintes	CHECK (status IN ('IN_PROGRESS','SUCCESS','PARTIAL','FAILED'))
Index	(shop_id, started_at DESC)

9.4 Compatibilité PostgreSQL et MySQL

Le MLD est volontairement neutre. **PostgreSQL est recommandé** (H-18) ; MySQL 8 reste possible avec les adaptations ci-dessous, aucune ne remettant en cause le modèle.

Sujet	PostgreSQL (recommandé)	MySQL 8 : adaptation
UUID	Type natif uuid (16 octets)	CHAR(36) ou BINARY(16) ; Django gère la conversion (UUIDField)
Horodatages	TIMESTAMPTZ (stocké en UTC)	DATETIME(6) en UTC ; Django USE_TZ = True
Contraintes CHECK	Appliquées depuis toujours	Appliquées à partir de MySQL 8.0.16 ; vérifier la version

Sujet	PostgreSQL (recommandé)	MySQL 8 : adaptation
Index partiels (ex. produits actifs)	CREATE INDEX … WHERE deleted_at IS NULL	Non disponible : index complet ou colonne générée
JSON (error_details)	JSONB indexable	JSON
Opérations atomiques sur le stock	UPDATE … SET current_stock = current_stock + :delta	Identique ; niveau d'isolation REPEATABLE READ par défaut à connaître
Unicité avec NULL multiples (user.email)	Plusieurs NULL autorisés	Plusieurs NULL autorisés

Tableau 9.2 : Écarts entre PostgreSQL et MySQL

9.5 Vérification de la normalisation

- **1re forme normale** : tous les attributs sont atomiques ; aucune liste dans une colonne (error_details, JSON technique, est la seule exception assumée).
- **2e forme normale** : aucune table n'a de clé composée, donc aucune dépendance partielle possible.
- **3e forme normale** : aucun attribut non-clé ne dépend d'un autre attribut non-clé, *sauf les redondances délibérées* listées ci-dessous.

9.6 Redondances et instantanés assumés

Élément	Nature	Justification	Garde-fou
product.current_stock	Redondance contrôlée (dérivée des mouvements)	Lecture instantanée du stock sur téléphone sans sommer tout le journal ; fonctionne hors ligne	Mis à jour par le service dans la transaction du mouvement ; commande de réconciliation périodique
sale.total_amount	Redondance contrôlée (somme des lignes)	Historique rapide ; contrôle d'intégrité à la synchronisation	CHECK ≥ 0 ; le serveur recalcule et rejette TOTAL_MISMATCH
sale_item.product_name, unit_price, unit_cost	**Instantanés historiques** (pas une redondance : valeurs au moment de la vente)	Le reçu et le bénéfice d'hier doivent rester vrais même si le produit change	Aucune API de modification ; RG-09
stock_movement.shop_id	Colonne de cloisonnement dénormalisée	Pull et filtres par boutique sans jointure	Le service vérifie shop_id = product.shop_id

Tableau 9.3 : Dénormalisations assumées

9.7 Colonnes techniques ajoutées au niveau logique

Colonne	Où	Rôle
shop_id	Toutes les tables métier	Isolation entre boutiques ; imposée par le serveur à partir de l'utilisateur authentifié
created_at	Tables synchronisées	Instant de création selon l'appareil (valeur métier, non fiable pour ordonner)
updated_at	Tables modifiables	Dernière modification selon l'appareil ; départage deux modifications concurrentes
server_updated_at	Tables synchronisées	Horloge du **serveur** ; curseur du pull ; fiable car monotone côté serveur
version	Tables modifiables	Compteur incrémenté par le serveur ; détecte qu'une modification part d'une version dépassée
deleted_at	Tables modifiables	Suppression logique et « pierre tombale » que la synchronisation propage aux autres appareils

Tableau 9.4 : Colonnes techniques

> **Où est le statut de synchronisation ?**
> Le **statut de synchronisation** (PENDING, SYNCED, ERROR…) n'est **pas** une colonne du modèle serveur : côté serveur toute ligne reçue est, par définition, synchronisée. Il vit uniquement dans la base locale du téléphone (chapitre 14).

10. Dictionnaire de données

Le dictionnaire décrit chaque champ de chaque table du MLD : description, type, taille, caractère obligatoire et exemple. Il est **généré à partir de la même source** que le MLD, ce qui garantit leur cohérence. **PK** = clé primaire, **FK** = clé étrangère, **UQ** = unique. Les tailles des types fixes sont données en octets de stockage PostgreSQL.

10.1 currency (Devise)

Champ	Description	Type	Taille	Oblig.	Exemple
code PK	Code ISO 4217 ; clé naturelle (pas d'UUID : donnée de référence figée).	CHAR	3	Oui	`CDF`
name	Libellé de la devise.	VARCHAR	50	Oui	`Franc congolais`
symbol	Symbole affiché à côté des montants.	VARCHAR	5	Oui	`FC`
decimal_places	Décimales d'arrondi et d'affichage (CDF : 0 ; USD : 2).	SMALLINT	2 o	Oui	`0`

10.2 app_user (Utilisateur)

Champ	Description	Type	Taille	Oblig.	Exemple
id PK	Identifiant unique (également présent dans le jeton JWT).	UUID	16 o	Oui	`7c1e9b0a-52d4-4f0e-8a3b-1d9c6e4f2a71`
phone UQ	Téléphone au format E.164 ; identifiant de connexion.	VARCHAR	20	Oui	`+243810000001`
email UQ	Adresse e-mail facultative.	VARCHAR	254	Non	`rose@example.com`
password	Empreinte (hash) du mot de passe, jamais le mot de passe en clair.	VARCHAR	128	Oui	`argon2$argon2id$v=19$...`
full_name	Nom complet du commerçant.	VARCHAR	150	Oui	`Rose Mbuyi`
is_active	Compte actif ; false = connexion refusée.	BOOLEAN	1 o	Oui	`true`
is_staff	Accès à l'interface /admin/ Django (équipe technique uniquement).	BOOLEAN	1 o	Oui	`false`
is_superuser	Super-utilisateur Django (équipe technique uniquement).	BOOLEAN	1 o	Oui	`false`
last_login	Dernière connexion réussie.	TIMESTAMPTZ	8 o	Non	`2026-10-04T08:15:30Z`
created_at	Date de création du compte.	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
updated_at	Dernière modification du profil.	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`

10.3 shop (Boutique)

Champ	Description	Type	Taille	Oblig.	Exemple
id PK	Identifiant unique de la boutique.	UUID	16 o	Oui	`b2f4a6c8-19e3-47d5-9b0a-3c8e5d7f1a24`
owner_id FK→app_user · UQ	Propriétaire (compte). UNIQUE en MVP : une boutique par commerçant.	UUID	16 o	Oui	`7c1e9b0a-52d4-4f0e-8a3b-1d9c6e4f2a71`
name	Nom commercial affiché sur les reçus.	VARCHAR	150	Oui	`Boutique Maman Rose`
address	Adresse ou localisation (reçus).	VARCHAR	255	Non	`Av. du Marché n°12`
phone	Téléphone de la boutique (reçus).	VARCHAR	20	Non	`+243810000001`
reference_currency_code FK→currency	Devise de référence pour consolider le chiffre d'affaires et le bénéfice.	CHAR	3	Oui	`CDF`
created_at	Date de création.	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
updated_at	Dernière modification.	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`

10.4 category (Catégorie)

Champ	Description	Type	Taille	Oblig.	Exemple
id PK	Identifiant généré par le client (UUID v4).	UUID	16 o	Oui	`3a9d5e17-c4b2-4086-b7f1-92e0d6a8c453`
shop_id FK→shop	Boutique propriétaire (imposée par le serveur).	UUID	16 o	Oui	`b2f4a6c8-19e3-47d5-9b0a-3c8e5d7f1a24`
name	Nom de la catégorie (doublons tolérés côté base, signalés par l'interface).	VARCHAR	100	Oui	`Boissons`
created_at	Instant de création selon l'horloge de l'appareil (immuable).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
updated_at	Dernière modification selon l'horloge de l'appareil (sert à départager deux modifications concurrentes).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
server_updated_at	Instant d'écriture côté serveur ; curseur de la synchronisation descendante (pull).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
version	Numéro de version incrémenté par le serveur à chaque modification acceptée (détection de conflit).	INTEGER	4 o	Oui	`3`
deleted_at	Suppression logique (archivage). NULL = actif. Sert aussi de « pierre tombale » pour la synchronisation.	TIMESTAMPTZ	8 o	Non	`null`

10.5 product (Produit)

Champ	Description	Type	Taille	Oblig.	Exemple
id PK	Identifiant généré par le client (UUID v4).	UUID	16 o	Oui	`e5a7c9b1-6d3f-4a28-8c14-5f0b2e9d7a36`
shop_id FK→shop	Boutique propriétaire.	UUID	16 o	Oui	`b2f4a6c8-19e3-47d5-9b0a-3c8e5d7f1a24`
category_id FK→category	Catégorie (facultative). ON DELETE SET NULL.	UUID	16 o	Non	`3a9d5e17-c4b2-4086-b7f1-92e0d6a8c453`
name	Désignation du produit.	VARCHAR	150	Oui	`Sucre 1 kg`
purchase_price	Prix d'achat unitaire courant (dernier prix connu), ≥ 0.	DECIMAL	14,2	Oui	`5000.00`
purchase_currency_code FK→currency	Devise du prix d'achat.	CHAR	3	Oui	`CDF`
selling_price	Prix de vente unitaire courant, ≥ 0.	DECIMAL	14,2	Oui	`7000.00`
selling_currency_code FK→currency	Devise du prix de vente.	CHAR	3	Oui	`CDF`
current_stock	Stock courant = somme des mouvements. Propriété du serveur ; jamais poussée par le client.	INTEGER	4 o	Oui	`44`
low_stock_threshold	Seuil d'alerte : alerte si current_stock ≤ seuil (0 = alerte à la rupture).	INTEGER	4 o	Oui	`5`
created_at	Instant de création selon l'horloge de l'appareil (immuable).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
updated_at	Dernière modification selon l'horloge de l'appareil (sert à départager deux modifications concurrentes).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
server_updated_at	Instant d'écriture côté serveur ; curseur de la synchronisation descendante (pull).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
version	Numéro de version incrémenté par le serveur à chaque modification acceptée (détection de conflit).	INTEGER	4 o	Oui	`3`
deleted_at	Suppression logique (archivage). NULL = actif. Sert aussi de « pierre tombale » pour la synchronisation.	TIMESTAMPTZ	8 o	Non	`null`

10.6 exchange_rate (Taux de change)

Champ	Description	Type	Taille	Oblig.	Exemple
id PK	Identifiant généré par le client.	UUID	16 o	Oui	`4d6f8a0c-e2b4-4c71-95a3-1e7b9d5c3f08`
shop_id FK→shop	Boutique propriétaire.	UUID	16 o	Oui	`b2f4a6c8-19e3-47d5-9b0a-3c8e5d7f1a24`
base_currency_code FK→currency	Devise de base (1 unité de cette devise…).	CHAR	3	Oui	`USD`
quote_currency_code FK→currency	Devise cotée (… vaut `rate` unités de cette devise).	CHAR	3	Oui	`CDF`
rate	Taux : nombre d'unités de la devise cotée pour 1 unité de la devise de base (> 0).	DECIMAL	18,6	Oui	`2800.000000`
effective_at	Instant d'entrée en vigueur du taux (horloge de l'appareil).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
created_at	Création (appareil).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
server_updated_at	Réception côté serveur (curseur de pull).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`

10.7 sale (Vente)

Champ	Description	Type	Taille	Oblig.	Exemple
id PK	Identifiant généré par le client ; garantit l'idempotence de la synchronisation.	UUID	16 o	Oui	`a8c0e2f4-7b9d-4153-8e6a-2f4c6b8d0a91`
shop_id FK→shop	Boutique propriétaire.	UUID	16 o	Oui	`b2f4a6c8-19e3-47d5-9b0a-3c8e5d7f1a24`
exchange_rate_id FK→exchange_rate	Taux en vigueur au moment de la vente (ligne immuable). ON DELETE PROTECT.	UUID	16 o	Oui	`4d6f8a0c-e2b4-4c71-95a3-1e7b9d5c3f08`
currency_code FK→currency	Devise de la vente (choisie à l'encaissement) ; tous les montants de la vente sont dans cette devise.	CHAR	3	Oui	`CDF`
receipt_number	Numéro de reçu lisible : date (AAMMJJ) + code court dérivé de l'UUID. Indexé, non unique.	VARCHAR	16	Oui	`261004-K7QF9M`
sold_at	Instant de validation (horloge de l'appareil).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
status	COMPLETED (validée) ou CANCELLED (annulée).	VARCHAR	10	Oui	`COMPLETED`
total_amount	Total de la vente, redondance contrôlée : somme des lignes (quantité × prix unitaire).	DECIMAL	14,2	Oui	`21000.00`
cancelled_at	Instant d'annulation ; renseigné si et seulement si status = CANCELLED.	TIMESTAMPTZ	8 o	Non	`null`
cancel_reason	Motif d'annulation (facultatif).	VARCHAR	255	Non	`Erreur de saisie`
server_updated_at	Instant d'écriture serveur (curseur de pull ; mis à jour lors d'une annulation).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`

10.8 sale_item (Ligne de vente)

Champ	Description	Type	Taille	Oblig.	Exemple
id PK	Identifiant généré par le client (référencé par les mouvements de stock).	UUID	16 o	Oui	`0f2b4d6e-8a1c-4e35-b7d9-6c8a0e2f4b57`
sale_id FK→sale	Vente parente. ON DELETE CASCADE (composition).	UUID	16 o	Oui	`a8c0e2f4-7b9d-4153-8e6a-2f4c6b8d0a91`
product_id FK→product	Produit vendu. ON DELETE PROTECT.	UUID	16 o	Oui	`e5a7c9b1-6d3f-4a28-8c14-5f0b2e9d7a36`
product_name	Désignation figée au moment de la vente (instantané).	VARCHAR	150	Oui	`Sucre 1 kg`
quantity	Quantité vendue, entière et > 0.	INTEGER	4 o	Oui	`3`
unit_price	Prix de vente unitaire figé, dans la devise de la vente (converti et arrondi si besoin).	DECIMAL	14,2	Oui	`7000.00`
unit_cost	Coût d'achat unitaire figé, dans la devise de la vente.	DECIMAL	14,2	Oui	`5000.00`

10.9 stock_movement (Mouvement de stock)

Champ	Description	Type	Taille	Oblig.	Exemple
id PK	Identifiant généré par le client.	UUID	16 o	Oui	`91c3e5a7-2b4d-4f68-a0e1-7d9b3c5f8e12`
shop_id FK→shop	Boutique (colonne technique de cloisonnement, dénormalisée).	UUID	16 o	Oui	`b2f4a6c8-19e3-47d5-9b0a-3c8e5d7f1a24`
product_id FK→product	Produit concerné. ON DELETE PROTECT.	UUID	16 o	Oui	`e5a7c9b1-6d3f-4a28-8c14-5f0b2e9d7a36`
sale_item_id FK→sale_item	Ligne de vente à l'origine du mouvement (types SALE et SALE_CANCEL).	UUID	16 o	Non	`0f2b4d6e-8a1c-4e35-b7d9-6c8a0e2f4b57`
movement_type	INITIAL, RESTOCK, SALE, SALE_CANCEL ou ADJUSTMENT.	VARCHAR	20	Oui	`RESTOCK`
quantity	Variation signée : positive pour une entrée, négative pour une sortie ; jamais 0.	INTEGER	4 o	Oui	`30`
reason	Motif ; obligatoire pour ADJUSTMENT.	VARCHAR	255	Non	`Sachet percé`
created_at	Instant du mouvement (horloge de l'appareil).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
server_updated_at	Réception côté serveur (curseur de pull).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`

10.10 sync_session (Synchronisation)

Champ	Description	Type	Taille	Oblig.	Exemple
id PK	Identifiant du lot, généré par le client (clé d'idempotence du lot).	UUID	16 o	Oui	`c6e8a0b2-4d6f-4a89-93c1-5b7d9f1e3a60`
shop_id FK→shop	Boutique concernée.	UUID	16 o	Oui	`b2f4a6c8-19e3-47d5-9b0a-3c8e5d7f1a24`
device_id	Identifiant de l'appareil (généré au premier lancement) ; simple attribut.	UUID	16 o	Oui	`d1f3b5a7-9c0e-4b24-a6d8-8e0a2c4f6b19`
started_at	Début du traitement (horloge serveur).	TIMESTAMPTZ	8 o	Oui	`2026-10-04T08:15:30Z`
finished_at	Fin du traitement.	TIMESTAMPTZ	8 o	Non	`2026-10-04T08:15:30Z`
status	IN_PROGRESS, SUCCESS, PARTIAL ou FAILED.	VARCHAR	12	Oui	`SUCCESS`
received_count	Éléments reçus.	INTEGER	4 o	Oui	`12`
accepted_count	Éléments créés ou mis à jour.	INTEGER	4 o	Oui	`10`
duplicate_count	Éléments déjà synchronisés (idempotence).	INTEGER	4 o	Oui	`1`
conflict_count	Conflits résolus par une règle.	INTEGER	4 o	Oui	`0`
rejected_count	Éléments rejetés (validation).	INTEGER	4 o	Oui	`1`
error_details	Détail des conflits et rejets (JSON ; type JSON sous MySQL).	JSONB	var.	Non	`[{"id":"…","code":"TOTAL_MISMATCH"}]`

11. Gestion du stock

11.1 Principe : un journal, pas un compteur

Le stock n'est jamais « modifié » : on **ajoute une ligne au journal** des mouvements (table *stock_movement*) et le stock courant en résulte. C'est le principe du livre de comptes : on ne gomme jamais une écriture, on la compense par une autre. Il répond directement à l'exigence « pouvoir retrouver **pourquoi** le stock a changé ».

- **Source de vérité** : la somme des mouvements d'un produit (RG-05).
- **Lecture rapide** : la colonne *product.current_stock* est une copie dérivée, mise à jour dans la même transaction que le mouvement (redondance contrôlée).
- **Synchronisation** : on échange des **variations** (+30, −3), jamais des valeurs (« le stock est 44 »). Deux appareils qui vendent hors ligne ne s'écrasent donc jamais : leurs variations s'additionnent (RG-19).
- **Aucune modification ni suppression** d'un mouvement : une erreur se corrige par un mouvement compensatoire (ajustement ou annulation).

11.2 Types de mouvements

Type	Famille	Signe	Origine	Motif	Exemple
INITIAL	Entrée	+	Création du produit (stock de départ)	—	+20 à la création de « Sucre 1 kg »
RESTOCK	Entrée	+	Réapprovisionnement (achat reçu)	—	+30 après livraison du grossiste
SALE_CANCEL	Entrée	+	Annulation d'une vente validée	—	+4 quand une vente de 4 unités est annulée
SALE	Sortie	−	Ligne d'une vente validée	—	−3 pour 3 unités vendues
ADJUSTMENT	Ajustement	±	Inventaire, casse, perte, usage personnel	**Obligatoire**	−1 « sachet percé »

Tableau 11.1 : Les cinq types de mouvements (contraintes CHECK au chapitre 9)

11.3 Exemple chiffré : le journal d'un produit

Produit « Sucre 1 kg ». Le stock courant est la colonne de droite : on peut lire la vie du stock ligne par ligne.

Date	Type	Quantité	Cause	Stock courant
04/10 08:00	INITIAL	+20	Création du produit	**20**
04/10 10:15	SALE	−3	Vente S1 (3 unités)	**17**
04/10 14:40	SALE	−2	Vente S4 (2 unités)	**15**
05/10 07:50	RESTOCK	+30	Livraison grossiste	**45**
05/10 09:20	ADJUSTMENT	−1	Motif : sachet percé	**44**
05/10 11:05	SALE	−4	Vente S6 (4 unités)	**40**
05/10 11:20	SALE_CANCEL	+4	Annulation de S6 (erreur de saisie)	**44**

Tableau 11.2 : Journal du produit « Sucre 1 kg » (somme finale : 44)

11.4 Répondre aux questions du commerçant

La modélisation permet de connaître à tout moment chaque indicateur demandé. Les requêtes ci-dessous (PostgreSQL) servent aussi de spécification pour les écrans et les tests.

Question	Réponse
Stock actuel	*product.current_stock* (copie) ou Σ quantity des mouvements (vérité)
Stock initial	Mouvement de type INITIAL du produit
Entrées / sorties / ajustements	Somme des quantités par famille de type sur la période
Ventes	Mouvements SALE (et SALE_CANCEL pour les retours)
Réapprovisionnements	Mouvements RESTOCK
Pourquoi le stock a changé	Colonnes movement_type, reason, sale_item_id (vente d'origine), created_at

Tableau 11.3 : Où trouver chaque information de stock

```
-- Stock courant d'un produit (source de vérité)
SELECT product_id, SUM(quantity) AS stock
FROM stock_movement WHERE shop_id = :shop GROUP BY product_id;

-- Stock à une date donnée (reconstitution)
SELECT COALESCE(SUM(quantity), 0) FROM stock_movement
WHERE product_id = :product AND created_at <= :instant;

-- Entrées, sorties et ajustements d'une période
SELECT movement_type, COUNT(*) AS nb, SUM(quantity) AS total
FROM stock_movement
WHERE product_id = :product AND created_at BETWEEN :debut AND :fin
GROUP BY movement_type;

-- Contrôle de réconciliation : doit retourner 0 ligne
SELECT p.id, p.current_stock, COALESCE(SUM(m.quantity), 0) AS recalcule
FROM product p LEFT JOIN stock_movement m ON m.product_id = p.id
GROUP BY p.id HAVING p.current_stock <> COALESCE(SUM(m.quantity), 0);
```

11.5 Intégrité et concurrence

- **Une seule porte d'entrée** : la fonction de service *apply_movement()* crée le mouvement et met à jour *current_stock* dans la même transaction. Aucune vue ni signal ne touche au stock ailleurs.
- **Mise à jour atomique** : *current_stock = current_stock + variation* exécuté par la base (expression *F()* de Django), jamais « lire, calculer, écrire » qui perd des mises à jour en cas de requêtes simultanées.
- **Garde-fou contre le double décompte** : la contrainte UNIQUE (sale_item_id, movement_type) interdit qu'une même ligne de vente décrémente deux fois le stock, même si un lot est rejoué.
- **Réconciliation** : une commande planifiée (management command) compare *current_stock* et la somme des mouvements, corrige l'écart et le journalise ; à lancer chaque nuit au début.
- **Pull du stock** : chaque modification de *current_stock* met aussi à jour *server_updated_at* du produit (sans toucher à *version*), pour que les autres appareils reçoivent le nouveau stock.

11.6 Stock insuffisant et stock négatif

La caisse **bloque** une ligne dont la quantité dépasse le stock local (RG-08, H-05). En revanche le **serveur n'annule jamais une vente déjà réalisée** : deux appareils ayant vendu la dernière unité hors ligne produisent un stock négatif à la synchronisation. Le serveur l'accepte, signale le produit « stock à vérifier » dans la réponse et le commerçant corrige par un ajustement motivé. Aucun CHECK (current_stock ≥ 0) n'est donc posé en base.

> **Pourquoi on tolère le stock négatif**
> Refuser côté serveur une vente que le commerçant a déjà conclue physiquement ne corrigerait rien : la marchandise est partie et le client est servi. Le bon réflexe est de **signaler**, pas de rejeter.

11.7 Alerte de stock faible

- Règle (RG-18) : alerte si **current_stock ≤ low_stock_threshold**. Avec un seuil à 0 (valeur par défaut), l'alerte se déclenche à la rupture.

- L'alerte est **calculée**, pas stockée : aucune table d'alertes. Elle s'affiche après une vente (retour du service de stock) et dans l'écran de stock (filtre « stock faible »).
- Une notification poussée (push) est hors MVP ; elle exigerait un consentement et un service dédié.

12. Gestion des ventes

12.1 Structure d'une vente

Une vente est composée d'un **en-tête** (table *sale*) et de **lignes** (table *sale_item*). L'en-tête porte la devise, le taux utilisé et le total ; chaque ligne fige l'état commercial du produit au moment de la vente.

Élément	Valeur figée	Pourquoi
sale_item.product_name	Désignation du produit à l'instant T	Le reçu et l'historique restent lisibles si le produit est renommé ou archivé
sale_item.unit_price	Prix de vente unitaire **dans la devise de la vente** (converti et arrondi si besoin)	Si le prix catalogue change demain, la vente d'hier ne bouge pas
sale_item.unit_cost	Coût d'achat unitaire **dans la devise de la vente**	Le bénéfice d'hier reste juste même si le prix d'achat est mis à jour
sale.exchange_rate_id	Référence au taux en vigueur (ligne immuable)	Les ventes consolidées ne sont pas faussées par un taux ultérieur
sale.currency_code	Devise choisie à l'encaissement	Tous les montants de la vente sont dans cette devise
sale.total_amount	Σ (quantité × prix unitaire)	Redondance contrôlée ; vérifiée par le serveur

Tableau 12.1 : Ce qu'une vente fige au moment de la validation

12.2 Cycle de vie d'une vente

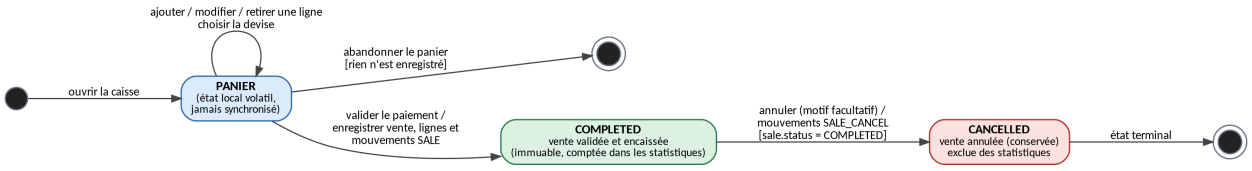


Figure 12.1 : Diagramme d'états d'une vente

- **PANIER** est un état *local et volatil* : il n'est ni persisté comme vente ni synchronisé. L'abandon d'un panier ne laisse aucune trace.
- **COMPLETED** : la vente est encaissée, immuable, comptée dans les statistiques ; ses mouvements SALE ont diminué le stock.
- **CANCELLED** : transition unique et définitive depuis COMPLETED. Elle ajoute des mouvements SALE_CANCEL qui rendent les quantités au stock et exclut la vente des statistiques. La vente reste visible dans l'historique, avec son motif (RG-10).
- Il n'existe **aucune suppression** et aucun retour à un état antérieur : la traçabilité financière prime.

12.3 Calcul d'une vente

1. Lire le **taux courant** de la boutique (dernier enregistrement exchange_rate) ; il sera référencé par la vente.
2. Pour chaque ligne : **unit_price** = prix de vente du produit converti dans la devise de la vente puis arrondi (RG-12) ; **unit_cost** = prix d'achat converti de la même façon.
3. **total_amount** = Σ quantité × unit_price (calcul exact : les prix sont déjà arrondis).
4. Écrire la vente, les lignes et les mouvements dans une seule transaction ; générer l'UUID, le numéro de reçu et marquer PENDING.

```
def convert_price(amount, src, dst, rate_usd_cdf, decimals_of):
    # rate_usd_cdf : nombre de CDF pour 1 USD (valeur de l'enregistrement exchange_rate)
```

```
    if src == dst:
        value = amount
    elif src == 'USD' and dst == 'CDF':
        value = amount * rate_usd_cdf
    elif src == 'CDF' and dst == 'USD':
        value = amount / rate_usd_cdf
    else:
        raise ValueError('paire non prise en charge')
    return quantize(value, decimals_of(dst), ROUND_HALF_UP)   # un seul arrondi, ici
```

12.4 Bénéfice : formules et exemple

Indicateur	Formule	Remarque
Chiffre d'affaires d'une ligne	quantité × unit_price	Dans la devise de la vente
Coût d'une ligne	quantité × unit_cost	Instantané : jamais recalculé
Bénéfice d'une ligne	**(unit_price − unit_cost) × quantité**	RG-13
Marge (%)	bénéfice ÷ chiffre d'affaires × 100	Non définie si le chiffre d'affaires est 0
Période	Ventes COMPLETED uniquement	Les ventes CANCELLED sont exclues

Tableau 12.2 : Formules de calcul

Exemple. Taux du jour : 1 USD = 2 800 CDF. « Sucre 1 kg » : achat 5 000 CDF, vente 7 000 CDF. « Huile 1 L » : achat 3,00 USD, vente 5,00 USD.

Vente	Devise	Lignes	Chiffre d'affaires	Coût	Bénéfice	En CDF (taux 2 800)
S1	CDF	3 × Sucre @ 7 000 (coût 5 000)	21 000 CDF	15 000 CDF	**6 000 CDF**	CA 21 000 · bénéfice 6 000
S2	USD	2 × Huile @ 5,00 (coût 3,00)	10,00 USD	6,00 USD	**4,00 USD**	CA 28 000 · bénéfice 11 200
S3	CDF	1 × Huile : 5,00 × 2 800 = 14 000 (coût 3,00 × 2 800 = 8 400)	14 000 CDF	8 400 CDF	**5 600 CDF**	CA 14 000 · bénéfice 5 600
Total consolidé en CDF			**63 000**	**40 200**	**22 800**	Marge : 22 800 ÷ 63 000 = **36,19 %**

Tableau 12.3 : Trois ventes et leur consolidation (chaque vente est convertie avec son propre taux)

> **Anciennes ventes inchangées**
> **Cohérence dans le temps.** Si le taux passe demain à 2 900 CDF, la vente S2 vaut toujours 28 000 CDF car elle référence *son* taux (2 800). Si le prix du sucre passe à 8 000 CDF, la vente S1 reste à 7 000 CDF car la ligne a figé son prix. Aucune statistique passée ne peut changer sans qu'une écriture explicite (annulation) ne le décide.

12.5 Reçu et numéro de reçu

- Le reçu n'a pas d'entité : c'est une **représentation** de la vente. Il est construit localement (texte structuré ; image ou PDF facultatifs) pour fonctionner hors ligne.
- **Contenu** : nom, adresse et téléphone de la boutique, date et heure, numéro de reçu, lignes (désignation, quantité, prix unitaire, total de ligne), total et devise, mention du taux si une conversion a eu lieu.
- **Numéro de reçu** : *AAMMJJ-XXXXXX* (date + six caractères en base 32 dérivés de l'UUID, ex. 261004-K7QF9M). Un compteur séquentiel est impossible hors ligne sans coordination entre appareils (voir chapitre 16).
- **Avertissement** : ce reçu est un justificatif commercial, pas une facture fiscale (H-12).

```
Boutique Maman Rose
Av. du Marché n°12 · +243 81 000 0001
Reçu 261004-K7QF9M · 04/10/2026 10:15
-----------------------------------
3 x Sucre 1 kg      7 000   21 000 FC
-----------------------------------
TOTAL                        21 000 FC
```

```
Merci de votre visite !
```

12.6 Statistiques : un calcul, deux endroits

Les statistiques sont calculées **localement** à partir d'IndexedDB (pour fonctionner hors ligne) et **côté serveur** (pour le web et la vérification). Le risque est que les deux divergent. Les règles sont donc spécifiées une seule fois ici : agrégation sur les ventes COMPLETED ; conversion de chaque vente vers la devise de référence avec **son** taux, arrondie à la précision de cette devise **par vente**, puis somme ; marge calculée sur les totaux et non en moyenne de marges.

> **Bonne pratique : tests de parité**
> Écrire un fichier de cas de test partagé (JSON) contenant des ventes d'entrée et les indicateurs attendus, lu à la fois par les tests Python et par les tests TypeScript. Une divergence devient alors un test rouge, pas une réclamation d'un commerçant.

13. Gestion des devises CDF et USD

13.1 Pourquoi une simple conversion automatique ne suffit pas

- Le **taux varie** et le commerçant applique *son* taux, pas un taux officiel : il doit pouvoir le saisir et le changer.
- Les ventes **passées** ne doivent pas changer quand le taux change : il faut conserver le taux utilisé.
- Le prix d'achat et le prix de vente d'un produit peuvent être dans **deux devises différentes** (achat en USD, vente en CDF) : le bénéfice n'est calculable qu'avec un taux.
- Les **arrondis** diffèrent d'une devise à l'autre (le CDF se paie sans centimes) : sans règle unique, le total d'un ticket ne serait pas égal à la somme de ses lignes.

13.2 Devises prises en charge

Code	Nom	Symbole	Décimales	Remarque
CDF	Franc congolais	FC	0	Les centimes ne s'utilisent pas en pratique (H-10)
USD	Dollar américain	$	2	Précision au cent

Tableau 13.1 : Table currency (valeurs initiales)

13.3 Le taux de change

- Table **exchange_rate** en **ajout seul** : base (USD), cotée (CDF), taux (ex. 2 800), date d'effet. Saisir un nouveau taux = ajouter une ligne ; aucune ligne n'est modifiée ni supprimée (RG-20).
- Le **taux courant** est la ligne la plus récente de la boutique ; il est disponible hors ligne (copie locale).
- Chaque vente **référence** le taux courant à l'instant de sa validation (RG-11). Le taux est donc obligatoire avant la première vente : il est demandé à l'inscription.
- **Alerte de taux ancien** (proposée : plus de 48 h) pour éviter de vendre avec un taux périmé (risque R-03).

13.4 Conversion et arrondi

Un prix converti est arrondi **une seule fois**, à la précision de la devise de la vente, par la méthode commerciale (demi vers le haut). Les prix de ligne étant déjà arrondis, les totaux sont ensuite des calculs exacts : le total d'un reçu est toujours égal à la somme de ses lignes.

Prix catalogue	Taux (1 USD)	Devise de la vente	Calcul	Prix de ligne
3,00 USD	2 800	CDF	3,00 × 2 800	**8 400 CDF**
7 000 CDF	2 800	USD	7 000 ÷ 2 800	**2,50 USD**
7 500 CDF	2 800	USD	7 500 ÷ 2 800 = 2,678571…	**2,68 USD** (arrondi au cent)
2,50 USD	2 765,5	CDF	2,50 × 2 765,5 = 6 913,75	**6 914 CDF** (arrondi à l'unité)
7 000 CDF	2 800	CDF	aucune conversion	**7 000 CDF**

Tableau 13.2 : Exemples de conversion et d'arrondi

13.5 Ce qui est conservé : montant, devise, taux, montant converti

Information	Où elle est conservée	Décision
Montant (de la vente et des lignes)	sale.total_amount, sale_item.unit_price et unit_cost	Stocké, dans la devise de la vente
Devise	sale.currency_code	Stockée par vente (jamais « déduite »)
Taux de change utilisé	sale.exchange_rate_id → exchange_rate.rate	Référence vers une ligne **immuable** (pas de copie du nombre : 3e forme normale respectée)
Montant converti (en devise de référence)	*Non stocké*	Dérivable de façon déterministe : montant × taux, avec la règle d'arrondi. Le stocker créerait une redondance à garder cohérente
Prix catalogue d'origine (avant conversion)	*Non stocké*	Retrouvable en divisant par le taux (à l'arrondi près). Si un audit fin est exigé : ajouter list_price et list_currency_code sur la ligne

Tableau 13.3 : Conservation des informations financières

> **Décision D6**
> **Une vente = une devise.** Le commerçant choisit CDF ou USD à l'encaissement ; toutes les lignes sont exprimées dans cette devise. Le paiement en deux devises ou la monnaie rendue dans une autre devise supposent l'entité Paiement (H-06), hors MVP.

13.6 Garantie d'immuabilité : démonstration

Jour	Événement	Taux enregistré	Valeur de S2 (10,00 USD) en CDF
Lundi	Vente S2 en USD, taux courant 2 800	T1 = 2 800 (ligne exchange_rate)	S2 → T1 : 10 × 2 800 = **28 000**
Mardi	Le commerçant saisit un nouveau taux 2 900	T2 = 2 900 (nouvelle ligne ; T1 inchangée)	S2 → toujours T1 : **28 000**
Mardi	Nouvelle vente S7 en USD	S7 → T2	S7 : 10 × 2 900 = **29 000**

Tableau 13.4 : Un changement de taux n'altère pas les ventes passées

13.7 Cas limites et règles d'interface

- **Aucun taux défini** : la caisse propose de le saisir avant d'accepter une vente (le taux est obligatoire, NOT NULL).
- **Produit sans devise explicite** : impossible ; les deux devises du produit sont obligatoires (FK NOT NULL).
- **Affichage** : montants formatés selon *decimal_places* avec le symbole de la devise (« 7 000 FC », « 2,50 $ ») ; séparateurs adaptés à la langue.
- **Transport JSON** : les montants circulent en **chaînes décimales** (« 7000.00 »), jamais en nombres flottants ; en TypeScript, calcul sur entiers de la plus petite unité ou bibliothèque décimale.
- **Taux inversé** : un seul taux USD→CDF est saisi ; le sens inverse se déduit (division) pour éviter deux taux incohérents.

14. Fonctionnement offline-first

14.1 Principes

1. **L'écriture locale d'abord.** Toute action (vente, mouvement, produit) est écrite dans la base locale avant tout appel réseau. La confirmation à l'utilisateur vient de cette écriture.
2. **Le réseau n'est jamais bloquant.** Aucun écran de la caisse n'attend le serveur.
3. **Un seul chemin de code.** En ligne ou hors ligne, la même fonction écrit en local ; seule la synchronisation diffère.
4. **La synchronisation est idempotente et rejouable.** Les UUID générés par le client rendent tout envoi répétable sans doublon.
5. **Rien n'est supprimé avant confirmation.** Un enregistrement local reste jusqu'à ce que le serveur l'ait accepté (RG-17).
6. **Le serveur reste l'autorité sur le stock courant** et sur la validation ; le client conserve l'autorité sur ce qu'il a vu et fait.

14.2 Architecture technique cible

Le client est une **PWA** (application web installable) construite avec Next.js : l'interface React s'appuie sur des services métier TypeScript, une base locale IndexedDB, un moteur de synchronisation et un client Axios. Un **Service Worker** met en cache les fichiers de l'application pour qu'elle s'ouvre sans réseau ; les **données**, elles, sont dans IndexedDB (jamais dans le cache du Service Worker).

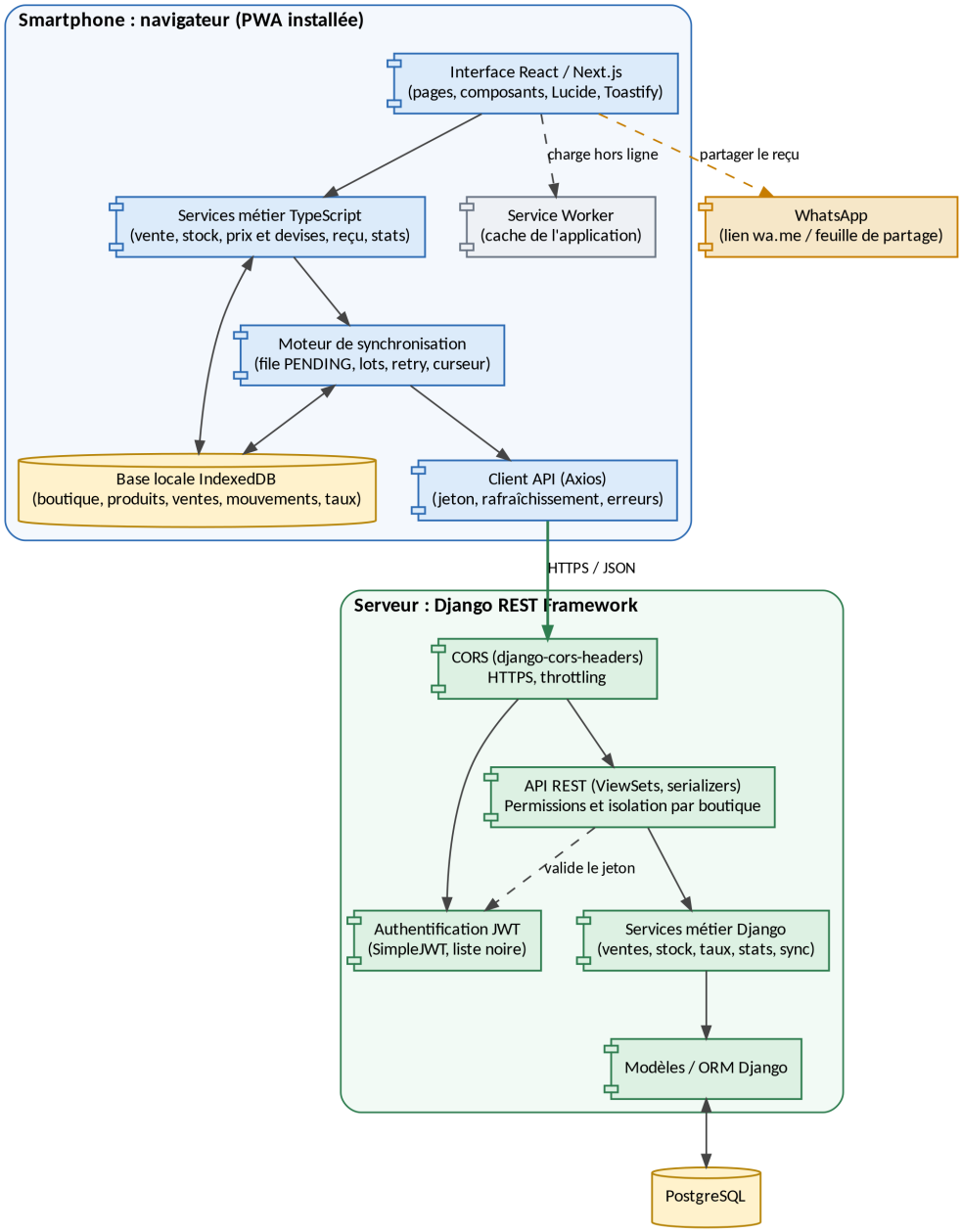


Figure 14.1 : Diagramme de composants : client PWA, serveur Django, base de données, WhatsApp

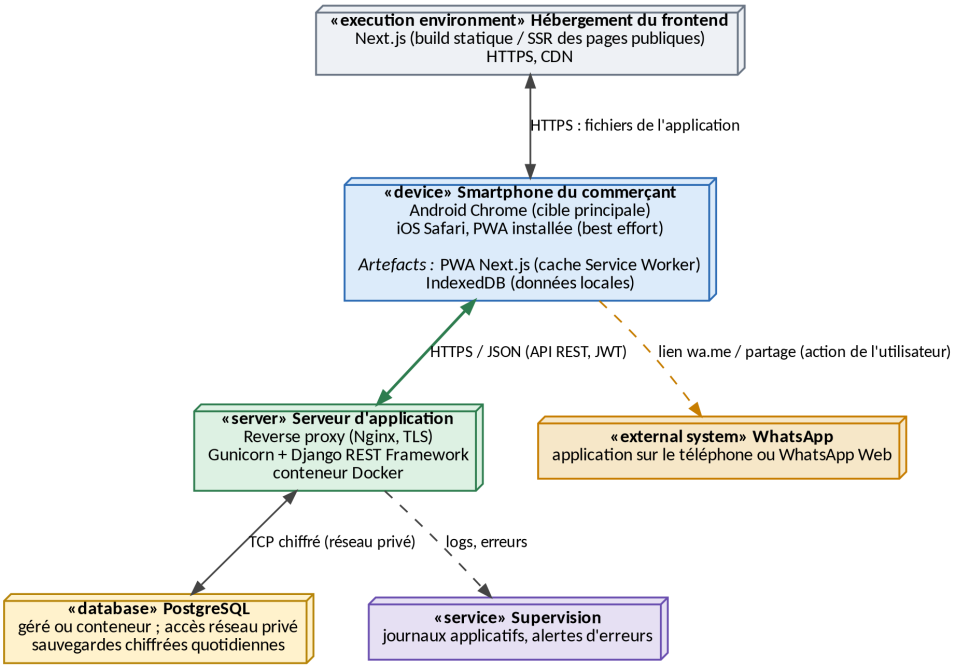


Figure 14.2 : Diagramme de déploiement indicatif (hébergeurs à choisir)

14.3 Base locale (IndexedDB)

Le client reproduit le modèle serveur dans IndexedDB (par exemple via une bibliothèque comme Dexie, qui simplifie les transactions et les index). Chaque enregistrement synchronisable porte des **champs purement locaux** :

Magasin (store)	Contenu	Particularités
products, categories, exchange_rates	Copie des données de la boutique	Champs locaux : sync_status, sync_attempts, last_error, base_version
sales (lignes imbriquées)	Ventes et leurs lignes	Écriture atomique avec les mouvements (une transaction IndexedDB)
stock_movements	Journal de stock	Ajout seul ; index par produit et par date
shop, profile	Boutique et profil (lecture seule hors ligne)	Rafraîchis à la connexion et à chaque pull
meta	device_id (UUID), curseur de pull, date de dernière synchronisation, taux courant	Valeurs clé/valeur ; device_id généré au premier lancement

Tableau 14.1 : Magasins de la base locale

Champ local	Valeurs	Rôle
sync_status	PENDING · SYNCING · SYNCED · REJECTED	Où en est l'enregistrement (diagramme d'états, chapitre 15)
sync_attempts	entier	Compte les tentatives pour calculer l'attente (backoff)
last_error	code ou null	Dernière réponse d'erreur du serveur (affichée au support)
base_version	entier	Version serveur sur laquelle repose une modification locale (détection de conflit)

Tableau 14.2 : Champs locaux de synchronisation (absents du modèle serveur)

14.4 Détection de la connectivité

- **Ne pas se fier à navigator.onLine** : il indique seulement qu'un réseau est présent, pas qu'Internet fonctionne.
- Méthode retenue : écouter les événements *online* / *offline* **et** confirmer par un appel léger à */health/* avec délai court (quelques secondes). Toute requête en échec est aussi traitée comme « hors ligne ».

- L'état est affiché en permanence (bandeau et compteur « n à synchroniser ») pour que le commerçant sache où il en est.

14.5 Déclencheurs de synchronisation

- Retour de la connexion confirmé ; ouverture de l'application ; fin d'une vente si la connexion est présente (opportuniste) ; minuterie périodique tant que l'application est visible ; bouton manuel.
- **Ne pas dépendre de la Background Sync API** : son support est inégal selon les navigateurs et les systèmes. Elle peut être ajoutée comme amélioration, jamais comme mécanisme principal.
- **Un seul moteur actif à la fois** : un verrou (API Web Locks quand elle existe, sinon verrou applicatif) évite que deux onglets synchronisent en parallèle.

14.6 Persistance du stockage et risque de perte

- Un navigateur peut purger le stockage d'un site peu utilisé ou sous pression d'espace. Mesures : demander le **stockage persistant** (*navigator.storage.persist()*), encourager l'**installation de la PWA** sur l'écran d'accueil, synchroniser dès que possible, afficher le nombre d'éléments non synchronisés.
- Sur iOS, les règles de conservation du stockage des sites web sont plus restrictives : le support est en best effort (H-14) et l'installation sur l'écran d'accueil est fortement recommandée.
- Un **export de secours** des données en attente (fichier) peut être proposé avant une déconnexion ou une désinstallation (évolution).

Risque principal
La perte de données non synchronisées est le risque n°1 d'une application offline-first (R-01). Elle se combat par trois leviers cumulés : synchroniser tôt, rendre le compteur d'éléments en attente très visible, et ne jamais supprimer localement avant confirmation.

14.7 Ce qui fonctionne sans Internet

Cas d'utilisation	Hors ligne
UC01 Créer un compte	Non
UC02 Se connecter	Partiel
UC03 Se déconnecter	Partiel
UC04 Gérer son profil	Lecture seule
UC05 Gérer le taux de change	Oui
UC06 Ajouter un produit	Oui
UC07 Modifier un produit	Oui
UC08 Supprimer (archiver) un produit	Oui
UC09 Consulter les produits	Oui
UC10 Gérer les catégories	Oui
UC11 Définir le stock initial	Oui
UC12 Consulter le stock	Oui
UC13 Consulter les mouvements de stock	Oui
UC14 Réapprovisionner un produit	Oui
UC15 Ajuster le stock	Oui
UC16 Signaler un stock faible	Oui
UC17 Enregistrer les mouvements de stock	Oui
UC18 Enregistrer une vente	Oui
UC19 Ajouter des produits à la vente	Oui
UC20 Calculer automatiquement le total	Oui
UC21 Choisir la devise (CDF ou USD)	Oui
UC22 Annuler une vente	Oui
UC23 Consulter l'historique des ventes	Oui
UC24 Consulter les statistiques	Oui
UC25 Consulter le chiffre d'affaires	Oui
UC26 Consulter le bénéfice	Oui
UC27 Générer un reçu	Oui
UC28 Partager un reçu via WhatsApp	Partiel
UC29 Travailler hors ligne	Oui
UC30 Synchroniser les données	Non (requiert réseau)
UC31 Résoudre un conflit de synchronisation	Non (requiert réseau)

Tableau 14.3 : Disponibilité hors ligne de chaque cas d'utilisation

14.8 Limites connues

- La **première connexion** et la **création de compte** exigent Internet (le serveur doit créer le compte) ; ensuite l'application s'ouvre sans réseau tant que la session locale existe.
- La **modification du profil et de la boutique** exige une connexion ; elle est en lecture seule hors ligne.
- Les **statistiques** portent sur les données locales ; celles d'un autre appareil n'apparaissent qu'après la synchronisation.
- L'**envoi WhatsApp** prépare le message hors ligne, mais WhatsApp lui-même a besoin de sa propre connexion pour partir.

15. Stratégie de synchronisation

15.1 Choix de conception

Deux familles de données, deux méthodes. Les **enregistrements** (produits, catégories, ventes, taux) se synchronisent par leur **état** : on envoie l'enregistrement tel qu'il est. Le **stock** se synchronise par ses **variations** : on envoie les mouvements. Cette distinction supprime la principale cause de conflit (deux appareils qui écrasent chacun le stock de l'autre).

15.2 Périmètre : que synchronise-t-on, dans quel sens, avec quelle règle ?

Entité	Nature	Créée hors ligne	Sens	Règle en cas de conflit
exchange_rate	Ajout seul	Oui	Push + pull	Aucun conflit : insertion idempotente
category	Modifiable	Oui	Push + pull	Version + dernière modification gagnante (updated_at)
product	Modifiable (hors stock)	Oui	Push + pull	Idem ; *current_stock* n'est jamais poussé par le client
stock_movement	Ajout seul	Oui	Push + pull	Aucun conflit ; le serveur recalcule current_stock ; stock négatif toléré et signalé
sale + sale_item	Ajout seul (statut à sens unique)	Oui	Push + pull	Aucun conflit ; l'annulation l'emporte (CANCELLED est terminal)
shop, app_user	Autorité serveur	Non (créés en ligne)	Pull (+ modification en ligne)	Le serveur gagne
currency	Référence figée	Non	Embarquée dans l'application	—
sync_session	Journal serveur	Id généré par le client	Push (journal)	Non synchronisée vers le client

Tableau 15.1 : Périmètre et règles de synchronisation

15.3 Protocole d'envoi (push)

1. Le client sélectionne les enregistrements PENDING (et ERROR réessayables), les **ordonne par dépendance** (taux → catégories → produits → mouvements manuels → ventes avec lignes et mouvements de vente) et les découpe en lots de 50 éléments au plus.
2. Il envoie *POST /sync/push/* avec un **sync_id** (UUID du lot), son **device_id** et les changements groupés par entité.
3. Le serveur authentifie, déduit la boutique de l'utilisateur et traite **chaque élément dans sa propre transaction**.
4. Il répond avec un résultat par élément et les **stocks recalculés** des produits touchés ; il journalise le lot (sync_session).
5. Le client met à jour le statut local de chaque élément (SYNCED, ERROR ou REJECTED) et applique les stocks serveur.

```
POST /api/v1/sync/push/
{
  "sync_id": "c6e8a0b2-4d6f-4a89-93c1-5b7d9f1e3a60",
  "device_id": "d1f3b5a7-9c0e-4b24-a6d8-8e0a2c4f6b19",
  "changes": {
    "sales": [{
      "id": "a8c0e2f4-7b9d-4153-8e6a-2f4c6b8d0a91",
      "exchange_rate_id": "4d6f8a0c-e2b4-4c71-95a3-1e7b9d5c3f08",
      "currency_code": "CDF",
      "receipt_number": "261004-K7QF9M",
      "sold_at": "2026-10-04T10:15:12Z",
      "status": "COMPLETED",
      "total_amount": "21000.00",
      "items": [{
```

```
        "id": "0f2b4d6e-8a1c-4e35-b7d9-6c8a0e2f4b57", "product_id": "e5a7c9b1-6d3f-4a28-8c14-5f0b2e9d7a36",
        "product_name": "Sucre 1 kg", "quantity": 3,
        "unit_price": "7000.00", "unit_cost": "5000.00",
        "stock_movement_id": "91c3e5a7-2b4d-4f68-a0e1-7d9b3c5f8e12"
      }]
    }]
  }
}
```

```
200 OK
{
  "sync_id": "c6e8a0b2-4d6f-4a89-93c1-5b7d9f1e3a60", "status": "PARTIAL", "server_time": "2026-10-04T10:16:03.512Z",
  "results": [
    {"entity": "sales", "id": "a8c0e2f4-7b9d-4153-8e6a-2f4c6b8d0a91", "result": "CREATED"},
    {"entity": "products", "id": "e5a7c9b1-6d3f-4a28-8c14-5f0b2e9d7a36", "result": "DUPLICATE"},
    {"entity": "sales", "id": "a1b2c3d4-0000-4000-8000-000000000001", "result": "REJECTED", "code": "TOTAL_MISMATCH"}
  ],
  "stock": [{"product_id": "e5a7c9b1-6d3f-4a28-8c14-5f0b2e9d7a36", "current_stock": 44}],
  "warnings": [{"product_id": "e5a7c9b1-6d3f-4a28-8c14-5f0b2e9d7a36", "code": "NEGATIVE_STOCK"}]
}
```

15.4 Protocole de réception (pull)

- *GET /sync/pull/?since=<curseur>&limit=200* retourne les enregistrements de la boutique dont **server_updated_at** est postérieur au curseur, par entité, avec un nouveau curseur et un indicateur *has_more*.
- Le curseur est basé sur l'**horloge du serveur**, jamais sur celle du téléphone (qui peut être fausse). Le premier pull d'un nouvel appareil part d'un curseur vide (synchronisation complète, par pages).
- **Recouvrement** : le serveur relit une courte marge (ex. 2 minutes) avant le curseur, car des transactions peuvent s'achever dans un ordre légèrement différent de leur horodatage. Comme appliquer deux fois un même changement est sans effet (upsert par UUID et version), la marge est gratuite.
- À l'application d'un changement reçu, le client **n'écrase jamais** un enregistrement local PENDING : il signale d'abord le conflit potentiel (comparaison de versions).

15.5 Idempotence

Rejouer un lot ne change rien : l'**UUID** identifie l'élément ; un UUID déjà présent dans la boutique renvoie DUPLICATE sans écriture ; la contrainte UNIQUE (sale_item_id, movement_type) interdit tout double décompte de stock. L'idempotence est la propriété qui autorise des tentatives illimitées après un échec réseau, y compris quand la réponse s'est perdue alors que le serveur avait bien enregistré.

15.6 Types de conflits et résolutions

Situation	Détection	Résolution	Résultat
Élément déjà envoyé (réponse perdue, nouvel essai)	UUID déjà présent dans la boutique	Ne rien écrire	DUPLICATE (succès)
Même produit modifié sur deux appareils	base_version ≠ version serveur	Dernière modification gagnante (updated_at, puis le serveur en cas d'égalité) ; l'autre version est journalisée	CONFLICT_RESOLVED
Même unité vendue par deux appareils hors ligne	current_stock < 0 après application	Accepter les deux ventes (elles ont eu lieu) ; signaler	CREATED + avertissement NEGATIVE_STOCK
Vente annulée sur un appareil, consultée sur un autre	status CANCELLED reçu au pull	CANCELLED est terminal : il l'emporte	Mise à jour locale
Dépendance manquante (vente envoyée avant son produit)	FK absente	Rejeter temporairement ; le client réessaie après le produit	REJECTED (MISSING_DEPENDENCY, réessayable)
Total incohérent (bug client)	total ≠ Σ lignes	Rejeter, conserver localement, alerter le support	REJECTED (TOTAL_MISMATCH)
UUID existant dans une autre boutique	shop_id différent	Rejeter sans révéler l'existence	REJECTED (FORBIDDEN)
Horloge du téléphone aberrante	sold_at très éloigné de l'heure serveur	Accepter si plausible ; au-delà (ex. > 48 h dans le futur), rejeter	CREATED ou REJECTED (CLOCK_SKEW)

Tableau 15.2 : Catalogue des conflits

15.7 Le stock courant côté client

Pour que le stock affiché reste juste hors ligne *et* après la synchronisation, le client applique la règle : **stock affiché = stock confirmé par le serveur + Σ mouvements locaux non encore confirmés**. Quand le serveur renvoie le stock recalculé d'un produit, le client remplace le « stock confirmé » ; les mouvements encore PENDING restent ajoutés par-dessus. Ainsi une vente faite pendant une synchronisation n'est jamais perdue ni comptée deux fois.

15.8 Erreurs et nouvelles tentatives

Réponse	Nature	Action du client
Échec réseau, délai dépassé, 5xx, 429	Transitoire	Remettre PENDING ; réessayer avec attente exponentielle (5 s, 15 s, 1 min, 5 min, plafond 15 min) + aléa (jitter)
401 token expiré	Session	Renouveler le jeton puis rejouer ; si impossible, suspendre et demander la reconnexion (données conservées)
REJECTED réessayable (MISSING_DEPENDENCY)	Ordre	Remettre PENDING après les dépendances
REJECTED définitif (TOTAL_MISMATCH, FORBIDDEN, CLOCK_SKEW)	Données	Marquer REJECTED, conserver, avertir le commerçant et permettre l'export pour le support
200 DUPLICATE / CREATED / UPDATED / CONFLICT_RESOLVED	Succès	Marquer SYNCED, enregistrer version et server_updated_at

Tableau 15.3 : Traitement des réponses

15.9 États d'un enregistrement local

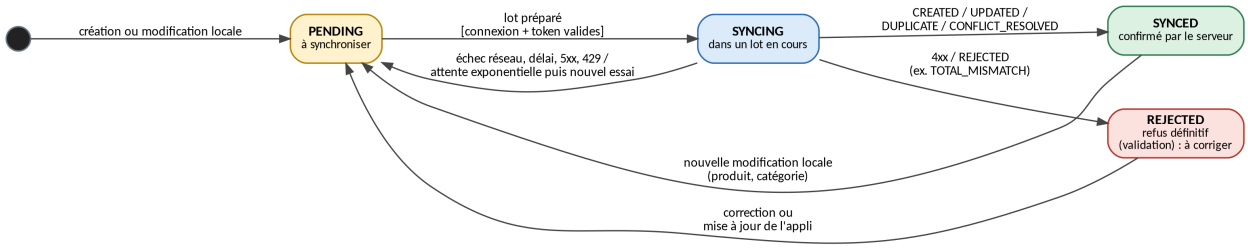


Figure 15.1 : Diagramme d'états : statut de synchronisation d'un enregistrement local

15.10 Journal de synchronisation (serveur)

Chaque lot crée une ligne *sync_session* : appareil, début et fin, statut (IN_PROGRESS, SUCCESS, PARTIAL, FAILED), compteurs (reçus, acceptés, doublons, conflits, rejetés) et détails JSON des anomalies. Ce journal répond à la question du support « pourquoi ma vente n'apparaît-elle pas ? » sans fouiller les journaux applicatifs. Il n'est pas renvoyé au client et peut être purgé après 90 jours.

15.11 Plan de test de la synchronisation

- Lot rejoué deux fois : un seul enregistrement, résultat DUPLICATE au second passage.
- Coupure réseau au milieu d'un lot : aucun doublon, aucune perte, reprise correcte.
- Deux appareils vendent la dernière unité hors ligne : deux ventes, stock −1 signalé.
- Deux appareils modifient le prix d'un même produit : conflit résolu et journalisé.
- Vente envoyée avant son produit : rejet réessayable puis succès.
- Horloge du téléphone décalée de plusieurs jours : curseur de pull correct (basé serveur).
- Jeton expiré pendant un lot : renouvellement transparent puis rejeu.
- Tentative d'envoi d'un UUID d'une autre boutique : rejet FORBIDDEN, sans fuite d'information.

16. Gestion des identifiants UUID

16.1 Pourquoi des UUID

Un identifiant auto-incrémenté (1, 2, 3…) est attribué par la base : il suppose une base centrale joignable. Hors ligne, deux téléphones créeraient tous deux la « vente n°42 ». Un **UUID v4** (128 bits dont 122 aléatoires) est généré **sur l'appareil**, sans coordination, avec une probabilité de collision négligeable (de l'ordre de 10^{-19} pour un milliard d'identifiants). C'est ce qui permet l'écriture locale d'abord et la synchronisation idempotente.

16.2 Décision entité par entité

Table	Identifiant	Créée hors ligne	Justification
product, category	UUID v4 (client)	Oui	Créés à la caisse sans réseau ; l'identifiant sert de clé d'idempotence.
sale, sale_item	UUID v4 (client)	Oui	La vente est l'objet le plus critique ; son UUID garantit qu'elle n'est enregistrée qu'une fois.
stock_movement	UUID v4 (client)	Oui	Chaque variation doit être unique et traçable ; le serveur ne la régénère pas.
exchange_rate	UUID v4 (client)	Oui	Saisi hors ligne ; référencé par les ventes.
sync_session	UUID v4 (client)	Oui (id du lot)	Clé d'idempotence du lot ; permet de relier réponse et requête.
app_user	UUID v4 (serveur)	Non (création en ligne)	Pas d'énumération (/users/1, /users/2…), identifiant stable dans le jeton JWT (claim sub), type uniforme dans toute l'API.
shop	UUID v4 (serveur)	Non	Racine de tout le modèle ; même motifs que l'utilisateur.
currency	**CHAR(3) ISO 4217** (clé naturelle)	—	Donnée de référence figée, deux lignes, connue du client en dur : un UUID n'apporterait que de la complexité.
Tables internes Django / SimpleJWT	Entier (par défaut du framework)	—	auth_permission, django_migrations, token_blacklist_* : non métier, jamais exposées, laissées telles quelles.

Tableau 16.1 : Choix de l'identifiant par table (UUID partout où c'est justifié, pas ailleurs)

16.3 Version, génération et format

- **Version 4** (aléatoire), standardisée par la RFC 9562 (qui remplace la RFC 4122). Client : *crypto.randomUUID()* (disponible dans les contextes sécurisés, donc en HTTPS). Serveur : *uuid.uuid4()* de Python.
- **Stockage** : type uuid natif en PostgreSQL (16 octets), CHAR(36) ou BINARY(16) en MySQL ; en JSON, chaîne canonique en minuscules (« 8-4-4-4-12 »).
- **Alternative à étudier plus tard** : UUID v7 (ordonné dans le temps), qui améliore la localité des index sur de très grosses tables. Inutile aux volumes de Doku (H-16) ; le passage resterait possible sans changer le schéma (même type).

16.4 Sécurité des identifiants fournis par le client

- **Validation stricte du format** à l'entrée (sérialiseur) : une chaîne qui n'est pas un UUID valide est rejetée (400).
- **Un UUID n'est pas un secret** et ne remplace pas l'autorisation : toute lecture ou écriture est filtrée par la boutique de l'utilisateur authentifié.
- **Anti-détournement** : un client malveillant pourrait envoyer l'UUID d'un enregistrement d'une autre boutique ; le serveur le rejette (FORBIDDEN) sans révéler s'il existe.
- Le **shop_id n'est jamais pris dans la requête** ; il est imposé par le serveur (chapitre 17).

16.5 Numéro de reçu : lisible, sans compteur

Les UUID sont illisibles pour un client qui rappelle son reçu. Le **numéro de reçu** (*AAMMJJ-XXXXXX*) est dérivé de l'UUID de la vente : date locale + six caractères d'un alphabet de 32 symboles sans ambiguïté (type Crockford : sans I, L, O, U). Un compteur séquentiel (1, 2, 3…) est impossible hors ligne sans coordination entre appareils. Le numéro n'est pas contraint UNIQUE (un rejet à la synchronisation serait pire que l'homonymie) ; la clé fiable reste l'UUID.

Ventes dans la même journée	Probabilité qu'au moins deux reçus aient le même numéro
100	environ 0,0005 % par jour (30 bits : $n^2 \div 2^{31}$)
500	environ 0,012 % par jour

Tableau 16.2 : Risque d'homonymie du numéro de reçu (la recherche par numéro affiche alors les deux ventes)

17. Principes de sécurité

17.1 Actifs à protéger et menaces

Les données de Doku sont des **données commerciales sensibles** (ventes, marges, stocks) : leur divulgation à un concurrent ou leur altération nuirait directement au commerçant. Elles sont de plus conservées sur un téléphone qui peut être perdu, prêté ou volé.

Menace	Exemple	Contre-mesure principale
Accès à la boutique d'un autre commerçant (IDOR)	Changer un identifiant dans une requête	Filtre systématique par boutique côté serveur ; shop_id jamais lu dans la requête ; UUID non devinables mais non considérés comme secrets
Vol ou rejeu de jeton	Jeton copié depuis le stockage du navigateur	Access token court (15 min) en mémoire ; refresh en cookie HttpOnly, rotatif, en liste noire
Devinette de mot de passe	Essais répétés sur /auth/login/	Limitation de débit, message d'erreur générique, hachage Argon2
Injection SQL	Chaîne forgée dans un champ de recherche	ORM Django (requêtes paramétrées), aucune concaténation SQL
XSS (script injecté)	Nom de produit contenant du HTML	Échappement natif de React ; pas de dangerouslySetInnerHTML ; politique CSP
CSRF	Site tiers qui déclenche une action	Authentification par en-tête Bearer ; cookie refresh en SameSite + en-tête personnalisé ; CORS restreint
Interception réseau	Wi-Fi public	HTTPS partout, HSTS, cookies Secure
Téléphone perdu ou prêté	Quelqu'un ouvre l'application	Session limitée, déconnexion explicite ; verrouillage par code PIN envisagé (évolution)
Données altérées à la synchronisation	Client modifié qui envoie un total faux	Validation serveur (total = Σ lignes), contraintes CHECK, journal de synchronisation
Fuite via les journaux	Mot de passe ou jeton dans un log	Journaux structurés sans donnée sensible ; masquage des champs

Tableau 17.1 : Menaces et contre-mesures

17.2 Authentification

- **Identifiant** : téléphone E.164 + mot de passe (H-01). **Mots de passe** : jamais stockés en clair ; hachage **Argon2** (ou PBKDF2 de Django en repli) ; politique minimale (longueur, liste de mots de passe courants) via les validateurs de Django.
- **JWT (SimpleJWT)** : *access* à 15 minutes, *refresh* à 30 jours avec **rotation** et **liste noire** après rotation. Valeurs par défaut de la bibliothèque (5 min / 1 jour, sans rotation) à remplacer ; l'application de liste noire (token_blacklist) est à ajouter aux applications installées.
- **Hors ligne** : l'application fonctionne sans jeton valide (tout est local) ; le jeton n'est requis qu'à la synchronisation. Un refresh expiré (longue absence de réseau) impose une reconnexion **sans effacer les données en attente**.
- **Stockage côté client** : access token en **mémoire** ; refresh en cookie **HttpOnly + Secure + SameSite=Lax**, posé par le serveur (vues de connexion/renouvellement personnalisées au-dessus de SimpleJWT). Hypothèse : application et API sous le même domaine (sous-domaines), sinon SameSite=None et protection CSRF renforcée.
- **Limitation de débit** : throttling DRF (login plus strict) ; extension possible avec un blocage temporaire après échecs répétés.

17.3 Autorisation et isolation par boutique

Chaque ressource appartient à une boutique ; chaque boutique à un utilisateur. L'autorisation est donc simple mais **non négociable** : tout queryset est filtré sur la boutique de l'utilisateur authentifié, et toute création impose cette boutique. Les *permissions* DRF (IsAuthenticated, et une permission d'objet pour les accès par identifiant) sont complétées par ce filtre, car un identifiant valide n'est pas une autorisation.

```
# catalog/views.py (extrait)
class ShopScopedMixin:
    """Toute requête est limitée à la boutique de l'utilisateur authentifié (RG-02)."""
    def get_queryset(self):
        return super().get_queryset().filter(shop=self.request.user.shop)

    def perform_create(self, serializer):
        serializer.save(shop=self.request.user.shop)           # jamais lu dans la requête


class ProductViewSet(ShopScopedMixin, viewsets.ModelViewSet):
    queryset = Product.objects.filter(deleted_at__isnull=True).select_related('category')
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticated]
    filter_backends = [SearchFilter, OrderingFilter]
    search_fields = ['name']

    def perform_destroy(self, instance):                       # DELETE = archivage (RG-04)
        instance.deleted_at = timezone.now()
        instance.version += 1
        instance.save(update_fields=['deleted_at', 'version', 'server_updated_at'])
```

17.4 Validation des données

Couche	Rôle	Exemples
Client (React)	Confort : retours immédiats	Champs obligatoires, quantité entière > 0, format du numéro WhatsApp
Serveur : serializers DRF	**Barrière de sécurité** : tout est revalidé	Types, longueurs, UUID valides, devises connues, boutique imposée, montants en décimal
Serveur : services métier	Règles métier	Total = Σ lignes, quantités positives, dépendances présentes, plausibilité de la date
Base de données	Dernier filet	CHECK (prix ≥ 0, signes des mouvements), FK, UNIQUE (sale_item, type)

Tableau 17.2 : Validation en quatre couches (la couche client n'est jamais une protection)

17.5 Injections, XSS et CSRF

- **SQL** : uniquement l'ORM ; si une requête brute est inévitable, utiliser les paramètres (jamais de f-string dans le SQL).
- **XSS** : React échappe par défaut ; interdire *dangerouslySetInnerHTML* ; encoder le texte du reçu dans l'URL WhatsApp (encodeURIComponent) ; en-tête *Content-Security-Policy* sur le frontend.
- **CSRF** : l'API authentifiée par en-tête *Authorization* n'est pas exposée au CSRF ; l'unique endpoint à cookie (/auth/refresh/) exige un en-tête personnalisé et SameSite, et n'accepte que les origines autorisées.

17.6 CORS, transport et configuration

```
# settings.py (extraits)
from datetime import timedelta

INSTALLED_APPS += ['rest_framework', 'rest_framework_simplejwt.token_blacklist', 'corsheaders',
                   'core', 'accounts', 'catalog', 'inventory', 'sales', 'sync']
AUTH_USER_MODEL = 'accounts.User'
MIDDLEWARE = ['corsheaders.middleware.CorsMiddleware',           # avant CommonMiddleware
              'django.middleware.security.SecurityMiddleware', ...]
USE_TZ = True                                                    # horodatages en UTC
PASSWORD_HASHERS = ['django.contrib.auth.hashers.Argon2PasswordHasher',   # pip install argon2-cffi
                    'django.contrib.auth.hashers.PBKDF2PasswordHasher']

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': ['rest_framework_simplejwt.authentication.JWTAuthentication'],
    'DEFAULT_PERMISSION_CLASSES': ['rest_framework.permissions.IsAuthenticated'],
    'DEFAULT_THROTTLE_CLASSES': ['rest_framework.throttling.AnonRateThrottle', 'rest_framework.throttling.UserRateThrottle'],
    'DEFAULT_THROTTLE_RATES': {'anon': '30/min', 'user': '600/min'},
    'EXCEPTION_HANDLER': 'core.api.exception_handler',           # format d'erreur unique
}
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=15),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=30),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'UPDATE_LAST_LOGIN': True,
}
CORS_ALLOWED_ORIGINS = ['https://app.doku.example']            # liste explicite, jamais '*' avec les cookies
CORS_ALLOW_CREDENTIALS = True
CSRF_TRUSTED_ORIGINS = ['https://app.doku.example']
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000
SESSION_COOKIE_SECURE = CSRF_COOKIE_SECURE = True
```

- **django-cors-headers** : lister explicitement les origines ; jamais *CORS_ALLOW_ALL_ORIGINS* en production ; *CORS_ALLOW_CREDENTIALS* seulement pour le cookie de renouvellement.
- **Secrets** (clé secrète, base de données) dans les variables d'environnement, jamais dans le dépôt ; DEBUG désactivé ; ALLOWED_HOSTS renseigné.

17.7 Protection des données commerciales et personnelles

- **Minimisation** : le numéro WhatsApp du client n'est pas conservé (H-08) ; pas de fichier clients en MVP.
- **Cloisonnement** : aucune requête ne peut croiser deux boutiques ; les sauvegardes de la base sont chiffrées et leur accès restreint.
- **Suppression de compte** : procédure explicite (les clés étrangères en PROTECT empêchent une suppression en cascade silencieuse des ventes) ; durée de conservation à définir avec le commerçant.
- **Journaux** : identifiants de requête, utilisateur, boutique, code d'erreur ; *jamais* de mot de passe, de jeton ni de contenu de vente complet.

17.8 Gestion des erreurs

- Format d'erreur unique (chapitre 19) ; pas de trace d'exécution ni de détail interne dans les réponses.
- Messages d'authentification génériques ; codes d'erreur stables et exploitables par le client (TOTAL_MISMATCH, FORBIDDEN…).
- Supervision des erreurs (par exemple un service de suivi d'erreurs) et alertes sur les pics d'échecs de connexion ou de rejets de synchronisation.

17.9 Données stockées sur le téléphone

IndexedDB n'est pas chiffrée. Un téléphone déverrouillé donne accès aux données de l'application ; c'est un risque accepté en MVP et atténué par : session à durée limitée, déconnexion explicite, purge des données après une déconnexion *seulement si tout est synchronisé*, et, comme évolution, un code PIN d'ouverture.

Bonne pratique : tests d'isolation
Écrire dès le premier sprint trois tests de sécurité automatiques : (1) un utilisateur A ne peut lire ni modifier aucune ressource de la boutique B ; (2) un lot de synchronisation contenant un UUID de B est rejeté ; (3) une requête sans jeton reçoit 401. Ils protègent contre la régression la plus grave possible.

18. Correspondance avec Django

18.1 Organisation en applications

Application	Modèles	Contient aussi
core	Currency, ExchangeRate, classes abstraites (ShopOwned, Timestamped, Mutable)	pricing.py (conversion, arrondi), format d'erreur de l'API
accounts	User, Shop	Inscription, connexion, vues cookie/JWT, permissions
catalog	Category, Product	Serializers et ViewSets du catalogue
inventory	StockMovement, MovementType	services.py : apply_movement ; commande de réconciliation du stock
sales	Sale, SaleItem, SaleStatus	services.py : create_sale, cancel_sale ; stats.py ; reçu (JSON)
sync	SyncSession	Vues push/pull, services de traitement du lot

Tableau 18.1 : Applications Django et responsabilités

18.2 Table → modèle → ressource API

Entité Merise	Table (MLD)	Modèle Django	Ressource API
DEVISE	`currency`	core.Currency	/currencies/
UTILISATEUR	`app_user`	accounts.User	/users/me/, /auth/
BOUTIQUE	`shop`	accounts.Shop	/shops/me/
CATEGORIE	`category`	catalog.Category	/categories/
PRODUIT	`product`	catalog.Product	/products/, /stock/
TAUX_CHANGE	`exchange_rate`	core.ExchangeRate	/exchange-rates/
VENTE	`sale`	sales.Sale	/sales/
LIGNE_VENTE	`sale_item`	sales.SaleItem	/sales/ (imbriqué)
MOUVEMENT_STOCK	`stock_movement`	inventory.StockMovement	/stock/movements/
SYNCHRONISATION	`sync_session`	sync.SyncSession	/sync/ (journal)

Tableau 18.2 : Correspondance entre les couches

18.3 Règles de nommage entre couches

- **Base ↔ Django ↔ JSON** : mêmes noms en *snake_case* anglais. Pour les relations, l'attribut Django porte le nom sans suffixe (*shop*, *category*, *purchase_currency*) ; la colonne est *shop_id*, *category_id*, *purchase_currency_code* (via *db_column* pour les devises) ; l'API expose le nom de la **colonne** grâce à *source=* dans les serializers.
- **Enumérations** : *models.TextChoices* (MovementType, SaleStatus) → colonnes VARCHAR avec CHECK ; mêmes valeurs en JSON et en TypeScript.
- **Montants** : *DecimalField* → NUMERIC ; sérialisés en chaînes par DRF (comportement par défaut), jamais en flottants.
- **Table user** : *db_table = 'app_user'* et *AUTH_USER_MODEL = 'accounts.User'* à définir **avant la première migration** (changer plus tard est très coûteux).

> **Piège classique**
> Définir le modèle utilisateur personnalisé **avant** la toute première migration. Remplacer le modèle utilisateur d'un projet déjà migré est l'une des opérations les plus pénibles de Django.

18.4 Modèles

Les extraits suivants ont été **vérifiés** dans un projet Django de test (contrôle de configuration, génération des migrations et scénario complet : conversion, vente idempotente, stock, annulation, rejet d'un total faux avec annulation de transaction). Ils s'écrivent avec *CheckConstraint(condition=…)*, la syntaxe en vigueur depuis Django 5.1 et valable pour la version LTS 5.2 comme pour les versions 6.x.

```
# core/models.py
import uuid
from django.db import models
from django.db.models import F, Q


class Currency(models.Model):
    code = models.CharField(max_length=3, primary_key=True)           # ISO 4217
    name = models.CharField(max_length=50)
    symbol = models.CharField(max_length=5)
    decimal_places = models.PositiveSmallIntegerField()

    class Meta:
        db_table = 'currency'
        constraints = [models.CheckConstraint(condition=Q(decimal_places__lte=3), name='currency_decimals_ck')]


class ShopOwned(models.Model):
    """Base : UUID généré par le client, boutique (tenant), horloge serveur."""
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    shop = models.ForeignKey('accounts.Shop', on_delete=models.PROTECT)
    server_updated_at = models.DateTimeField(auto_now=True)           # curseur du pull

    class Meta:
        abstract = True


class Timestamped(ShopOwned):
    created_at = models.DateTimeField()                               # horloge de l'appareil

    class Meta:
        abstract = True


class Mutable(Timestamped):
    updated_at = models.DateTimeField()                               # horloge de l'appareil
    version = models.PositiveIntegerField(default=1)
    deleted_at = models.DateTimeField(null=True, blank=True)          # suppression logique

    class Meta:
        abstract = True


class ExchangeRate(Timestamped):                                      # ajout seul
    base_currency = models.ForeignKey(Currency, on_delete=models.PROTECT, db_column='base_currency_code', related_name='+')
    quote_currency = models.ForeignKey(Currency, on_delete=models.PROTECT, db_column='quote_currency_code', related_name='+')
    rate = models.DecimalField(max_digits=18, decimal_places=6)
    effective_at = models.DateTimeField()

    class Meta:
        db_table = 'exchange_rate'
        constraints = [
            models.CheckConstraint(condition=Q(rate__gt=0), name='exrate_positive_ck'),
            models.CheckConstraint(condition=~Q(base_currency=F('quote_currency')), name='exrate_pair_ck'),
        ]
        indexes = [models.Index(fields=['shop', 'base_currency', 'quote_currency', '-effective_at'], name='exrate_lookup_idx'),
                   models.Index(fields=['shop', 'server_updated_at'], name='exrate_pull_idx')]
```

```
# accounts/models.py
import uuid
from django.conf import settings
from django.contrib.auth.models import AbstractBaseUser, BaseUserManager, PermissionsMixin
from django.db import models


class UserManager(BaseUserManager):
    use_in_migrations = True

    def create_user(self, phone, password=None, **extra):
        if not phone:
            raise ValueError('Le numéro de téléphone est obligatoire')
        user = self.model(phone=phone, **extra)
        user.set_password(password)
        user.save(using=self._db)
        return user
```

```
    def create_superuser(self, phone, password=None, **extra):
        extra.setdefault('is_staff', True)
        extra.setdefault('is_superuser', True)
        return self.create_user(phone, password, **extra)


class User(AbstractBaseUser, PermissionsMixin):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    phone = models.CharField(max_length=20, unique=True)               # E.164, identifiant de connexion
    email = models.EmailField(max_length=254, unique=True, null=True, blank=True)
    full_name = models.CharField(max_length=150)
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)                      # accès /admin/ (équipe technique)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    USERNAME_FIELD = 'phone'
    REQUIRED_FIELDS = ['full_name']
    objects = UserManager()

    class Meta:
        db_table = 'app_user'                                          # « user » est réservé en PostgreSQL


class Shop(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    owner = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.PROTECT, related_name='shop')  # MVP : 1 boutique
    name = models.CharField(max_length=150)
    address = models.CharField(max_length=255, null=True, blank=True)
    phone = models.CharField(max_length=20, null=True, blank=True)
    reference_currency = models.ForeignKey('core.Currency', on_delete=models.PROTECT, db_column='reference_currency_code',
                                           related_name='+', default='CDF')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'shop'
```

```
# catalog/models.py
from django.db import models
from django.db.models import Q
from core.models import Currency, Mutable


class Category(Mutable):
    name = models.CharField(max_length=100)

    class Meta:
        db_table = 'category'
        indexes = [models.Index(fields=['shop', 'server_updated_at'], name='cat_pull_idx'),
                   models.Index(fields=['shop', 'deleted_at'], name='cat_active_idx')]


class Product(Mutable):
    category = models.ForeignKey(Category, null=True, blank=True, on_delete=models.SET_NULL)
    name = models.CharField(max_length=150)
    purchase_price = models.DecimalField(max_digits=14, decimal_places=2)
    purchase_currency = models.ForeignKey(Currency, on_delete=models.PROTECT, db_column='purchase_currency_code', related_name='+')
    selling_price = models.DecimalField(max_digits=14, decimal_places=2)
    selling_currency = models.ForeignKey(Currency, on_delete=models.PROTECT, db_column='selling_currency_code', related_name='+')
    current_stock = models.IntegerField(default=0)                     # dérivé des mouvements ; jamais poussé par le client
    low_stock_threshold = models.PositiveIntegerField(default=0)

    class Meta:
        db_table = 'product'
        constraints = [models.CheckConstraint(condition=Q(purchase_price__gte=0) & Q(selling_price__gte=0), name='product_prices_ck')]
        indexes = [models.Index(fields=['shop', 'deleted_at', 'name'], name='prod_list_idx'),
                   models.Index(fields=['shop', 'category'], name='prod_cat_idx'),
                   models.Index(fields=['shop', 'server_updated_at'], name='prod_pull_idx')]

    def is_low_stock(self) -> bool:
        return self.current_stock <= self.low_stock_threshold
```

```
# inventory/models.py
from django.db import models
from django.db.models import Q
from core.models import Timestamped


class MovementType(models.TextChoices):
    INITIAL = 'INITIAL'
    RESTOCK = 'RESTOCK'
    SALE = 'SALE'
    SALE_CANCEL = 'SALE_CANCEL'
    ADJUSTMENT = 'ADJUSTMENT'
```

```
class StockMovement(Timestamped):                                    # ajout seul : ni UPDATE ni DELETE
    product = models.ForeignKey('catalog.Product', on_delete=models.PROTECT)
    sale_item = models.ForeignKey('sales.SaleItem', null=True, blank=True, on_delete=models.PROTECT, related_name='movements')
    movement_type = models.CharField(max_length=20, choices=MovementType.choices)
    quantity = models.IntegerField()                                 # signée : + entrée, - sortie
    reason = models.CharField(max_length=255, null=True, blank=True)

    class Meta:
        db_table = 'stock_movement'
        constraints = [
            models.CheckConstraint(condition=~Q(quantity=0), name='mov_qty_nonzero_ck'),
            models.CheckConstraint(condition=~Q(movement_type='SALE') | Q(quantity__lt=0), name='mov_sale_negative_ck'),
            models.CheckConstraint(condition=~Q(movement_type__in=['INITIAL', 'RESTOCK', 'SALE_CANCEL']) | Q(quantity__gt=0),
                                   name='mov_entry_positive_ck'),
            models.CheckConstraint(condition=~Q(movement_type='ADJUSTMENT') | Q(reason__isnull=False), name='mov_adjust_reason_ck'),
            models.UniqueConstraint(fields=['sale_item', 'movement_type'], name='mov_item_type_uq'),
        ]
        indexes = [models.Index(fields=['product', 'created_at'], name='mov_product_idx'),
                   models.Index(fields=['shop', 'server_updated_at'], name='mov_pull_idx')]
```

```
# sales/models.py
import uuid
from django.db import models
from django.db.models import Q
from core.models import ShopOwned


class SaleStatus(models.TextChoices):
    COMPLETED = 'COMPLETED'
    CANCELLED = 'CANCELLED'


class Sale(ShopOwned):                                               # porte sold_at (et non created_at)
    exchange_rate = models.ForeignKey('core.ExchangeRate', on_delete=models.PROTECT, related_name='sales')
    currency = models.ForeignKey('core.Currency', on_delete=models.PROTECT, db_column='currency_code', related_name='+')
    receipt_number = models.CharField(max_length=16)
    sold_at = models.DateTimeField()
    status = models.CharField(max_length=10, choices=SaleStatus.choices, default=SaleStatus.COMPLETED)
    total_amount = models.DecimalField(max_digits=14, decimal_places=2)
    cancelled_at = models.DateTimeField(null=True, blank=True)
    cancel_reason = models.CharField(max_length=255, null=True, blank=True)

    class Meta:
        db_table = 'sale'
        constraints = [
            models.CheckConstraint(condition=Q(total_amount__gte=0), name='sale_total_ck'),
            models.CheckConstraint(condition=(Q(status='CANCELLED') & Q(cancelled_at__isnull=False))
                                   | (~Q(status='CANCELLED') & Q(cancelled_at__isnull=True)), name='sale_cancel_ck'),
        ]
        indexes = [models.Index(fields=['shop', '-sold_at'], name='sale_history_idx'),
                   models.Index(fields=['shop', 'receipt_number'], name='sale_receipt_idx'),
                   models.Index(fields=['shop', 'server_updated_at'], name='sale_pull_idx')]


class SaleItem(models.Model):                                        # instantané : jamais recalculé
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    sale = models.ForeignKey(Sale, on_delete=models.CASCADE, related_name='items')
    product = models.ForeignKey('catalog.Product', on_delete=models.PROTECT)
    product_name = models.CharField(max_length=150)
    quantity = models.PositiveIntegerField()
    unit_price = models.DecimalField(max_digits=14, decimal_places=2)
    unit_cost = models.DecimalField(max_digits=14, decimal_places=2)

    class Meta:
        db_table = 'sale_item'
        constraints = [models.CheckConstraint(condition=Q(quantity__gt=0), name='item_qty_ck'),
                       models.CheckConstraint(condition=Q(unit_price__gte=0) & Q(unit_cost__gte=0), name='item_amounts_ck')]
```

```
# sync/models.py
import uuid
from django.db import models


class SyncSession(models.Model):                                     # journal technique, non synchronisé vers le client
    class Status(models.TextChoices):
        IN_PROGRESS = 'IN_PROGRESS'
        SUCCESS = 'SUCCESS'
        PARTIAL = 'PARTIAL'
        FAILED = 'FAILED'

    id = models.UUIDField(primary_key=True, editable=False)          # généré par le client (clé d'idempotence du lot)
    shop = models.ForeignKey('accounts.Shop', on_delete=models.PROTECT)
    device_id = models.UUIDField()
    started_at = models.DateTimeField()
    finished_at = models.DateTimeField(null=True, blank=True)
```

```
    status = models.CharField(max_length=12, choices=Status.choices, default=Status.IN_PROGRESS)
    received_count = models.IntegerField(default=0)
    accepted_count = models.IntegerField(default=0)
    duplicate_count = models.IntegerField(default=0)
    conflict_count = models.IntegerField(default=0)
    rejected_count = models.IntegerField(default=0)
    error_details = models.JSONField(null=True, blank=True)

    class Meta:
        db_table = 'sync_session'
        indexes = [models.Index(fields=['shop', '-started_at'], name='sync_recent_idx')]
```

18.5 Services métier (couche applicative)

```
# core/pricing.py
from decimal import Decimal, ROUND_HALF_UP


def quantize(value: Decimal, decimals: int) -> Decimal:
    return value.quantize(Decimal(1).scaleb(-decimals), rounding=ROUND_HALF_UP)


def convert_price(amount: Decimal, src: str, dst: str, usd_cdf: Decimal, decimals_of: dict) -> Decimal:
    """usd_cdf = nombre de CDF pour 1 USD (valeur de exchange_rate.rate). Un seul arrondi, ici (RG-12)."""
    if src == dst:
        value = amount
    elif (src, dst) == ('USD', 'CDF'):
        value = amount * usd_cdf
    elif (src, dst) == ('CDF', 'USD'):
        value = amount / usd_cdf
    else:
        raise ValueError(f'Paire non prise en charge : {src} -> {dst}')
    return quantize(value, decimals_of[dst])
```

```
# inventory/services.py
import uuid
from django.db.models import F
from django.utils import timezone
from catalog.models import Product
from inventory.models import StockMovement


def apply_movement(product, movement_type, quantity, *, sale_item=None, reason=None, movement_id=None, created_at=None):
    """Seule porte d'entrée pour modifier le stock (RG-05). À appeler dans une transaction."""
    movement = StockMovement.objects.create(
        id=movement_id or uuid.uuid4(), shop_id=product.shop_id, product=product, sale_item=sale_item,
        movement_type=movement_type, quantity=quantity, reason=reason, created_at=created_at or timezone.now())
    # mise à jour atomique par la base ; server_updated_at alimente le pull ; version n'est pas touchée
    Product.objects.filter(pk=product.pk).update(current_stock=F('current_stock') + quantity,
                                                 server_updated_at=timezone.now())
    return movement
```

```
# sales/services.py
from decimal import Decimal
from django.core.exceptions import PermissionDenied, ValidationError
from django.db import transaction
from django.utils import timezone
from catalog.models import Product
from core.models import ExchangeRate
from inventory.models import MovementType
from inventory.services import apply_movement
from sales.models import Sale, SaleItem, SaleStatus


@transaction.atomic
def create_sale(shop, data):
    """Crée une vente de façon idempotente. Retourne (vente, créée ?)."""
    existing = Sale.objects.filter(pk=data['id']).first()
    if existing:
        if existing.shop_id != shop.id:
            raise PermissionDenied('FORBIDDEN')            # UUID d'une autre boutique : rejet sans détail
        return existing, False                             # DUPLICATE : succès idempotent
    rate = ExchangeRate.objects.get(pk=data['exchange_rate_id'], shop=shop)
    sale = Sale.objects.create(id=data['id'], shop=shop, exchange_rate=rate, currency_id=data['currency_code'],
                               receipt_number=data['receipt_number'], sold_at=data['sold_at'], total_amount=Decimal('0'))
    total = Decimal('0')
    for it in data['items']:
        product = Product.objects.select_for_update().get(pk=it['product_id'], shop=shop)
        item = SaleItem.objects.create(id=it['id'], sale=sale, product=product, product_name=it['product_name'],
                                       quantity=it['quantity'], unit_price=Decimal(it['unit_price']),
                                       unit_cost=Decimal(it['unit_cost']))
```

```
        total += item.quantity * item.unit_price
        apply_movement(product, MovementType.SALE, -item.quantity, sale_item=item,
                       movement_id=it.get('stock_movement_id'), created_at=sale.sold_at)
    if total != Decimal(data['total_amount']):
        raise ValidationError('TOTAL_MISMATCH')           # la transaction de la vente est annulée
    sale.total_amount = total
    sale.save(update_fields=['total_amount', 'server_updated_at'])
    return sale, True


@transaction.atomic
def cancel_sale(shop, sale_id, reason=''):
    sale = Sale.objects.select_for_update().get(pk=sale_id, shop=shop)
    if sale.status == SaleStatus.CANCELLED:
        return sale                                       # idempotent
    for item in sale.items.select_related('product'):
        apply_movement(item.product, MovementType.SALE_CANCEL, item.quantity, sale_item=item)
    sale.status, sale.cancelled_at, sale.cancel_reason = SaleStatus.CANCELLED, timezone.now(), reason or None
    sale.save(update_fields=['status', 'cancelled_at', 'cancel_reason', 'server_updated_at'])
    return sale
```

Décision d'architecture

Pourquoi des fonctions de service plutôt que des signaux ou de la logique dans les vues : la règle « une vente crée ses mouvements et décrémente le stock » est ainsi écrite **à un seul endroit**, appelée à l'identique par l'API en ligne et par la synchronisation, et testable sans serveur HTTP.

18.6 Réglages de sécurité et vues

Les réglages de sécurité (JWT, CORS, throttling) sont donnés au chapitre 17. Exemple d'un ViewSet filtré par boutique avec archivage (la suppression est un archivage) :

```
# catalog/views.py (extrait)
class ShopScopedMixin:
    """Toute requête est limitée à la boutique de l'utilisateur authentifié (RG-02)."""
    def get_queryset(self):
        return super().get_queryset().filter(shop=self.request.user.shop)

    def perform_create(self, serializer):
        serializer.save(shop=self.request.user.shop)           # jamais lu dans la requête


class ProductViewSet(ShopScopedMixin, viewsets.ModelViewSet):
    queryset = Product.objects.filter(deleted_at__isnull=True).select_related('category')
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticated]
    filter_backends = [SearchFilter, OrderingFilter]
    search_fields = ['name']

    def perform_destroy(self, instance):                       # DELETE = archivage (RG-04)
        instance.deleted_at = timezone.now()
        instance.version += 1
        instance.save(update_fields=['deleted_at', 'version', 'server_updated_at'])


class ProductSerializer(serializers.ModelSerializer):
    category_id = serializers.PrimaryKeyRelatedField(source='category', queryset=Category.objects.all(), allow_null=True, required=False
    purchase_currency_code = serializers.PrimaryKeyRelatedField(source='purchase_currency', queryset=Currency.objects.all())
    selling_currency_code = serializers.PrimaryKeyRelatedField(source='selling_currency', queryset=Currency.objects.all())

    class Meta:
        model = Product
        fields = ['id', 'category_id', 'name', 'purchase_price', 'purchase_currency_code', 'selling_price',
                  'selling_currency_code', 'current_stock', 'low_stock_threshold', 'created_at', 'updated_at',
                  'server_updated_at', 'version', 'deleted_at']
        read_only_fields = ['current_stock', 'server_updated_at', 'version']
```

18.7 Dépendances proposées

Paquet	Usage	Statut
django, djangorestframework	Cœur du backend	Imposé
djangorestframework-simplejwt	JWT, liste noire	Imposé (JWT)
django-cors-headers	CORS	Imposé
psycopg (v3)	Pilote PostgreSQL	Recommandé
argon2-cffi	Hachage Argon2	Recommandé
django-filter	Filtres (période, devise, statut)	Optionnel
drf-spectacular	Schéma OpenAPI et documentation de l'API	Optionnel
gunicorn, whitenoise	Serveur d'application, fichiers statiques de l'admin	Déploiement
pytest-django, factory-boy	Tests	Recommandé

Tableau 18.3 : Paquets Python (versions à figer au démarrage ; Django 5.2 LTS conseillé)

19. Correspondance avec l'API REST

19.1 Conventions

- **Préfixe et version** : */api/v1/* ; un changement incompatible crée */api/v2/*. **Format** : JSON UTF-8 ; dates ISO 8601 en UTC ; montants en chaînes décimales ; identifiants en UUID.
- **Ressources au pluriel anglais** (/products/, /sales/) ; actions non CRUD exprimées en sous-ressource verbe (/sales/{id}/cancel/).
- **Pagination** par curseur sur les listes volumineuses (ventes, mouvements) ; **filtres** par paramètres (?category=, ?search=, ?low=true, ?from=&to=, ?status=).
- **Idempotence** : les créations acceptent l'UUID fourni par le client ; renvoyer la même requête ne duplique rien (200 avec la ressource existante).
- **Limitation de débit** : 429 avec en-tête Retry-After ; le client applique l'attente exponentielle.

19.2 Verbes HTTP

Verbe	Rôle	Sûr	Idempotent	Usage dans Doku
GET	Lire	Oui	Oui	Lister, consulter, pull de synchronisation, statistiques
POST	Créer ou déclencher une action	Non	Non par défaut (idempotent ici grâce à l'UUID)	Créer produit, vente, mouvement ; /auth/login/ ; /sync/push/ ; /sales/{id}/cancel/
PUT	Remplacer entièrement	Non	Oui	Non utilisé (PATCH préféré : l'application mobile n'envoie que ce qui change)
PATCH	Modifier partiellement	Non	Oui (en pratique)	Modifier un produit, une catégorie, le profil, la boutique
DELETE	Supprimer	Non	Oui	**Archiver** (suppression logique) produit ou catégorie ; jamais sur les ventes ni les mouvements

Tableau 19.1 : Verbes HTTP et usage

19.3 Catalogue des routes

Les routes proposées par le porteur du projet sont toutes traitées : **/auth/, /users/, /products/, /categories/, /sales/, /stock/, /sync/** sont implémentées ; **/receipts/** devient */sales/{id}/receipt/* (le reçu n'est pas une ressource, chapitre 8) ; **/customers/** n'est **pas** implémentée (pas d'entité Client en MVP, H-08). Ont été ajoutées : /shops/, /currencies/, /exchange-rates/, /stats/ et /health/, nécessaires aux cas d'utilisation.

Méthode	URL (préfixe /api/v1)	Description	Auth.	UC
POST	`/auth/register/`	Créer un compte (utilisateur + boutique)	Public	01
POST	`/auth/login/`	Connexion ; retourne l'access token, pose le cookie refresh	Public	02
POST	`/auth/refresh/`	Renouveler l'access token (rotation du refresh)	Cookie refresh	02
POST	`/auth/logout/`	Invalider le refresh token	JWT	03
POST	`/auth/password/change/`	Changer le mot de passe	JWT	04
GET	`/users/me/`	Lire le profil	JWT	04
PATCH	`/users/me/`	Modifier le profil	JWT	04
GET	`/shops/me/`	Lire la boutique	JWT	04
PATCH	`/shops/me/`	Modifier la boutique	JWT	04
GET	`/currencies/`	Lister les devises (lecture seule)	JWT	05, 21
GET	`/exchange-rates/`	Historique des taux	JWT	05
POST	`/exchange-rates/`	Ajouter un taux (ajout seul)	JWT	05

Méthode	URL (préfixe /api/v1)	Description	Auth.	UC
GET	/categories/	Lister les catégories	JWT	10
POST	/categories/	Créer une catégorie	JWT	10
GET	/categories/{id}/	Lire une catégorie	JWT	10
PATCH	/categories/{id}/	Modifier une catégorie	JWT	10
DELETE	/categories/{id}/	Archiver une catégorie	JWT	10
GET	/products/	Lister / rechercher / filtrer les produits	JWT	09
POST	/products/	Créer un produit (+ stock initial)	JWT	06, 11
GET	/products/{id}/	Lire un produit	JWT	09
PATCH	/products/{id}/	Modifier un produit	JWT	07
DELETE	/products/{id}/	Archiver un produit	JWT	08
GET	/stock/	Niveaux de stock (filtre low=true)	JWT	12, 16
GET	/stock/movements/	Journal des mouvements	JWT	13
POST	/stock/movements/	Réapprovisionnement ou ajustement	JWT	11, 14, 15
GET	/sales/	Historique des ventes	JWT	23
POST	/sales/	Enregistrer une vente (mode en ligne)	JWT	18
GET	/sales/{id}/	Détail d'une vente	JWT	23
POST	/sales/{id}/cancel/	Annuler une vente	JWT	22
GET	/sales/{id}/receipt/	Données structurées du reçu	JWT	27
GET	/stats/summary/	CA, coût, bénéfice, marge sur une période	JWT	24, 25, 26
GET	/stats/daily/	Série quotidienne	JWT	24
GET	/stats/top-products/	Produits les plus vendus	JWT	24
POST	/sync/push/	Envoyer un lot de changements locaux	JWT	18, 29, 30, 31
GET	/sync/pull/	Récupérer les changements depuis un curseur	JWT	30
GET	/health/	Test de connectivité (réponse minimale)	Public	29, 30

Tableau 19.2 : Routes de l'API (UC = numéros des cas d'utilisation servis)

19.4 Exemples

Création d'un produit (en ligne)

```
POST /api/v1/products/           Authorization: Bearer <access>
{ "id": "e5a7c9b1-6d3f-4a28-8c14-5f0b2e9d7a36", "name": "Sucre 1 kg", "category_id": null,
  "purchase_price": "5000.00", "purchase_currency_code": "CDF",
  "selling_price": "7000.00",  "selling_currency_code": "CDF",
  "low_stock_threshold": 5, "initial_stock": 20,
  "created_at": "2026-10-04T08:00:00Z", "updated_at": "2026-10-04T08:00:00Z" }

201 Created  { ... "current_stock": 20, "version": 1, "server_updated_at": "2026-10-04T08:00:02Z" }
```

Pull de synchronisation

```
GET /api/v1/sync/pull/?since=2026-10-04T08:00:02.000Z&limit=200
200 OK
{ "cursor": "2026-10-04T10:16:03.512Z", "has_more": false,
  "changes": { "products": [ {...} ], "categories": [], "exchange_rates": [],
               "stock_movements": [ {...} ], "sales": [ {..., "items": [ {...} ]} ] } }
```

Format d'erreur unique

```
400 Bad Request
{ "error": { "code": "VALIDATION_ERROR", "message": "Données invalides.",
             "details": { "quantity": ["Doit être supérieur à 0."] } } }
```

Statut	Signification	Cas
200 / 201 / 204	Succès, création, sans contenu	Lecture, création, archivage ou déconnexion
400	Validation	Champ invalide, UUID mal formé, total incohérent (code TOTAL_MISMATCH dans le détail)
401	Non authentifié	Jeton absent, expiré ou révoqué
403	Interdit	Ressource d'une autre boutique (sans révéler son existence)
404	Introuvable	Identifiant inconnu *dans la boutique de l'utilisateur*
409	Conflit	Version dépassée (modification concurrente d'un produit)
429	Trop de requêtes	Limitation de débit atteinte (Retry-After)
5xx	Erreur serveur	Réessayable par le moteur de synchronisation

Tableau 19.3 : Codes de statut utilisés

Bonne pratique : contrat d'API
Générer le schéma OpenAPI avec **drf-spectacular** et le consommer côté Next.js pour produire les types TypeScript : le contrat d'API devient exécutable et les écarts entre le client et le serveur sont détectés à la compilation.

20. Vérification de cohérence globale

20.1 Méthode

La chaîne vérifiée est : **besoin → cas d'utilisation → activité / séquence → MCD → MLD → classes → API**. Les éléments du dossier (cas d'utilisation, tables, routes, besoins) sont décrits par des structures de données uniques dont sont générés les tableaux, les diagrammes de données et le dictionnaire ; les contrôles ci-dessous sont **exécutés par programme** à chaque génération du document. Ils ne remplacent pas la relecture humaine (section 20.4) mais garantissent l'absence de rupture mécanique.

20.2 Contrôles automatiques

	Contrôle	Détail
✓	MCD ↔ MLD : chaque clé étrangère du MLD provient d'une association du MCD	18 associations ; 0 FK manquante ; FK technique hors MCD : stock_movement.shop_id (tenant, dénormalisation assumée)
✓	UC → tables : toute table citée par un cas d'utilisation existe dans le MLD	65 références vérifiées
✓	UC → API : toute route citée par un cas d'utilisation existe dans le catalogue	44 références vérifiées
✓	API → UC : toute route sert au moins un cas d'utilisation existant	36 routes
✓	Table → fonctionnalité : chaque table du MLD est utilisée par un cas d'utilisation ou une route	10 tables ; non utilisées : aucune
✓	Besoin → UC : chaque cas d'utilisation répond à un besoin fonctionnel	12 besoins, 31 cas ; sans besoin : aucun
✓	BF → UC : tous les cas cités par les besoins existent	
✓	Diagrammes : les 5 activités, 5 séquences et 2 états sont chacun rattachés à au moins un cas d'utilisation	['A1', 'A2', 'A3', 'A4', 'A5'] · ['S1', 'S2', 'S3', 'S4', 'S5'] · ['ST1', 'ST2']
✓	UC sans diagramme dédié (consultation ou CRUD simple, décrits par leur fiche)	12 cas : UC04, UC05, UC07, UC08, UC09, UC10, UC13, UC15, UC23, UC24, UC25, UC26
✓	Chaque clé étrangère a une règle ON DELETE explicite	19 FK

Tableau 20.1 : Résultat des contrôles de cohérence (= conforme)

20.3 Matrices de traçabilité

La matrice des cas d'utilisation relie chaque cas à ses diagrammes, aux tables qu'il lit (R) ou écrit (W), aux routes d'API et à sa disponibilité hors ligne. La seconde matrice part des tables : à quels cas d'utilisation et à quelles routes répond chacune ?

UC	Cas d'utilisation	Act.	Séq.	État	Tables (R / W)	Routes API	Hors ligne
UC01	Créer un compte	—	S4	—	app_user W, shop W	`POST /auth/register/`	Non
UC02	Se connecter	—	S4	—	app_user R	`POST /auth/login/` `POST /auth/refresh/`	Partiel
UC03	Se déconnecter	—	S4	—	—	`POST /auth/logout/`	Partiel

UC04	Gérer son profil	—	—	—	app_user RW, shop RW	`GET /users/me/`
						`PATCH /users/me/`
						`GET /shops/me/`
						`PATCH /shops/me/`
						`POST /auth/password/change/`
							Lecture seule

UC05	Gérer le taux de change	—	—	—	exchange_rate W, currency R	`GET /exchange-rates/`
						`POST /exchange-rates/`
						`GET /currencies/`
							Oui

UC06	Ajouter un produit	A4	—	—	product W, category R, currency R	`POST /products/`	Oui
UC07	Modifier un produit	—	—	—	product RW	`PATCH /products/{id}/`	Oui
UC08	Supprimer (archiver) un produit	—	—	—	product W	`DELETE /products/{id}/`	Oui
UC09	Consulter les produits	—	—	—	product R, category R	`GET /products/` `GET /products/{id}/`	Oui

UC10	Gérer les catégories	—	—	—	category RW	`GET /categories/`
						`POST /categories/`
						`GET /categories/{id}/`
						`PATCH /categories/{id}/`
						`DELETE /categories/{id}/`
							Oui

UC11	Définir le stock initial	A4	—	—	stock_movement W, product W	`POST /stock/movements/`	Oui
UC12	Consulter le stock	A4	—	—	product R	`GET /stock/`	Oui
UC13	Consulter les mouvements de stock	—	—	—	stock_movement R, product R	`GET /stock/movements/`	Oui
UC14	Réapprovisionner un produit	A4	—	—	stock_movement W, product W	`POST /stock/movements/`	Oui
UC15	Ajuster le stock	—	—	—	stock_movement W, product W	`POST /stock/movements/`	Oui
UC16	Signaler un stock faible	A1 A4	—	—	product R	`GET /stock/?low=true`	Oui
UC17	Enregistrer les mouvements de stock	A1 A4	—	—	stock_movement W, product W	`POST /stock/movements/` `POST /sales/`	Oui
UC18	Enregistrer une vente	A1 A2	S1 S2	ST1	sale W, sale_item W, stock_movement W, product RW, exchange_rate R, currency R	`POST /sales/` `POST /sync/push/`	Oui
UC19	Ajouter des produits à la vente	A1	—	—	product R	— (local)	Oui
UC20	Calculer automatiquement le total	A1	—	—	product R, exchange_rate R, currency R	— (local)	Oui
UC21	Choisir la devise (CDF ou USD)	A1	—	—	currency R, exchange_rate R	— (local)	Oui
UC22	Annuler une vente	—	—	ST1	sale W, stock_movement W, product W	`POST /sales/{id}/cancel/`	Oui

UC	Cas d'utilisation	Act.	Séq.	État	Tables (R / W)	Routes API	Hors ligne
UC23	Consulter l'historique des ventes	—	—	—	sale R, sale_item R	`GET /sales/` `GET /sales/{id}/`	Oui

UC24	Consulter les statistiques	—	—	—	sale R, sale_item R, product R, exchange_rate R	`GET /stats/summary/`
						`GET /stats/daily/`
						`GET /stats/top-products/`
							Oui

UC25	Consulter le chiffre d'affaires	—	—	—	sale R, sale_item R, exchange_rate R	`GET /stats/summary/`	Oui
UC26	Consulter le bénéfice	—	—	—	sale R, sale_item R, exchange_rate R	`GET /stats/summary/`	Oui
UC27	Générer un reçu	A1 A5	S5	—	sale R, sale_item R, shop R	`GET /sales/{id}/receipt/`	Oui
UC28	Partager un reçu via WhatsApp	A5	S5	—	—	— (local)	Partiel
UC29	Travailler hors ligne	A2	S2	—	—	— (local)	Oui

UC30	Synchroniser les données	A3	S3	ST2	sync_session W, product RW, category RW, stock_movement RW, sale RW, sale_item RW, exchange_rate RW	`POST /sync/push/`
						`GET /sync/pull/`
						`GET /health/`
							Non (requiert réseau)

UC31	Résoudre un conflit de synchronisation	A3	S3	ST2	sync_session W	— (local)	Non (requiert réseau)

Tableau 20.2 : Matrice cas d'utilisation × diagrammes × tables × API

Table	Entité MCD	Classe UML / modèle Django	Cas d'utilisation (écriture)	Cas d'utilisation (lecture)	Routes
currency	DEVISE	core.Currency	—	05, 06, 18, 20, 21	1
app_user	UTILISATEUR	accounts.User	01, 04	02	5
shop	BOUTIQUE	accounts.Shop	01, 04	27	4
category	CATEGORIE	catalog.Category	10, 30	06, 09	5
product	PRODUIT	catalog.Product	06, 07, 08, 11, 14, 15, 17, 18, 22, 30	09, 12, 13, 16, 19, 20, 24	7
exchange_rate	TAUX_CHANGE	core.ExchangeRate	05, 30	18, 20, 21, 24, 25, 26	2
sale	VENTE	sales.Sale	18, 22, 30	23, 24, 25, 26, 27	8
sale_item	LIGNE_VENTE	sales.SaleItem	18, 30	23, 24, 25, 26, 27	7
stock_movement	MOUVEMENT_STOCK	inventory.StockMovement	11, 14, 15, 17, 18, 22, 30	13	5
sync_session	SYNCHRONISATION	sync.SyncSession	30, 31	—	1

Tableau 20.3 : Matrice tables × cas d'utilisation (numéros d'UC) × routes

20.4 Points d'attention relevés

La vérification a fait ressortir les écarts suivants. Aucun n'est une erreur de conception ; ce sont des choix à connaître ou des décisions à valider.

Type	Constat	Traitement
Attention	UC05, UC11, UC14, UC15, UC16, UC17, UC22, UC31 ne figuraient pas dans la liste initiale	Ajoutés car induits par les activités demandées ou par le bi-devise ; à valider (chapitre 4).
Attention	Hypothèses H-01 à H-19 non encore validées	Risque de reprise du modèle si une hypothèse structurante est fausse (H-01, H-02, H-04, H-06, H-11).
Attention	UC04 « Gérer son profil » : lecture seule hors ligne	Limite assumée : la modification exige une connexion (utilisateur et boutique sont l'autorité du serveur).
Attention	Statistiques : deux implémentations (client IndexedDB, serveur SQL)	Risque de divergence ; atténué par des règles uniques (chapitre 12) et un jeu de tests de parité.
Information	Aucune table pour UC27 « Générer un reçu » ni UC28 « Partager via WhatsApp »	Voulu : le reçu est un document dérivé de la vente (décision D7, chapitre 8).
Information	UC29 « Travailler hors ligne » n'a aucune table serveur	Il est réalisé par la base locale (chapitre 14) ; absence normale dans le MLD serveur.
Information	Le statut de synchronisation est absent du MLD	Voulu : il n'existe que côté client (chapitre 14) ; le serveur journalise les lots (sync_session).
Information	Routes /customers/ non implémentée et /receipts/ transformée	Conséquence directe de l'absence d'entités Client et Reçu (chapitres 8 et 19).
Information	stock_movement.shop_id est une clé étrangère hors MCD	Colonne de cloisonnement dénormalisée (chapitre 9) ; contrôlée par le service.
Information	Sale porte sold_at et non created_at	Les classes abstraites ne couvrent donc pas toutes les tables de la même façon (chapitre 18).
Information	La table sync_session n'a pas de cas d'utilisation visible par le commerçant	Justifiée : support et supervision ; rattachée à UC30 et UC31.
Information	Première connexion et création de compte impossibles hors ligne	Inhérent à l'authentification serveur ; documenté au chapitre 14.

Tableau 20.4 : Écarts, limites et décisions à valider

Conclusion de la vérification

Résultat : **aucune fonctionnalité sans donnée, aucune table sans fonctionnalité, aucune route sans cas d'utilisation** ; les écarts restants sont des choix assumés ou des hypothèses à valider. Le modèle est cohérent de bout en bout : *cas d'utilisation → activités et séquences → MCD → MLD → classes → modèles Django → routes → écrans.*

21. Conclusion et étapes suivantes

Ce dossier fournit une modélisation complète, cohérente et directement exploitable de Doku : 31 cas d'utilisation, 5 activités, 5 séquences, 2 diagrammes d'états, 1 diagramme de classes, 1 MCD (10 entités, 18 associations), 1 MLD (10 tables), un dictionnaire de 93 champs, et les règles de synchronisation, de sécurité, de correspondance Django et API.

21.1 Feuille de route proposée

Étape	Objectif	Livrables
0. Validation	Valider les hypothèses H-01 à H-19 et les cas ajoutés ; choisir hébergeur et base	Hypothèses signées, périmètre MVP figé
1. Maquettes	Dessiner les parcours : caisse, produit, stock, taux, historique, reçu	Maquettes testées auprès de vrais commerçants
2. Sprint 0	Dépôt Git (branches, conventions de commits), intégration continue, projets Django et Next.js vides, base PostgreSQL, tests d'isolation	Squelette déployable
3. Sprint 1 : « tranche verticale »	Compte + connexion JWT, produits, base locale, **une vente hors ligne synchronisée de bout en bout**	Premier prototype de synchronisation (le plus gros risque, à lever tôt)
4. Sprint 2	Caisse complète (devises, taux, panier multi-produits), stock et mouvements, alertes	Caisse utilisable
5. Sprint 3	Historique, annulation, reçus et partage WhatsApp, statistiques	MVP fonctionnel
6. Sprint 4	Durcissement : sécurité, performance sur téléphone d'entrée de gamme, supervision, pilote auprès de commerçants	MVP en pilote

Tableau 21.1 : Roadmap indicative (à affiner avec le backlog produit)

> **Conseil de mentor**
> Commencer par la **tranche verticale** la plus risquée : une vente créée sans réseau, synchronisée, visible sur un second appareil, avec un stock correct. Si cette tranche fonctionne, le reste du projet est du développement ordinaire. Tant qu'elle n'est pas prouvée, tout le reste est une hypothèse.

Annexe A : glossaire

Terme	Définition
Access token / refresh token	Jetons JWT : le premier, court, autorise les appels ; le second, long, sert à en obtenir un nouveau.
Append-only (ajout seul)	Table où l'on n'insère que ; une correction est une nouvelle ligne.
Backoff exponentiel / jitter	Attente croissante entre deux tentatives, avec une part d'aléa pour éviter que tous les appareils réessaient ensemble.
CORS	Mécanisme du navigateur qui décide si une page d'un domaine peut appeler l'API d'un autre.
CSRF	Attaque où un site tiers fait exécuter une action au navigateur d'un utilisateur connecté.
Curseur (cursor)	Repère (ici un horodatage serveur) indiquant jusqu'où un client a déjà reçu les changements.
Idempotence	Propriété d'une opération qui, répétée, a le même effet qu'une seule fois.
IndexedDB	Base de données intégrée au navigateur, asynchrone et transactionnelle, pour les gros volumes locaux.
JWT	JSON Web Token : jeton signé contenant l'identité et l'expiration (RFC 7519).
Ledger (journal)	Registre d'écritures jamais modifiées dont la somme donne l'état courant.
Last-write-wins (LWW)	Règle de résolution : la dernière modification l'emporte.
MCD / MLD / MPD	Modèles conceptuel, logique et physique de données (Merise).
Offline-first	Conception où l'usage hors ligne est le cas nominal, la connexion une optimisation.
PWA	Progressive Web App : application web installable avec Service Worker et stockage local.
Service Worker	Script du navigateur qui s'exécute en arrière-plan et met en cache les ressources de l'application.
Snapshot (instantané)	Valeur copiée au moment d'un événement pour ne plus dépendre de la source.
Soft delete (suppression logique)	Marquer un enregistrement comme supprimé au lieu de l'effacer.
Tenant (locataire)	Ici, la boutique : périmètre d'isolation des données.
Tombstone (pierre tombale)	Marque de suppression qui permet de propager l'effacement aux autres appareils.
UUID	Identifiant universel de 128 bits, générable sans coordination (RFC 9562).
wa.me	Domaine des liens « Click to Chat » de WhatsApp qui ouvrent une conversation avec un message prérempli.

Annexe B : MPD, script SQL de référence

Script PostgreSQL **généré à partir du MLD** puis vérifié par l'analyseur syntaxique de PostgreSQL. Il sert de référence physique ; en pratique le schéma sera créé par les migrations Django (la correspondance des types et des contraintes est donnée au chapitre 18).

```
-- MPD de référence : PostgreSQL 14+. La source de vérité du schéma restera les migrations Django.

CREATE TABLE currency (
  code CHAR(3) NOT NULL,
  name VARCHAR(50) NOT NULL,
  symbol VARCHAR(5) NOT NULL,
  decimal_places SMALLINT NOT NULL,
  CONSTRAINT pk_currency PRIMARY KEY (code),
  CONSTRAINT ck_currency_1 CHECK (decimal_places BETWEEN 0 AND 3)
);

CREATE TABLE app_user (
  id UUID NOT NULL,
  phone VARCHAR(20) NOT NULL,
  email VARCHAR(254),
  password VARCHAR(128) NOT NULL,
  full_name VARCHAR(150) NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  is_staff BOOLEAN NOT NULL DEFAULT FALSE,
  is_superuser BOOLEAN NOT NULL DEFAULT FALSE,
  last_login TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL,
  CONSTRAINT pk_app_user PRIMARY KEY (id),
  CONSTRAINT uq_app_user_phone UNIQUE (phone),
  CONSTRAINT uq_app_user_email UNIQUE (email)
);

CREATE TABLE shop (
  id UUID NOT NULL,
  owner_id UUID NOT NULL,
  name VARCHAR(150) NOT NULL,
  address VARCHAR(255),
  phone VARCHAR(20),
  reference_currency_code CHAR(3) NOT NULL DEFAULT 'CDF',
  created_at TIMESTAMPTZ NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL,
  CONSTRAINT pk_shop PRIMARY KEY (id),
  CONSTRAINT fk_shop_owner_id FOREIGN KEY (owner_id) REFERENCES app_user(id) ON DELETE RESTRICT,
  CONSTRAINT fk_shop_reference_currency_code FOREIGN KEY (reference_currency_code) REFERENCES currency(code) ON DELETE RESTRICT,
  CONSTRAINT uq_shop_owner_id UNIQUE (owner_id)
);

CREATE TABLE category (
  id UUID NOT NULL,
  shop_id UUID NOT NULL,
  name VARCHAR(100) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL,
  server_updated_at TIMESTAMPTZ NOT NULL,
  version INTEGER NOT NULL DEFAULT 1,
  deleted_at TIMESTAMPTZ,
  CONSTRAINT pk_category PRIMARY KEY (id),
  CONSTRAINT fk_category_shop_id FOREIGN KEY (shop_id) REFERENCES shop(id) ON DELETE RESTRICT,
  CONSTRAINT ck_category_1 CHECK (version >= 1)
);
CREATE INDEX idx_category_1 ON category (shop_id, server_updated_at);
CREATE INDEX idx_category_2 ON category (shop_id, deleted_at);

CREATE TABLE product (
  id UUID NOT NULL,
  shop_id UUID NOT NULL,
  category_id UUID,
  name VARCHAR(150) NOT NULL,
  purchase_price NUMERIC(14,2) NOT NULL,
  purchase_currency_code CHAR(3) NOT NULL,
  selling_price NUMERIC(14,2) NOT NULL,
  selling_currency_code CHAR(3) NOT NULL,
  current_stock INTEGER NOT NULL DEFAULT 0,
  low_stock_threshold INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL,
  server_updated_at TIMESTAMPTZ NOT NULL,
  version INTEGER NOT NULL DEFAULT 1,
  deleted_at TIMESTAMPTZ,
  CONSTRAINT pk_product PRIMARY KEY (id),
  CONSTRAINT fk_product_shop_id FOREIGN KEY (shop_id) REFERENCES shop(id) ON DELETE RESTRICT,
  CONSTRAINT fk_product_category_id FOREIGN KEY (category_id) REFERENCES category(id) ON DELETE SET NULL,
```

```
  CONSTRAINT fk_product_purchase_currency_code FOREIGN KEY (purchase_currency_code) REFERENCES currency(code) ON DELETE RESTRICT,
  CONSTRAINT fk_product_selling_currency_code FOREIGN KEY (selling_currency_code) REFERENCES currency(code) ON DELETE RESTRICT,
  CONSTRAINT ck_product_1 CHECK (purchase_price >= 0 AND selling_price >= 0),
  CONSTRAINT ck_product_2 CHECK (low_stock_threshold >= 0),
  CONSTRAINT ck_product_3 CHECK (version >= 1)
);
CREATE INDEX idx_product_1 ON product (shop_id, deleted_at, name);
CREATE INDEX idx_product_2 ON product (shop_id, category_id);
CREATE INDEX idx_product_3 ON product (shop_id, server_updated_at);

CREATE TABLE exchange_rate (
  id UUID NOT NULL,
  shop_id UUID NOT NULL,
  base_currency_code CHAR(3) NOT NULL,
  quote_currency_code CHAR(3) NOT NULL,
  rate NUMERIC(18,6) NOT NULL,
  effective_at TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL,
  server_updated_at TIMESTAMPTZ NOT NULL,
  CONSTRAINT pk_exchange_rate PRIMARY KEY (id),
  CONSTRAINT fk_exchange_rate_shop_id FOREIGN KEY (shop_id) REFERENCES shop(id) ON DELETE RESTRICT,
  CONSTRAINT fk_exchange_rate_base_currency_code FOREIGN KEY (base_currency_code) REFERENCES currency(code) ON DELETE RESTRICT,
  CONSTRAINT fk_exchange_rate_quote_currency_code FOREIGN KEY (quote_currency_code) REFERENCES currency(code) ON DELETE RESTRICT,
  CONSTRAINT ck_exchange_rate_1 CHECK (rate > 0),
  CONSTRAINT ck_exchange_rate_2 CHECK (base_currency_code <> quote_currency_code)
);
CREATE INDEX idx_exchange_rate_1 ON exchange_rate (shop_id, base_currency_code, quote_currency_code, effective_at DESC);
CREATE INDEX idx_exchange_rate_2 ON exchange_rate (shop_id, server_updated_at);

CREATE TABLE sale (
  id UUID NOT NULL,
  shop_id UUID NOT NULL,
  exchange_rate_id UUID NOT NULL,
  currency_code CHAR(3) NOT NULL,
  receipt_number VARCHAR(16) NOT NULL,
  sold_at TIMESTAMPTZ NOT NULL,
  status VARCHAR(10) NOT NULL DEFAULT 'COMPLETED',
  total_amount NUMERIC(14,2) NOT NULL,
  cancelled_at TIMESTAMPTZ,
  cancel_reason VARCHAR(255),
  server_updated_at TIMESTAMPTZ NOT NULL,
  CONSTRAINT pk_sale PRIMARY KEY (id),
  CONSTRAINT fk_sale_shop_id FOREIGN KEY (shop_id) REFERENCES shop(id) ON DELETE RESTRICT,
  CONSTRAINT fk_sale_exchange_rate_id FOREIGN KEY (exchange_rate_id) REFERENCES exchange_rate(id) ON DELETE RESTRICT,
  CONSTRAINT fk_sale_currency_code FOREIGN KEY (currency_code) REFERENCES currency(code) ON DELETE RESTRICT,
  CONSTRAINT ck_sale_1 CHECK (status IN ('COMPLETED','CANCELLED')),
  CONSTRAINT ck_sale_2 CHECK (total_amount >= 0),
  CONSTRAINT ck_sale_3 CHECK ((status = 'CANCELLED') = (cancelled_at IS NOT NULL))
);
CREATE INDEX idx_sale_1 ON sale (shop_id, sold_at DESC);
CREATE INDEX idx_sale_2 ON sale (shop_id, receipt_number);
CREATE INDEX idx_sale_3 ON sale (shop_id, server_updated_at);

CREATE TABLE sale_item (
  id UUID NOT NULL,
  sale_id UUID NOT NULL,
  product_id UUID NOT NULL,
  product_name VARCHAR(150) NOT NULL,
  quantity INTEGER NOT NULL,
  unit_price NUMERIC(14,2) NOT NULL,
  unit_cost NUMERIC(14,2) NOT NULL,
  CONSTRAINT pk_sale_item PRIMARY KEY (id),
  CONSTRAINT fk_sale_item_sale_id FOREIGN KEY (sale_id) REFERENCES sale(id) ON DELETE CASCADE,
  CONSTRAINT fk_sale_item_product_id FOREIGN KEY (product_id) REFERENCES product(id) ON DELETE RESTRICT,
  CONSTRAINT ck_sale_item_1 CHECK (quantity > 0),
  CONSTRAINT ck_sale_item_2 CHECK (unit_price >= 0 AND unit_cost >= 0)
);
CREATE INDEX idx_sale_item_1 ON sale_item (sale_id);
CREATE INDEX idx_sale_item_2 ON sale_item (product_id);

CREATE TABLE stock_movement (
  id UUID NOT NULL,
  shop_id UUID NOT NULL,
  product_id UUID NOT NULL,
  sale_item_id UUID,
  movement_type VARCHAR(20) NOT NULL,
  quantity INTEGER NOT NULL,
  reason VARCHAR(255),
  created_at TIMESTAMPTZ NOT NULL,
  server_updated_at TIMESTAMPTZ NOT NULL,
  CONSTRAINT pk_stock_movement PRIMARY KEY (id),
  CONSTRAINT fk_stock_movement_shop_id FOREIGN KEY (shop_id) REFERENCES shop(id) ON DELETE RESTRICT,
  CONSTRAINT fk_stock_movement_product_id FOREIGN KEY (product_id) REFERENCES product(id) ON DELETE RESTRICT,
  CONSTRAINT fk_stock_movement_sale_item_id FOREIGN KEY (sale_item_id) REFERENCES sale_item(id) ON DELETE RESTRICT,
  CONSTRAINT ck_stock_movement_1 CHECK (movement_type IN ('INITIAL','RESTOCK','SALE','SALE_CANCEL','ADJUSTMENT')),
  CONSTRAINT ck_stock_movement_2 CHECK (quantity <> 0),
  CONSTRAINT ck_stock_movement_3 CHECK (movement_type <> 'SALE' OR quantity < 0),
  CONSTRAINT ck_stock_movement_4 CHECK (movement_type NOT IN ('INITIAL','RESTOCK','SALE_CANCEL') OR quantity > 0),
  CONSTRAINT ck_stock_movement_5 CHECK (movement_type <> 'ADJUSTMENT' OR reason IS NOT NULL),
  CONSTRAINT uq_stock_movement_sale_item_id_movement_type UNIQUE (sale_item_id, movement_type)
```

```
);
CREATE INDEX idx_stock_movement_1 ON stock_movement (product_id, created_at);
CREATE INDEX idx_stock_movement_2 ON stock_movement (shop_id, server_updated_at);

CREATE TABLE sync_session (
  id UUID NOT NULL,
  shop_id UUID NOT NULL,
  device_id UUID NOT NULL,
  started_at TIMESTAMPTZ NOT NULL,
  finished_at TIMESTAMPTZ,
  status VARCHAR(12) NOT NULL DEFAULT 'IN_PROGRESS',
  received_count INTEGER NOT NULL DEFAULT 0,
  accepted_count INTEGER NOT NULL DEFAULT 0,
  duplicate_count INTEGER NOT NULL DEFAULT 0,
  conflict_count INTEGER NOT NULL DEFAULT 0,
  rejected_count INTEGER NOT NULL DEFAULT 0,
  error_details JSONB,
  CONSTRAINT pk_sync_session PRIMARY KEY (id),
  CONSTRAINT fk_sync_session_shop_id FOREIGN KEY (shop_id) REFERENCES shop(id) ON DELETE RESTRICT,
  CONSTRAINT ck_sync_session_1 CHECK (status IN ('IN_PROGRESS','SUCCESS','PARTIAL','FAILED'))
);
CREATE INDEX idx_sync_session_1 ON sync_session (shop_id, started_at DESC);
```

Annexe C : références

- **RFC 9562** : Universally Unique IDentifiers (UUID), IETF, 2024 (remplace la RFC 4122). **RFC 7519** : JSON Web Token (JWT). **ISO 4217** : codes des monnaies. **UIT-T E.164** : plan de numérotation international.
- **WhatsApp** : documentation « Click to Chat » (liens wa.me) et documentation de la WhatsApp Business Platform (Cloud API), sur les sites de WhatsApp et de Meta for Developers. Tarification et règles de messages modèles à consulter à la date du projet.
- **Django** (docs.djangoproject.com) : notes de version (CheckConstraint, versions LTS), modèle utilisateur personnalisé. **Django REST framework** (django-rest-framework.org). **Simple JWT** (django-rest-framework-simplejwt.readthedocs.io). **django-cors-headers**.
- **MDN Web Docs** : IndexedDB API, Service Worker API, StorageManager.persist(), Web Locks API, Web Share API, crypto.randomUUID().
- **OWASP** : Application Security Verification Standard (ASVS) et Top 10, pour la liste de contrôle de sécurité.
- **Merise et UML** : cardinalités et règles de passage MCD → MLD ; UML 2.5 (diagrammes de cas d'utilisation, d'activité, de séquence, de classes, d'états, de composants, de déploiement).